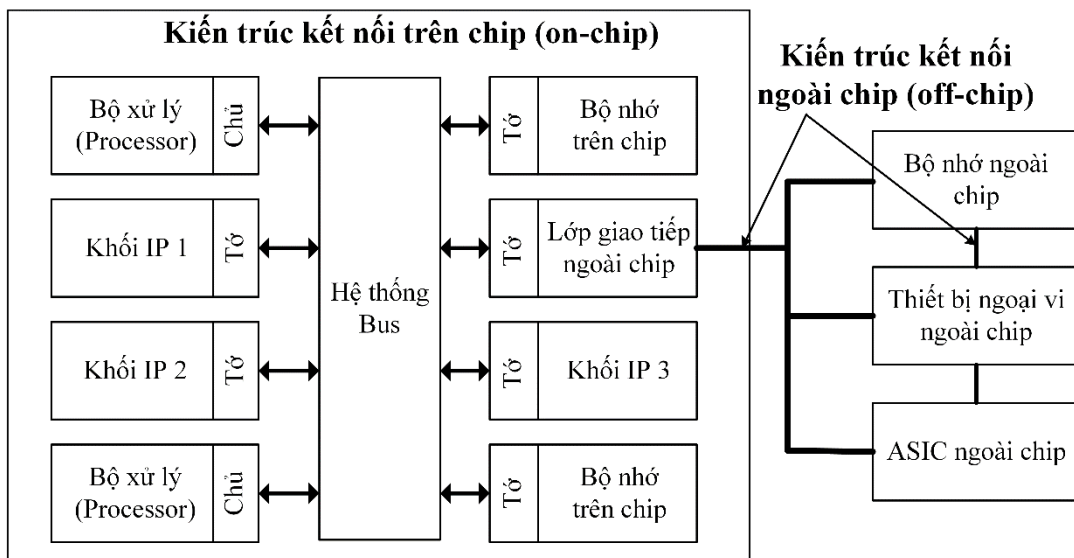


CHƯƠNG 4: HỆ THỐNG KẾT NỐI

4.1. Giới Thiệu Tổng Quan Kiến Trúc Kết Nối

Hệ thống kết nối trong SoC FPGA là mạng lưới các giao diện và bus kết nối các thành phần trong chip, bao gồm bộ vi xử lý, bộ nhớ, các khối logic và thiết bị ngoại vi. Hệ thống kết nối đóng vai trò quan trọng trong việc đảm bảo sự tương tác và giao tiếp mượt mà giữa các khối xử lý khác nhau, từ đó tối ưu hóa hiệu suất tổng thể của SoC. Nó giúp giảm thiểu độ trễ và tăng cường băng thông cho các ứng dụng yêu cầu truyền tải dữ liệu với tốc độ cao, như trong các thiết bị di động, viễn thông, và các hệ thống nhúng phức tạp. Việc thiết kế một hệ thống kết nối mạnh mẽ và linh hoạt là cần thiết để đảm bảo SoC hoạt động hiệu quả.



Hình 4.1: Sơ đồ mô tả kiến trúc kết nối trên chip và ngoài chip trong hệ thống SoC.

Hệ thống kết nối trong một SoC bao gồm các phân kết nối trên chip (on-chip) và kết nối ngoài chip (off-chip), đóng vai trò quan trọng trong việc truyền tải dữ liệu và điều khiển giữa các thành phần của hệ thống. Trong kiến trúc kết nối này, chủ (master) và tớ (slave) là các thuật ngữ được sử dụng để chỉ vai trò của các thành phần trong giao tiếp, trong đó thiết bị chủ điều khiển các giao dịch, còn thiết bị tớ thực hiện các yêu cầu từ chủ. Kết nối trên chip bao gồm các giao tiếp giữa bộ xử lý, bộ nhớ và các khối IP thông qua các bus nội bộ, trong khi kết nối ngoài chip đảm nhận việc giao tiếp với các thiết bị ngoại vi, bộ nhớ ngoài và các hệ thống khác ngoài chip. Kiến trúc kết nối trên chip, như thể hiện trong Hình 4.1, sử dụng một hệ thống bus để kết nối bộ xử lý (chủ), các khối IP và bộ nhớ trên chip (tớ). Bus trong kiến trúc kết nối này là hệ thống giao

tiếp, truyền tải dữ liệu, tín hiệu điều khiển và các thông tin giữa các thành phần, giúp điều phối các giao dịch giữa thiết bị chủ và thiết bị tớ. Trong đó, các bộ xử lý đóng vai trò là chủ, còn các khối IP và bộ nhớ là tớ. Hệ thống bus trên chip đảm nhiệm vai trò truyền tải dữ liệu và tín hiệu điều khiển giữa các thành phần này, tối ưu hóa khả năng giao tiếp và truyền tải dữ liệu. Ngoài ra, một lớp giao tiếp ngoài chip được sử dụng để kết nối các thành phần trên chip với các thiết bị ngoại vi và bộ nhớ ngoài chip. Các kết nối ngoài chip này thường sử dụng các giao thức phức tạp hơn và có băng thông cao để đáp ứng yêu cầu truyền tải dữ liệu lớn giữa hệ thống SoC và các thiết bị ngoại vi. Hệ thống bus và các lớp giao tiếp ngoài chip giúp mở rộng khả năng kết nối và tương tác của hệ thống, cho phép kết nối với nhiều loại thiết bị ngoại vi và bộ nhớ khác nhau.

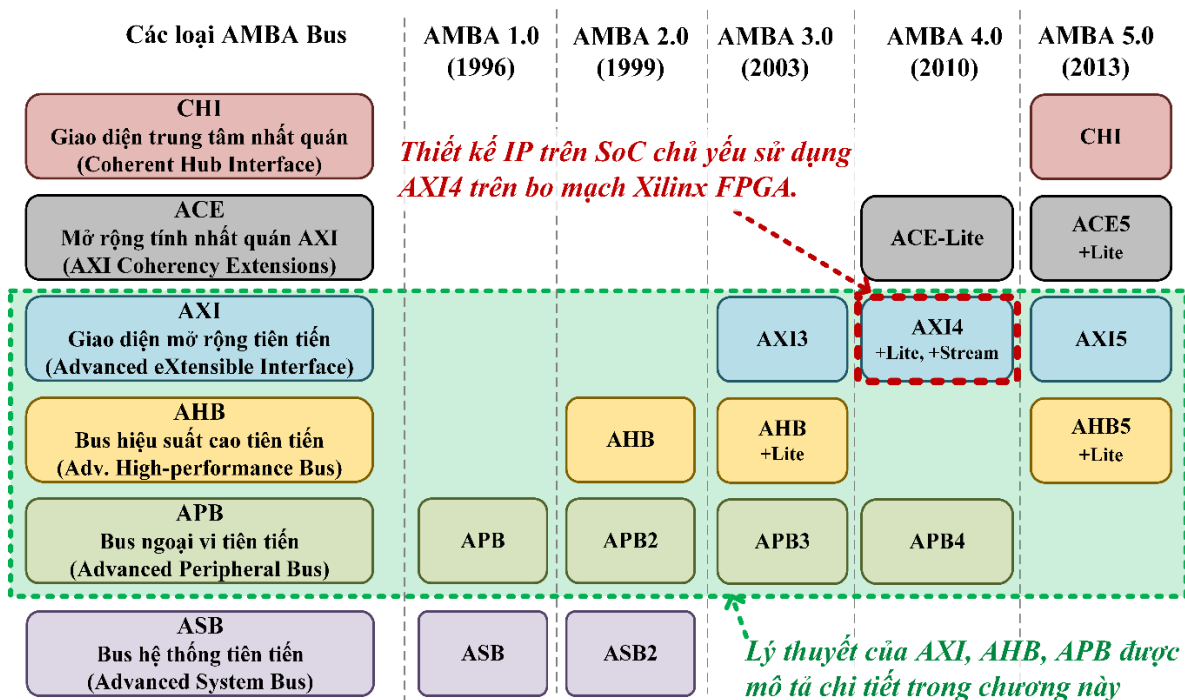
Mặc dù có nhiều tiêu chuẩn bus tồn tại, chúng chủ yếu tập trung vào kết nối ở mức bảng mạch và chưa chi phí phụ để hỗ trợ ghép kênh tín hiệu, phát hiện cấu hình, và xử lý các đặc tính điện, nhưng những chi phí này không áp dụng cho môi trường SoC. Đồng thời, cần có một tiêu chuẩn mở để cho phép tái sử dụng thiết kế tốt hơn và cho phép các nhà cung cấp IP phát triển các thiết bị ngoại vi cho nền tảng ARM. Kết quả là, ARM đã công bố đặc tả giao thức kiến trúc bus vi điều khiển nâng cao (AMBA) như một tiêu chuẩn mở (không tính phí bản quyền), và đây đã trở thành tiêu chuẩn giao diện bộ xử lý phổ biến nhất cho các bộ xử lý nhúng 32-bit. Do tính chất mở và đơn giản của nó, AMBA đã trở thành tiêu chuẩn thực tế cho giao diện bus trong kiến trúc hệ thống trên chip, với đặc tính chi phí thấp và độ trễ thấp, rất cần thiết cho các hệ thống nhúng tốc độ cao. Ngoài AMBA bus, Avalon Bus là một giao thức bus nổi bật trong các hệ thống SoC của Intel (trước đây là Altera), được thiết kế đặc biệt cho các hệ thống nhúng, giúp kết nối các bộ xử lý và các thiết bị ngoại vi một cách hiệu quả. Avalon Bus hỗ trợ các giao dịch giữa các thiết bị chủ và tớ thông qua các chu kỳ đồng bộ, tối ưu hóa việc truyền tải dữ liệu và điều phối các giao dịch, đồng thời duy trì hiệu suất cao trong các hệ thống yêu cầu băng thông lớn. Giao thức này đơn giản, dễ triển khai và được sử dụng rộng rãi trong các FPGA của Intel, đặc biệt là trong các dòng FPGA như Cyclone, Arria và Stratix, trở thành lựa chọn phổ biến cho các hệ thống SoC của Intel nhờ cung cấp một giải pháp bus đồng bộ hiệu quả.

Chương này sẽ giới thiệu các bus trong chuẩn AMBA dành cho các Xilinx FPGA, đồng thời cũng phân tích Avalon Bus trong các hệ thống SoC của Altera FPGA. AMBA

là một phần quan trọng trong việc kết nối các thiết bị và khối IP trong các hệ thống nhúng, đặc biệt trong các FPGA của Xilinx. Các giao thức này cung cấp giải pháp kết nối hiệu quả, hỗ trợ các giao dịch phức tạp với độ trễ thấp và băng thông cao. Cùng với đó, Avalon Bus, giao thức bus của Altera, được thiết kế đặc biệt để tối ưu hóa việc truyền tải dữ liệu và điều phối các giao dịch trong các hệ thống SoC, giúp kết nối các bộ xử lý và thiết bị ngoại vi một cách hiệu quả, đặc biệt là trong các dòng FPGA như Cyclone, Arria và Stratix. Cả hai giao thức AMBA và Avalon đều đóng vai trò quan trọng trong việc thiết kế các hệ thống SoC mạnh mẽ và linh hoạt, phục vụ cho các ứng dụng yêu cầu hiệu suất và khả năng mở rộng cao.

4.2. Giao thức AMBA

Giao thức AMBA là một tập hợp các giao thức interconnect dùng để giao tiếp giữa các khối hoặc IP khác nhau trong thiết kế SoC. AMBA được ARM giới thiệu vào năm 1996 và các đặc tả của nó được ARM cung cấp miễn phí như một tiêu chuẩn cho việc giao tiếp trên chip. Các giao tiếp trong AMBA dựa trên giao thức chủ-tớ.



Hình 4.2: Các phiên bản và đặc tả chính của kiến trúc bus AMBA qua các thế hệ.

Một SoC điển hình được sử dụng trong laptop hoặc máy tính để bàn có thể bao gồm nhiều thành phần như CPU, GPU, bộ nhớ, quản lý nguồn, âm thanh, video, DSP và các bộ điều khiển được tích hợp trên một chip duy nhất. Tiêu chuẩn AMBA cho phép giao

tiếp hiệu quả giữa các thành phần này, hỗ trợ phát triển thiết kế chính xác ngay từ đầu, thiết kế mô-đun, khả năng tái sử dụng, tương thích và khả năng mở rộng của một IP. Điều này giúp giảm thời gian đưa sản phẩm ra thị trường và tránh các thiết kế lại tốn kém. Ngoài ra, có rất nhiều giải pháp IP AMBA sẵn sàng sử dụng từ các nhà cung cấp IP như Synopsys và ARM. Các giải pháp xác minh IP (VIP) có tính linh hoạt và cấu hình cao rất hiệu quả cho việc xác minh chức năng toàn diện. Giải pháp Synopsys VC AutoTestbench cho phép tích hợp và cấu hình dễ dàng, nhanh chóng cho hàng trăm cổng AMBA nhất quán và không nhất quán cũng như các VIP tương ứng.

Kể từ năm 1996, AMBA đã phát triển và hiện nay đã ở thế hệ thứ 5, như mô tả ở Hình 4.2. APB (Bus ngoại vi tiên tiến) và ASB (Bus hệ thống tiên tiến) là những giao thức bus đầu tiên của AMBA. Phiên bản AMBA 2 vào năm 1999 đã giới thiệu AHB (Bus hiệu suất cao tiên tiến). AMBA 3 được giới thiệu vào năm 2003, bao gồm giao thức AXI (Giao diện mở rộng tiên tiến). AMBA 4 đã giới thiệu giao thức ACE (Mở rộng tính nhất quán AXI) vào năm 2010 và AMBA 5 giới thiệu CHI (Giao diện trung tâm nhất quán) vào năm 2013.

Các Loại Bus trong AMBA:

- ✓ **ASB:** Hỗ trợ các tính năng cho các hệ thống hiệu suất cao như truyền dữ liệu theo gói (hay còn gọi là truyền theo Burst), hoạt động truyền theo đường ống và nhiều bus thiết bị chủ.
- ✓ **APB:** Giao thức băng thông thấp, tối ưu hóa cho công suất thấp và độ phức tạp thấp, dùng để hỗ trợ các thiết bị ngoại vi. APB thường được sử dụng làm giao diện giá rẻ cho các thiết bị ngoại vi không yêu cầu hiệu suất cao. Mỗi lần chuyển dữ liệu mất ít nhất 2 chu kỳ.
- ✓ **AHB:** Được thiết kế để kết nối các bộ xử lý nhúng, như lõi bộ xử lý ARM, với các thiết bị ngoại vi hiệu năng cao, bộ điều khiển truy cập trực tiếp bộ nhớ (DMA), các bộ điều khiển trên chip, và các giao diện. Đây là một bus tốc độ cao, băng thông cao với kiến trúc tách biệt các bus đọc, ghi và bus rộng. Một bus rộng 32-bit được đề xuất trong tiêu chuẩn và chiều rộng dữ liệu có thể mở rộng tới 1024 bit. Các giao dịch đồng thời nhiều chủ/tớ được hỗ trợ. Nó cũng hỗ trợ các giao dịch chế độ Burst và các giao dịch tách biệt. Tất cả các giao dịch trên AHB được tham chiếu tới một clock đơn lẻ, giúp thiết kế cấp hệ thống dễ hiểu.

- ✓ **AXI:** Giao thức interconnect điểm-điểm vượt qua những hạn chế của các giao thức bus chia sẻ. Giao thức này nhắm đến các hệ thống hiệu suất cao và tần số cao với các tính năng chính bao gồm hỗ trợ nhiều giao dịch chờ đồng thời và khả năng hoàn thành dữ liệu không theo thứ tự. Nó cho phép thực hiện các giao dịch dựa trên Burst với chỉ địa chỉ bắt đầu được phát hành, đồng thời hỗ trợ truyền dữ liệu không căn chỉnh bằng cách sử dụng strobes. Giao thức này còn cho phép các giao dịch đọc và ghi diễn ra đồng thời, cùng với việc sử dụng kết nối đường ống để hoạt động ở tốc độ cao.
- ✓ **ACE:** ACE mở rộng giao thức AXI4 với các bộ nhớ đệm phân cứng nhất quán. Giao thức ACE đảm bảo tất cả các chủ đều thấy dữ liệu chính xác cho bất kỳ vị trí địa chỉ nào của từ.
- ✓ **CHI:** CHI định nghĩa các giao diện cho kết nối các bộ xử lý hoàn toàn nhất quán. Đây là một giao thức liên lạc dạng gói với các lớp Protocol, Link và Network, hỗ trợ tần số cao và truyền dữ liệu không chặn giữa các bộ xử lý.

Chương này tập trung vào ba giao thức APB, AHB, AXI vì chúng là những giao thức phổ biến nhất trong thiết kế SoC trên FPGA. Các giao thức này thuộc kiến trúc AMBA của ARM, cung cấp giải pháp linh hoạt và tối ưu cho việc kết nối các thành phần IP trên cùng một chip, từ các bộ xử lý đến các thiết bị ngoại vi. APB được sử dụng cho các thiết bị ngoại vi yêu cầu băng thông thấp, trong khi AHB phù hợp cho các thành phần cần băng thông cao và độ trễ thấp như DMA và bộ nhớ. AXI mang lại hiệu suất cao với khả năng xử lý nhiều giao dịch đồng thời, làm cho nó lý tưởng cho các ứng dụng yêu cầu hiệu suất tính toán cao. Đối với các nhà thiết kế FPGA, hiểu rõ các giao thức này giúp tối ưu hóa hiệu năng và khả năng mở rộng của SoC.

AXI4 thường được sử dụng cho hầu hết các chuẩn IP trong thiết kế của Xilinx FPGA do khả năng cung cấp hiệu suất cao, linh hoạt và dễ tích hợp với các IP cores khác nhau. AXI4 hỗ trợ các tính năng như truyền dữ liệu theo Burst, nhiều giao dịch đồng thời, và hoàn thành dữ liệu không theo thứ tự, giúp tối ưu hóa hiệu năng cho các hệ thống tính toán phức tạp. Phần cuối chương này sẽ phân tích chi tiết các tín hiệu và thiết kế của AXI4, đặc biệt là cách thức mà AXI4 được triển khai chuyên biệt cho các Xilinx FPGA nhằm khai thác tối đa khả năng của nền tảng này, giúp các nhà thiết kế hiểu rõ hơn về cách tích hợp và tối ưu hóa các IP cores sử dụng AXI4 trên FPGA.

4.3. Hệ thống bus APB

4.3.1. Giới thiệu về hệ thống bus APB

APB là một bus đơn giản chủ yếu được sử dụng cho kết nối các thiết bị ngoại vi. APB được giới thiệu như một phần của đặc tả AMBA 2, và các chức năng của nó đã được mở rộng trong AMBA 3 và AMBA 4 để hỗ trợ các trạng thái chờ, phản hồi lỗi và các thuộc tính truyền dữ liệu bổ sung (bao gồm hỗ trợ TrustZone). Hầu hết các hệ thống APB đều có độ rộng bus 32-bit. Mặc dù giao thức bus không có giới hạn về độ rộng bus, thông lệ chung cho các hệ thống dựa trên Arm là sử dụng bus ngoại vi 32-bit.

Mặc dù có thể kết nối trực tiếp một thiết bị ngoại vi với AHB, việc tách kết nối ngoại vi bằng APB có nhiều ưu điểm:

- ✓ Nhiều thiết kế hệ thống SoC chứa một số lượng lớn các thiết bị ngoại vi. Nếu chúng được kết nối với bus hệ thống AHB, chúng có thể làm giảm tần số tối đa của hệ thống do độ rộng tín hiệu cao và logic giải mã địa chỉ phức tạp. Việc nhóm các kết nối ngoại vi trong APB có thể giảm thiểu tác động đến hiệu suất của AHB.
- ✓ Một hệ thống con ngoại vi có thể hoạt động ở một tần số xung nhịp khác, hoặc được tắt nguồn mà không ảnh hưởng đến AHB.
- ✓ Các giao diện APB sử dụng một giao thức bus đơn giản hơn, giúp đơn giản hóa thiết kế ngoại vi cũng như giảm công sức xác minh.
- ✓ Hầu hết các thiết bị ngoại vi được thiết kế cho các bộ xử lý truyền thống có thể dễ dàng kết nối với APB vì các giao dịch APB không hoạt động theo kiểu đường ống.

4.3.2. Tín hiệu và kết nối APB

Bảng 4.1: Các tín hiệu điển hình của APB

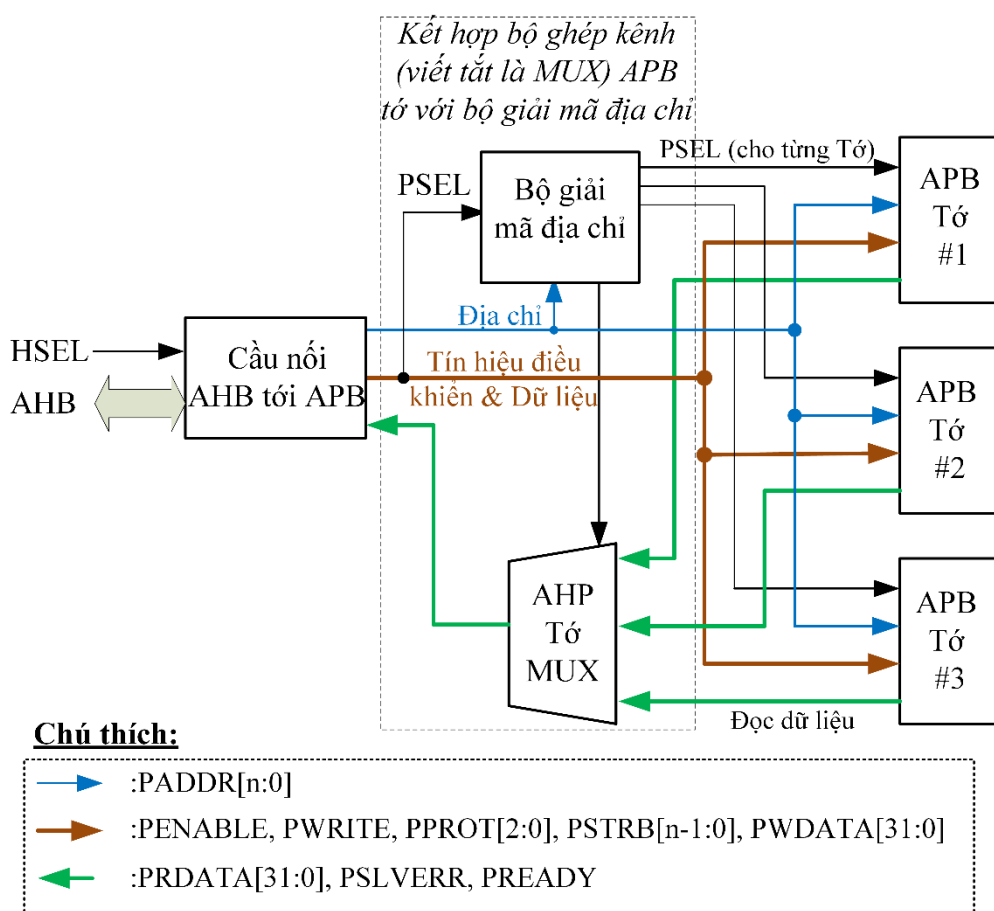
Tín hiệu	Chiều tín hiệu	Mô tả
PCLK	Nguồn xung nhịp → tất cả các khối APB	Tín hiệu xung nhịp chung
PRESETn	Nguồn reset → tất cả các khối APB	Tín hiệu reset mức thấp chung
PSEL	Bộ giải mã địa chỉ → Tổ	Chọn thiết bị tổ

PADDR[n:0]	Chủ → Tó	Bus địa chỉ (thường nhỏ hơn 32 bits)
PENABLE	Chủ → Tó	Kiểm soát truyền dữ liệu
PWRITE	Chủ → Tó	Kiểm soát Bảo vệ truyền dữ liệu
PPROT[2:0]	Chủ → Tó	Điều khiển việc bảo vệ truyền dữ liệu (chỉ có ở AMBA 4)
PSTRB[n-1:0]	Chủ → Tó	Tín hiệu byte strobe cho các hoạt động ghi (chỉ có ở AMBA 4)
PWDATA[31:0]	Chủ → Tó	Dữ liệu ghi
PRDATA[31:0]	Chủ → Tó	Dữ liệu đọc
PSLVERR	Tó → Chủ	Phản hồi từ Tó (từ AMBA 3 trở đi)
PREADY	Tó → Chủ	Tó sẵn sàng (từ AMBA 3 trở đi)

Trong một hệ thống APB điển hình, các tín hiệu sử dụng được mô tả trong Bảng 4.1. Cụ thể, một hệ thống APB hoạt động với một tín hiệu xung nhịp gọi là PCLK. Tín hiệu này được chia sẻ chung cho bus chủ (thường là một cầu nối AHB tới APB), bus tớ, và các khối hạ tầng bus. Tất cả các thanh ghi trên APB sẽ kích hoạt ở các cạnh lên của PCLK. Cũng có một tín hiệu reset mức thấp gọi là PRESETn. Khi tín hiệu này ở mức thấp, nó sẽ reset hệ thống APB ngay lập tức (reset không đồng bộ). Điều này cho phép hệ thống được reset ngay cả khi xung nhịp bị dừng. Tín hiệu PRESETn cần được đồng bộ hóa với PCLK để tránh các điều kiện thực hiện. Các tín hiệu còn lại như PSEL, PADDR, PENABLE, PWRITE, PWDATA, PRDATA, PSLVERR, và PREADY được sử dụng để quản lý quá trình truyền dữ liệu giữa chủ và tớ. Theo đó, khi PSEL được kích hoạt, nó chọn một thiết bị ngoại vi cụ thể để giao tiếp. Tín hiệu PADDR là địa chỉ của thiết bị ngoại vi được chọn và được gửi từ thiết bị chủ đến thiết bị tớ. Trong quá trình truyền dữ liệu, tín hiệu PENABLE được sử dụng để kiểm soát và xác nhận sự bắt đầu của quá trình truyền. Tín hiệu PWRITE xác định hướng truyền dữ liệu; nếu PWRITE ở mức cao (1), dữ liệu sẽ được ghi vào tớ, nếu PWRITE ở mức thấp (0), tớ sẽ đọc dữ liệu từ chủ. PWDATA mang dữ liệu từ chủ đến tớ trong các hoạt động ghi, trong khi PRDATA mang dữ liệu từ tớ đến chủ trong các hoạt động đọc. Các tín hiệu bổ sung như PSTRB trong AMBA 4 cung cấp thêm khả năng kiểm soát byte trong quá trình ghi dữ liệu, cho phép chỉ ghi vào các byte cụ thể thay vì toàn bộ từ dữ liệu. PPROT cung cấp các tính năng bảo vệ truyền dữ liệu, chẳng hạn như quyền truy cập và các thuộc tính bảo

vệ khác. Nếu có lỗi trong quá trình truyền, PSLVERR sẽ được kích hoạt để thông báo cho chủ về lỗi đó. Khi tớ đã sẵn sàng và quá trình truyền dữ liệu hoàn tất, tín hiệu PREADY sẽ được sử dụng để báo hiệu rằng tớ đã hoàn tất chu kỳ truyền, đảm bảo sự đồng bộ trong hệ thống.

Thông thường, APB chỉ chiếm một phần nhỏ không gian bộ nhớ. Do đó, bus địa chỉ của hệ thống APB (PADDR[n:0]) thường không quá 32 bit ($n < 32$). Bên cạnh đó, không tín hiệu có điều khiển kích thước truyền trên APB. Tất cả các lần truyền được coi là 32 bit và thông thường, hai bit cuối có giá trị nhỏ nhất (bit 1 và bit 0) của PADDR không được sử dụng vì phải căn chỉnh một lần truyền từ trên APB.

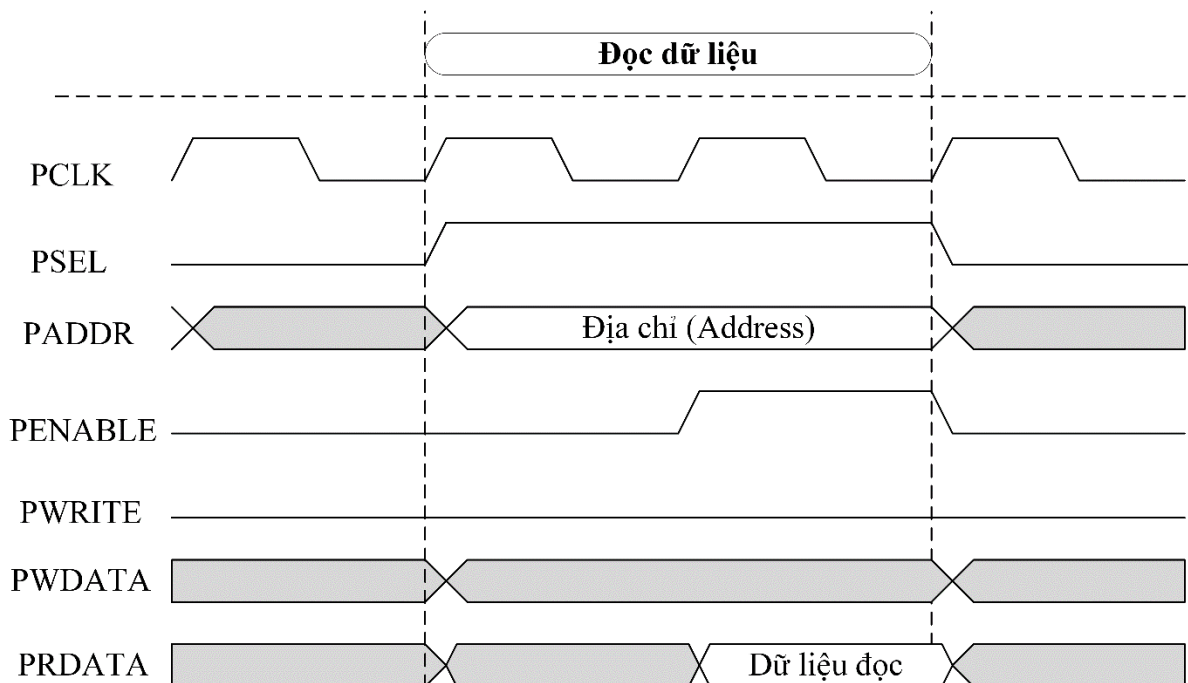


Hình 4.3: Một ví dụ về hệ thống APB.

Trong hầu hết các trường hợp, hệ thống APB có một cầu nối bus làm bus chủ kết nối APB với bus bộ xử lý chính (thường là AHB), như mô tả ở Hình 4.3. Ngoài ra, cần có bộ ghép kênh tớ APB và bộ giải mã địa chỉ. Bộ giải mã địa chỉ xác định thiết bị ngoại vi APB tớ nào được chọn thông qua tín hiệu PSEL, trong khi bộ ghép kênh (viết tắt là MUX) APB tớ quản lý việc truyền và nhận dữ liệu giữa AHB và các APB tớ. Điều này giúp hệ thống quản lý kết nối nhiều thiết bị ngoại vi một cách hiệu quả, tránh xung đột.

Lưu ý rằng trong các hệ thống đơn giản có cả AHB và APB, PCLK thường lấy từ cùng một nguồn xung nhịp với HCLK, và PRESETn lấy từ cùng một nguồn reset với HRESETn. Tuy nhiên, cũng có những hệ thống sử dụng các tần số HCLK và PCLK riêng biệt. Trong trường hợp đó, thiết kế cầu nối bus từ AHB đến APB cần phải có khả năng xử lý các giao dịch dữ liệu qua các tần số xung nhịp khác nhau hoặc các miền xung nhịp khác nhau. Ngoài PCLK, PRESETn, và PSEL, các **tín hiệu điều khiển & dữ liệu** đóng vai trò quan trọng trong việc truyền tải thông tin giữa AHB và các APB tới thông qua cầu nối và bộ ghép kênh. Các tín hiệu như PENABLE, PWRITE, PPROT, PSTRB, và PWDATA được sử dụng để điều khiển hoạt động ghi và quản lý dữ liệu từ chủ đến tớ. PENABLE và PWRITE xác định quá trình ghi hoặc đọc, trong khi PWDATA mang dữ liệu cần ghi từ chủ đến tớ. Khi thực hiện một hoạt động đọc, dữ liệu được các APB tớ gửi lại qua tín hiệu PRDATA, trong khi các tín hiệu như PSLVERR báo lỗi và PREADY báo hiệu khi tớ đã sẵn sàng cho một giao dịch mới. Các tín hiệu này được truyền qua AHB tớ MUX, đảm bảo dữ liệu được đọc chính xác và gửi đến AHB một cách hiệu quả.

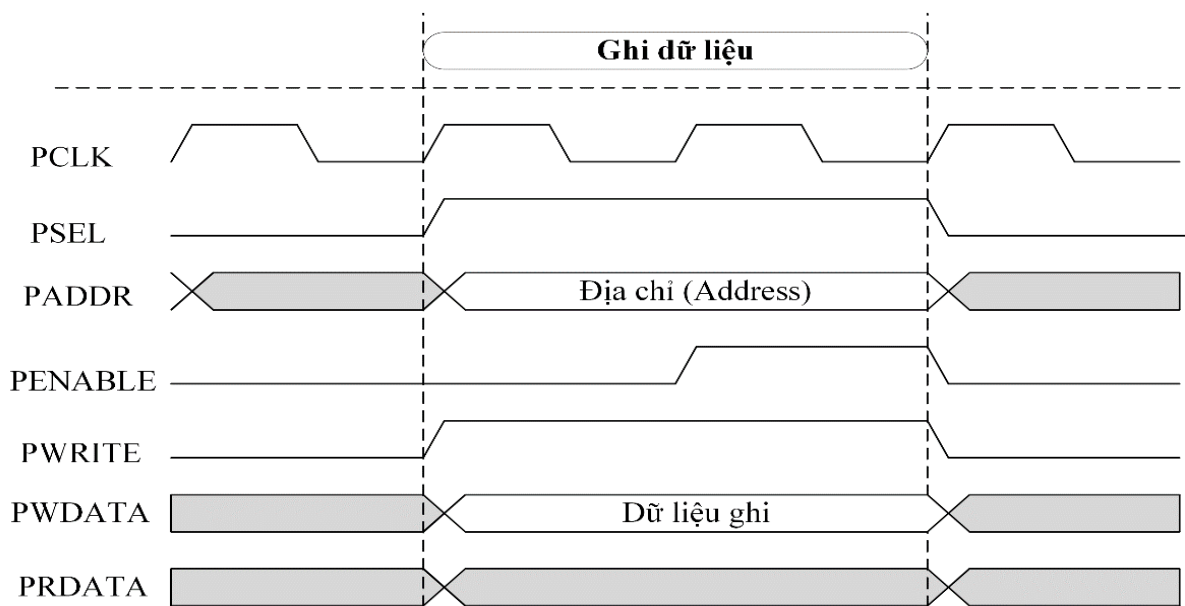
4.3.3. Chi tiết tín hiệu của APB trong việc ghi đọc dữ liệu



Hình 4.4: Sơ đồ thời gian chi tiết của tín hiệu đọc trong APB theo chuẩn AMBA 2.

Trong AMBA 2, mỗi lần truyền dữ liệu của APB cần tối thiểu hai chu kỳ xung nhịp. Đối với các thao tác đọc, dữ liệu cần phải sẵn sàng và hợp lệ ít nhất vào cuối chu kỳ

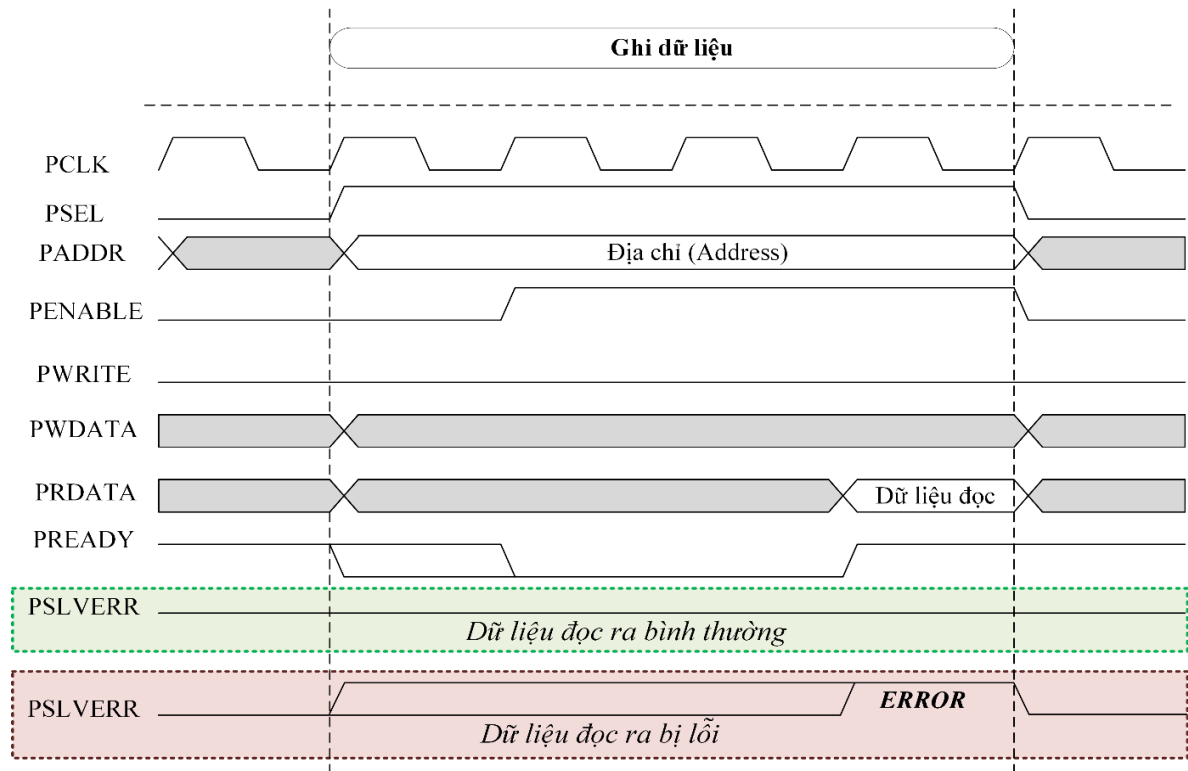
xung nhịp thứ hai, như minh họa trong Hình 4.4. Tín hiệu PCLK là xung nhịp chính điều phối toàn bộ hoạt động của APB, và tất cả các thay đổi tín hiệu đều diễn ra tại cạnh lên của PCLK. Khi PSEL được kích hoạt (mức cao), nó chọn thiết bị ngoại vi để bắt đầu giao tiếp. Sau khi PSEL được thiết lập, bus địa chỉ PADDR sẽ chỉ định địa chỉ của thiết bị ngoại vi cần truy cập. Tín hiệu PENABLE được đặt ở mức cao sau chu kỳ xung nhịp đầu tiên để xác nhận rằng quá trình truyền dữ liệu sẽ được thực hiện. Đối với hoạt động đọc, PWRITE được giữ ở mức thấp (0), chỉ định đây là một thao tác đọc. Trong trường hợp này, PRDATA mang dữ liệu từ tớ về chủ, và đến cuối chu kỳ xung nhịp thứ hai, PRDATA phải chứa giá trị hợp lệ để hoàn tất việc đọc dữ liệu.



Hình 4.5: Sơ đồ thời gian chi tiết của tín hiệu ghi trong APB theo chuẩn AMBA 2.

Trong quá trình truyền dữ liệu ghi trên APB ở AMBA2, thao tác ghi thực tế ở tớ có thể xảy ra trong chu kỳ xung nhịp đầu tiên hoặc chu kỳ thứ hai, tùy thuộc vào cách triển khai cụ thể. Do đó, chủ của APB phải đảm bảo rằng dữ liệu ghi (PWDATA) là hợp lệ trong cả hai chu kỳ xung nhịp. Ngoài ra, giữa hai lần truyền dữ liệu trên APB có thể có một số chu kỳ xung nhịp tùy ý, cho phép linh hoạt trong việc sắp xếp và điều phối các giao dịch dữ liệu. Trong sơ đồ thời gian ghi dữ liệu ở Hình 4.5, khi tín hiệu PSEL được kích hoạt, bus địa chỉ PADDR được thiết lập để chọn địa chỉ thiết bị ngoại vi muốn ghi. Sau khi PENABLE được kích hoạt sau chu kỳ xung nhịp đầu tiên, quá trình truyền dữ liệu sẽ diễn ra. PWRITE được đặt ở mức cao (giá trị 1) để xác định rằng đây là một hoạt động ghi. Tín hiệu PWDATA mang dữ liệu ghi từ chủ đến tớ, và dữ liệu này phải được giữ ổn định và hợp lệ trong suốt quá trình thực hiện ghi. Điều này đảm bảo rằng tớ nhận

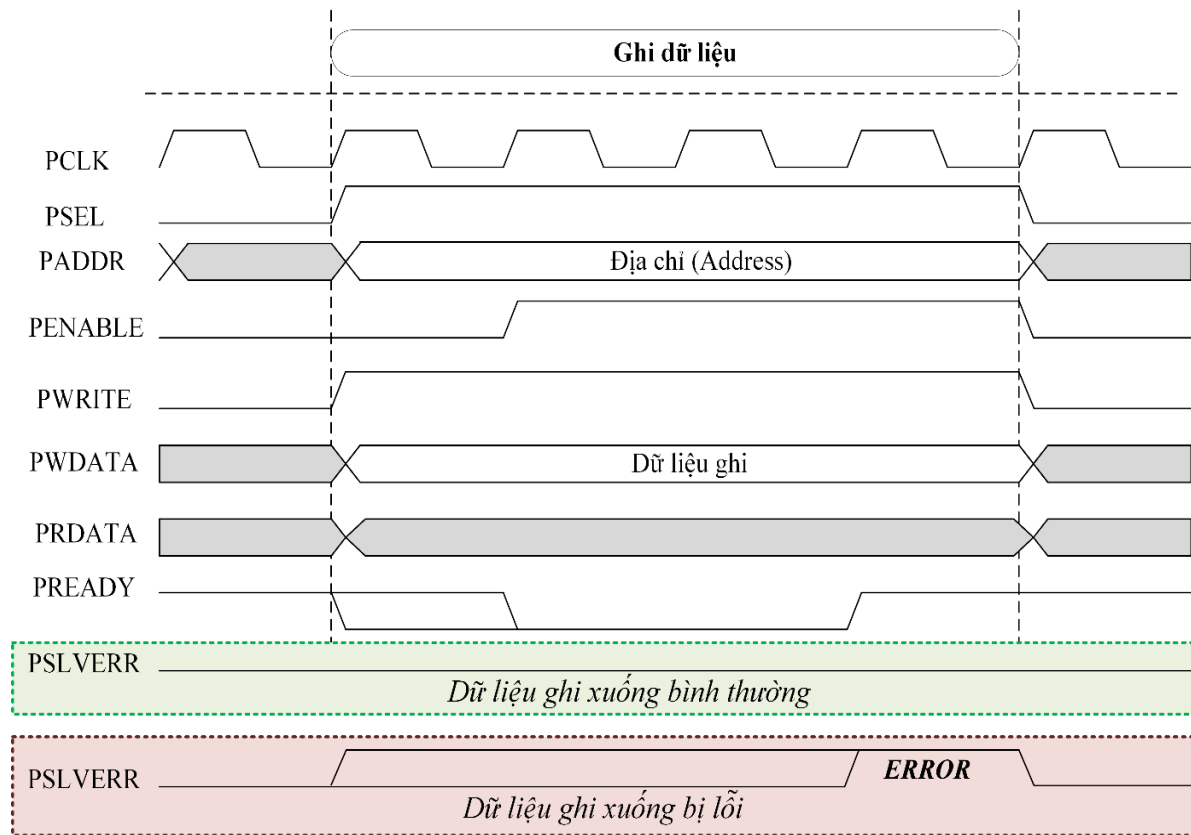
được dữ liệu chính xác bất kể thời điểm thực hiện ghi trong các chu kỳ xung nhịp được xác định.



Hình 4.6: Sơ đồ thời gian chi tiết của tín hiệu đọc trong APB với 3 trạng thái chờ và tín hiệu phản hồi theo chuẩn AMBA 3.

Trong chuẩn AMBA 3, mỗi APB tớ có thể kéo dài thời gian truyền dữ liệu bằng cách hủy kích hoạt tín hiệu đầu ra PREADY hoặc phản hồi với một lỗi bằng tín hiệu PSLVERR. Chẳng hạn, nếu một APB tớ cần 4 chu kỳ xung nhịp để hoàn thành một lần truyền đọc (tương đương với 3 trạng thái chờ), dạng sóng hoạt động đọc sẽ giống như trong Hình 4.6. Kết thúc quá trình truyền đọc được chỉ định khi tín hiệu PREADY được kích hoạt. Số chu kỳ tối thiểu cho một lần truyền dữ liệu APB vẫn là hai chu kỳ (tương tự như AMBA 2), và giá trị của PREADY trong chu kỳ đầu tiên của quá trình truyền sẽ bị bỏ qua. Điều này có nghĩa là, ngay cả khi giá trị của PREADY là logic 1, quá trình truyền vẫn phải mất ít nhất hai chu kỳ. Trong ví dụ được đề cập, AHB tớ phản hồi với trạng thái OKAY (được chỉ định bằng giá trị logic 0 trên PSLVERR khi PREADY là 1). Khi phản hồi lỗi được tạo ra, dữ liệu đọc từ AHB tớ có thể không chứa bất kỳ thông tin hữu ích nào và có thể bị loại bỏ. Giá trị của PSLVERR chỉ hợp lệ khi PREADY ở mức cao và không phải trong chu kỳ đầu tiên của quá trình truyền. Ví dụ, nếu PSLVERR

và PREADY đều ở mức cao trong chu kỳ đầu tiên của quá trình truyền, thì nó không được coi là phản hồi lỗi vì quá trình truyền APB phải có ít nhất hai chu kỳ xung nhịp.



Hình 4.7: Sơ đồ thời gian chi tiết của tín hiệu ghi trong APB với 3 trạng thái chờ và tín hiệu phản hồi theo chuẩn AMBA 3.

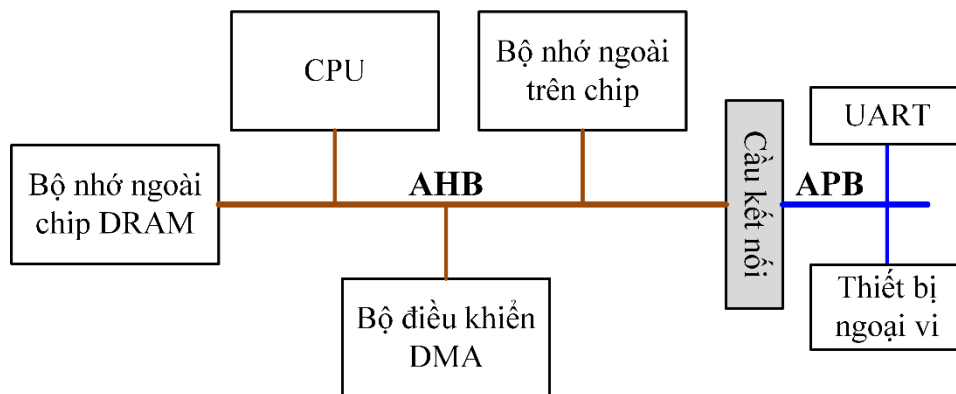
Tương tự như quá trình đọc, trong quá trình ghi, PREADY và PSLVERR đóng vai trò quyết định trạng thái ghi thành công hay thất bại như mô tả ở Hình 4.7. Khi PREADY được kích hoạt, tín hiệu PSLVERR xác định nếu dữ liệu ghi xuống là hợp lệ hoặc gặp lỗi (ERROR). Nếu PSLVERR ở mức thấp, dữ liệu được ghi thành công; nếu PSLVERR ở mức cao, ghi dữ liệu thất bại và có lỗi xảy ra.

Trong chuẩn AMBA 4, APB hoạt động tương tự như trong AMBA 3, nhưng được bổ sung thêm một số tín hiệu để cải thiện tính năng và độ an toàn của truyền dữ liệu. Cụ thể, tín hiệu PPROT[2:0] được thêm vào để điều khiển việc bảo vệ truyền dữ liệu, đảm bảo rằng chỉ các giao dịch có quyền hợp lệ mới được thực hiện. Bên cạnh đó, tín hiệu PSTRB[n-1:0] cũng được giới thiệu để cung cấp tín hiệu byte strobe cho các hoạt động ghi, cho phép kiểm soát chi tiết hơn đối với các byte được ghi vào thiết bị ngoại vi, giúp tối ưu hóa và linh hoạt hơn trong việc xử lý dữ liệu.

4.4. Hệ thống bus AHB

4.4.1. Giới thiệu về hệ thống bus AHB

Hầu hết các hệ thống dựa trên AHB đều sử dụng bus 32-bit, nhưng giao thức này được thiết kế để hỗ trợ các kích thước bus khác nhau. Độ rộng bus điển hình cho các hệ thống AHB là 32 hoặc 64-bit. Hình 4.8 mô tả một hệ thống điển hình sử dụng kiến trúc bus AMBA. Theo đó, AHB hình thành xương sống của hệ thống trên đó bộ xử lý ARM, giao diện bộ nhớ băng thông cao và bộ nhớ truy cập ngẫu nhiên (RAM), và bộ điều khiển truy cập bộ nhớ trực tiếp (DMA). Giao diện giữa bus AHB và bus APB chậm hơn là thông qua một module cầu kết nối. Không giống như các kiến trúc bus được thiết kế cho các hệ thống dựa trên PCB, AMBA AHB tránh việc triển khai tristate bằng cách sử dụng một sơ đồ kết nối chuyển mạch trung tâm. Phương pháp kết nối này cung cấp hiệu suất cao hơn và công suất thấp hơn so với việc sử dụng các bộ đệm tristate. Tất cả các chủ xác nhận các tín hiệu địa chỉ và điều khiển, chỉ ra loại truyền mà mỗi chủ yêu cầu. Một bộ giải mã trung tâm xác định chủ nào có địa chỉ và tín hiệu điều khiển định tuyến tới tất cả các thiết bị tới. Một mạch giải mã trung tâm chọn dữ liệu đọc thích hợp và tín hiệu phản hồi xác nhận từ tới có liên quan tham gia vào giao dịch.



Hình 4.8: Một hệ thống điển hình dựa trên bus AMBA dùng AHB và APB.

Tiêu chuẩn AHB đã trải qua nhiều lần phát hành và giai đoạn khác nhau:

- ✓ AMBA 2 AHB - phiên bản đầu tiên: Phiên bản này sử dụng một cặp tín hiệu bắt tay để phân giải xung đột giữa nhiều bus chủ.
- ✓ Thiết kế AHB đa lớp: Sản phẩm bộ thiết kế AMBA từ ARM đã giới thiệu một thành phần kết nối AHB gọi là ma trận AHB Bus. Thiết kế này cho phép truyền dữ liệu đồng thời trên bus trong các hệ thống nhiều chủ để tăng băng thông, loại

bỏ sự cần thiết của các tín hiệu yêu cầu và cấp quyền bus, và không chính thức được gọi là AHB Lite.

- ✓ Trong đặc tả AMBA 3, AHB Lite trở thành tên chính thức. Phiên bản này đã loại bỏ các tín hiệu Bus Request và Bus Granted và đơn giản hóa một số khía cạnh khác của giao thức AHB. Nó được sử dụng rộng rãi trong nhiều hệ thống vi xử lý ARM.
- ✓ Trong đặc tả AMBA 5, AHB đã được cập nhật để hỗ trợ TrustZone cho Armv8-M và bổ sung hỗ trợ chính thức cho các tín hiệu sideband truy cập độc quyền. Nó cũng giới thiệu một số cải tiến khác, bao gồm hỗ trợ thêm cho các thuộc tính bộ nhớ cache và các giải thích chi tiết hơn.

AHB 5 (hay còn gọi là AHB5) là phiên bản phát hành mới nhất của đặc tả AHB. Trong nhiều trường hợp, nó rất tương thích với phiên bản tiền nhiệm của nó và các thiết kế bus tổ tiên có được thiết kế cho AHB Lite có thể được tái sử dụng trong các hệ thống AHB5.

4.4.2. Tín hiệu và kết nối AHB

Hệ thống AHB hoạt động với tín hiệu xung nhịp gọi là HCLK. Tín hiệu này là chung cho tất cả các bus chủ, bus tớ và các khối cơ sở hạ tầng bus trong một phân đoạn bus. Tất cả các thanh ghi trên AHB kích hoạt tại các cạnh tăng của HCLK. Ngoài ra còn có một tín hiệu đặt lại hoạt động ở mức thấp gọi là HRESETn. Khi tín hiệu này ở mức thấp, nó sẽ đặt lại hệ thống AHB ngay lập tức (đặt lại không đồng bộ). Điều này cho phép hệ thống được đặt lại ngay cả khi xung nhịp đã dừng. Để hoạt động chính xác, bản thân tín hiệu HRESETn phải được đồng bộ hóa với HCLK để có thể tránh được tình trạng chạy đua. Mặt khác, nếu HRESETn hủy xác nhận vào cùng thời điểm với cạnh tăng của HCLK, bạn có thể thấy rằng một số phần của thanh ghi vẫn được đặt lại tại cạnh xung nhịp và một số thì không.

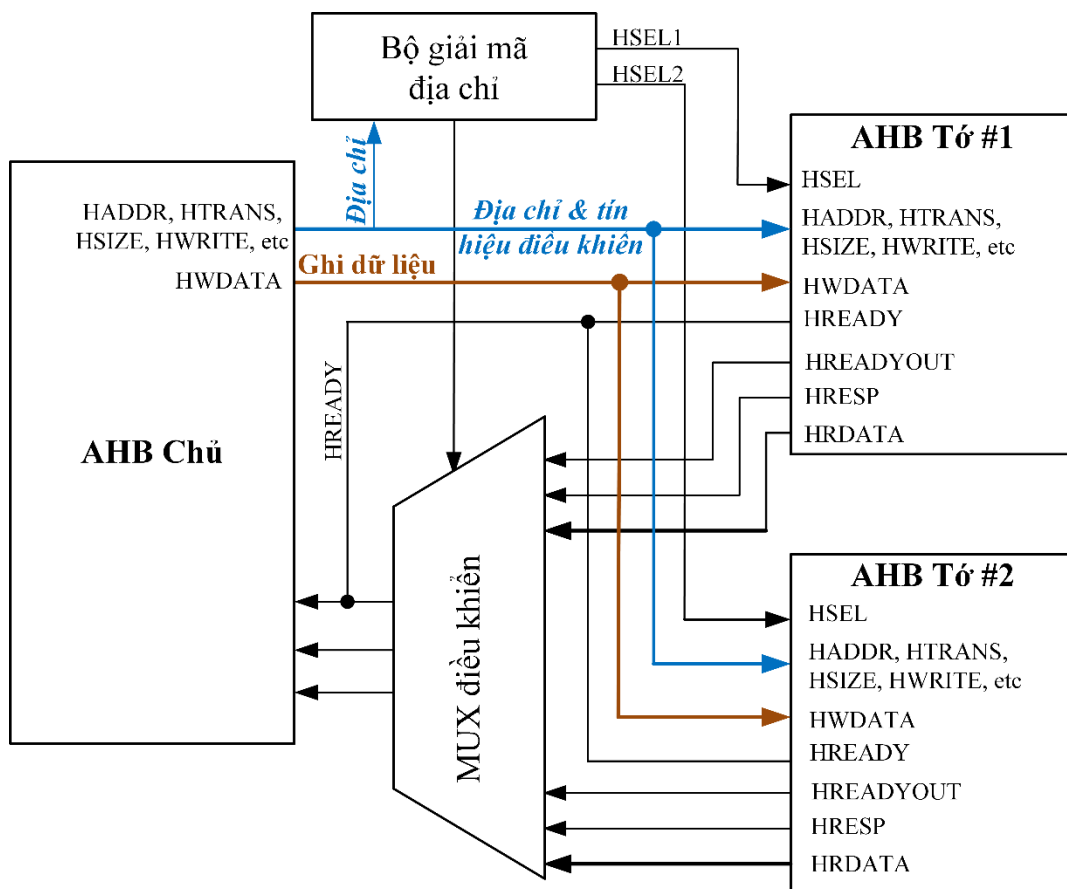
Bảng 4.2: Các tín hiệu điển hình của AHB

Tín hiệu	Chiều tín hiệu	Mô tả
HCLK	Nguồn xung nhịp→ tất cả các khối AHB	Tín hiệu xung nhịp chung

HRESETn	Nguồn reset → tất cả các khối AHB	Tín hiệu reset mức thấp chung
HSEL	Bộ giải mã địa chỉ → Tó	Chọn thiết bị tó
HADDR[31:0]	Chủ → Tó	Bus địa chỉ
HTRANS[1:0]	Chủ → Tó	Kiểm soát truyền dữ liệu
HWRITE	Chủ → Tó	Kiểm soát ghi (1=Ghi, 0=Đọc)
HSIZE[2:0]	Chủ → Tó	Kiểm soát kích thước truyền dữ liệu
HBURST[2:0]	Chủ → Tó	Kiểm soát truyền dữ liệu kiểu Burst
HPROT[3:0] / HPROT[6:0]	Chủ → Tó	Kiểm soát bảo vệ truyền dữ liệu (4 bit trong AHB-Lite và 7 bit trong AHB5)
HMASTLOCK	Chủ → Tó	Kiểm soát khóa truyền dữ liệu
HMASTER[3:0]	Thành phần bus → Bus tó	Xác định bus chủ hiện tại
HWDATA[31:0]	Chủ → Tó	Dữ liệu ghi (thường là 32-bit, nhưng có thể là 64-bit trên hệ thống 64-bit)
HRDATA[31:0]	Tó → Chủ	Dữ liệu đọc (thường là 32-bit, nhưng có thể là 64-bit trên hệ thống 64-bit)
HRESP[n:0] / HRESP	Tó → Chủ	Phản hồi từ tó (độ rộng 2 bit trong AMBA 2, 1 bit trong AHB-Lite/AHB5)
HREADY (HREADYOUT)	Tó → Chủ (HREADYOUT), Thành phần bus → các tó khác (HREADY)	Tó sẵn sàng (truyền dữ liệu hoàn tất). Tó được chọn hiện tại sẽ truyền tín hiệu HREADY đến chủ và tất cả các AHB tó khác. Do đó, một AHB tó có đầu vào HREADY và đầu ra HREADYOUT.

Đối với một hệ thống AHB thông thường, ta có thể tìm thấy hầu hết các tín hiệu được mô tả trong Bảng 4.2. Trong hệ thống AHB, các tín hiệu điều khiển và dữ liệu được sử dụng để quản lý quá trình truyền dữ liệu giữa chủ và tó. HADDR[31:0] là bus địa chỉ, được gửi từ chủ đến tó để chỉ định địa chỉ mục tiêu trong quá trình truyền dữ liệu. Tín hiệu HTRANS[1:0] được sử dụng để kiểm soát việc truyền dữ liệu, xác định

loại giao dịch (như IDLE, BUSY, NONSEQUENTIAL, SEQUENTIAL), trong khi HWRITE xác định hướng truyền dữ liệu (1 cho ghi và 0 cho đọc). HSIZE[2:0] chỉ định kích thước của mỗi lần truyền (byte, halfword, word, v.v.), và HBURST[2:0] kiểm soát loại Burst trong quá trình truyền dữ liệu, cho phép xử lý hiệu quả các chuỗi truyền dữ liệu liên tiếp. HPROT là tín hiệu bảo vệ, cung cấp thông tin về loại truy cập và mức độ bảo vệ dữ liệu, giúp tăng cường tính bảo mật và toàn vẹn dữ liệu, đặc biệt với các ứng dụng yêu cầu bảo mật cao. HMASTER[3:0] chỉ ra chủ hiện tại đang kiểm soát bus và tham gia vào giao dịch, giúp quản lý việc truy cập bus trong hệ thống đa chủ. Trong khi đó, HREADY và HREADYOUT được sử dụng để chỉ ra rằng tớ đã sẵn sàng và việc truyền dữ liệu đã hoàn tất, cho phép chủ biết khi nào có thể bắt đầu giao dịch tiếp theo. Các tín hiệu này cùng hoạt động để đảm bảo tính đồng bộ và hiệu quả trong các giao dịch trên bus AHB. Cuối cùng, các tín hiệu HWDATA[31:0] và HRDATA[31:0] được sử dụng để truyền dữ liệu ghi và đọc giữa chủ và tớ, trong khi HRESP báo cáo trạng thái của giao dịch (OKAY, ERROR, RETRY, SPLIT), cung cấp thông tin phản hồi về kết quả của các giao dịch để xử lý lỗi kịp thời.



Hình 4.9: Hệ thống AHB đơn giản với một AHB chủ và hai AHB tớ.

Thiết kế đơn giản với một bus chủ (ví dụ: bộ xử lý Cortex-M) và nhiều bus tớ được mô tả như Hình 4. 9. Trong hệ thống này, AHB Chủ gửi các tín hiệu địa chỉ và điều khiển như HADDR, HTRANS, HSIZE, HWRITE, và tín hiệu dữ liệu ghi HWDATA đến bộ giải mã địa chỉ và bộ MUX điều khiển. Bộ giải mã địa chỉ nhận tín hiệu địa chỉ từ chủ và giải mã để xác định tớ nào sẽ được chọn cho giao dịch hiện tại bằng cách tạo ra các tín hiệu chọn tớ như HSEL1, HSEL2. Các tín hiệu địa chỉ và điều khiển này sau đó được định tuyến đến các tớ thông qua bộ MUX điều khiển. Hai AHB Tớ trong hệ thống (Tớ #1 và Tớ #2) nhận các tín hiệu đã giải mã, bao gồm các tín hiệu địa chỉ, điều khiển, và dữ liệu từ AHB chủ. Mỗi tớ sẽ phản hồi bằng cách gửi dữ liệu đọc (HRDATA) và phản hồi trạng thái (HRESP, HREADY) trở lại qua bộ MUX điều khiển. Bộ MUX điều khiển sau đó sẽ đa hợp các phản hồi từ các tớ và gửi chúng về cho chủ. Thiết kế này cho phép AHB chủ giao tiếp với nhiều AHB tớ trên cùng một bus, quản lý giao dịch thông qua các tín hiệu địa chỉ, điều khiển, và dữ liệu, đồng thời xử lý các phản hồi từ các tớ một cách hiệu quả.

Các tín hiệu trong hệ thống AHB được chia thành hai nhóm: tín hiệu pha địa chỉ và tín hiệu pha dữ liệu.

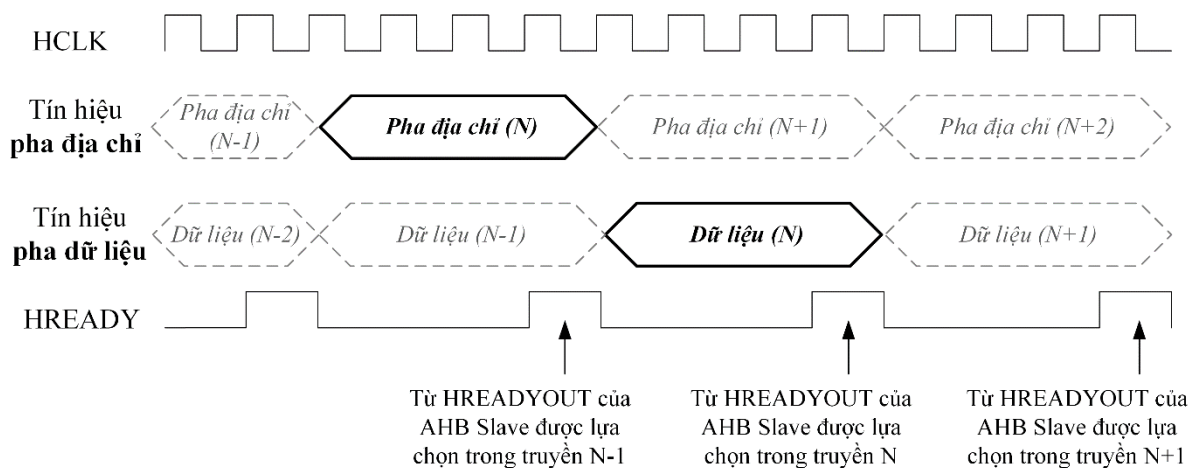
Tín hiệu Pha Địa Chỉ:

- ❖ HADDR, HTRANS, HSEL, HWRITE, HSIZE: Đây là các tín hiệu bắt buộc dùng để điều khiển quá trình truyền dữ liệu và địa chỉ trên bus AHB.
- ❖ Tùy chọn:
 - ✓ HPROT: Tín hiệu bảo vệ giao dịch, cung cấp thông tin về tính chất của giao dịch như quyền ưu tiên, người dùng không đặc quyền, và bảo vệ bộ nhớ.
 - ✓ HBURST: Tín hiệu chỉ định kiểu truyền Burst, được sử dụng để tối ưu hóa hiệu suất cho các giao dịch dữ liệu liên tiếp.
 - ✓ HMASTLOCK: Tín hiệu kiểm soát khóa truyền dữ liệu, chỉ có trong AMBA 2 AHB, cho phép khóa bus cho một chủ nhất định trong suốt quá trình truyền.
 - ✓ HEXCL: Tín hiệu chỉ ra rằng giao dịch hiện tại là quyền truy cập đọc quyền, chỉ có trong AMBA 5 AHB, được sử dụng trong các hệ thống có nhiều chủ để kiểm soát quyền truy cập duy nhất.
 - ✓ HAUSER: Dải tần người dùng tùy chọn cho tín hiệu pha địa chỉ, chỉ có trong AMBA 5 AHB, cho phép truyền các thông tin bổ sung do người dùng định nghĩa.

Tín hiệu Pha Dữ Liệu:

- ❖ HWDATA, HRDATA, HRESP, HREADY, HREADYOUT: Đây là các tín hiệu chính được sử dụng trong pha dữ liệu của giao dịch. HWDATA và HRDATA truyền dữ liệu viết và đọc, trong khi HRESP, HREADY, và HREADYOUT xác định trạng thái và độ sẵn sàng của các giao dịch.
- ❖ Tùy chọn:
 - ✓ HEXOKAY: Tín hiệu phản hồi thành công truy cập đọc quyền, chỉ có trong AMBA 5 AHB, dùng để xác nhận rằng giao dịch đọc quyền đã được thực hiện thành công.
 - ✓ HWUSER/HRUSER: Dải tần người dùng tùy chọn cho tín hiệu pha dữ liệu, chỉ có trong AMBA 5 AHB, cho phép truyền thông tin bổ sung tùy theo yêu cầu của người dùng trong suốt quá trình truyền dữ liệu.

Mỗi lần truyền dữ liệu trong AHB bao gồm hai pha: pha địa chỉ và pha dữ liệu. Các lần truyền này được thực hiện theo kiểu pipelined, cho phép pha địa chỉ của một lần truyền có thể chồng lên pha dữ liệu của lần truyền trước đó. Mỗi pha được kết thúc khi tín hiệu HREADYOUT (HREADY) từ AHB trở nên hiệu lực tại trong pha dữ liệu được kích hoạt. Tín hiệu HREADYOUT từ các AHB trở được ghép kênh bởi bộ ghép kênh tứ, tạo thành tín hiệu HREADY cho toàn hệ thống. Bộ ghép kênh này hoạt động trong pha dữ liệu của mỗi lần truyền. Việc điều khiển bộ ghép kênh có thể được tạo ra từ bộ giải mã AHB, hoặc từ các tín hiệu HSEL và HREADY.



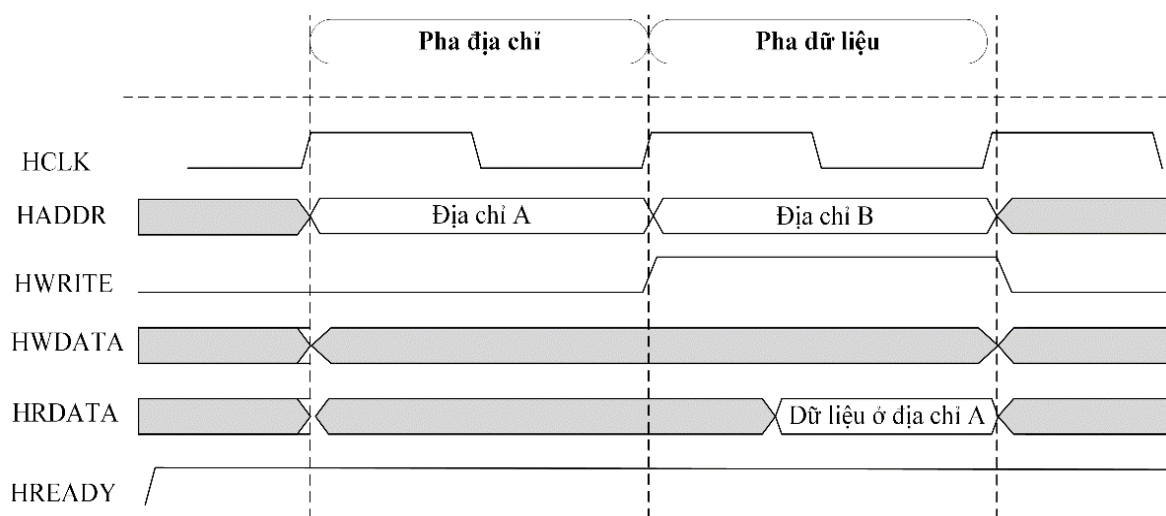
Hình 4.10: Phân chia quá trình truyền dữ liệu thành giai đoạn địa chỉ và dữ liệu.

Trong Hình 4.10, ta thấy rõ quá trình phân chia và phối hợp giữa pha địa chỉ và pha dữ liệu trong các lần truyền dữ liệu. Trong khi pha địa chỉ của một lần truyền mới (N)

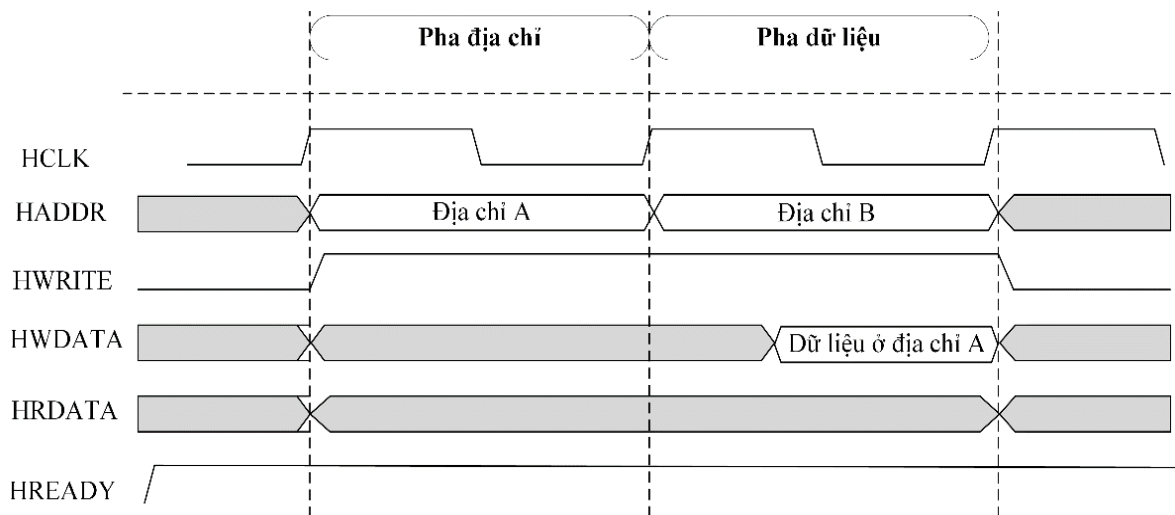
bắt đầu, pha dữ liệu của lần truyền trước đó (N-1) vẫn đang diễn ra. Mỗi lần truyền dữ liệu được kết thúc bằng tín hiệu HREADYOUT từ AHB Tó được chọn, báo hiệu kết thúc pha dữ liệu và cho phép hệ thống bắt đầu pha địa chỉ của lần truyền tiếp theo.

4.4.3. Chi tiết tín hiệu của AHB trong việc ghi đọc dữ liệu cơ bản

Phần này trình bày chi tiết về các tín hiệu của AHB trong quá trình ghi và đọc dữ liệu cơ bản, bao gồm cách thức hoạt động và sự phối hợp giữa các tín hiệu chính như HCLK, HADDR, HWRITE, HWDATA, HRDATA, và HREADY, như mô tả ở Hình 4.11 và 5.12. Quá trình truyền dữ liệu trong AHB bao gồm hai pha: pha địa chỉ và pha dữ liệu. Pha địa chỉ kéo dài trong một chu kỳ của HCLK, trừ khi bị kéo dài bởi lần truyền trước đó, trong khi pha dữ liệu có thể yêu cầu nhiều chu kỳ HCLK. Tín hiệu HREADY được sử dụng để kiểm soát số chu kỳ cần thiết để hoàn thành quá trình truyền dữ liệu. Tín hiệu HWRITE điều khiển hướng truyền dữ liệu đến hoặc từ Chủ; khi HWRITE ở mức cao, đó là lần truyền ghi và Chủ sẽ gửi dữ liệu lên bus dữ liệu ghi, HWDATA, còn khi HWRITE ở mức thấp, đó là lần truyền đọc và Tó phải tạo dữ liệu trên bus dữ liệu đọc HRDATA. Quá trình truyền đơn giản nhất là không có trạng thái chờ, bao gồm một chu kỳ địa chỉ và một chu kỳ dữ liệu. Ví dụ này cho thấy cách các pha địa chỉ và dữ liệu của quá trình truyền xảy ra trong các chu kỳ xung nhịp khác nhau, với pha địa chỉ của bất kỳ quá trình truyền nào cũng diễn ra trong pha dữ liệu của lần truyền trước đó. Sự chồng chéo này cho phép hoạt động hiệu suất cao trong cấu trúc đường ống của bus, đồng thời cung cấp đủ thời gian để Tó đưa ra phản hồi cho một lần truyền.

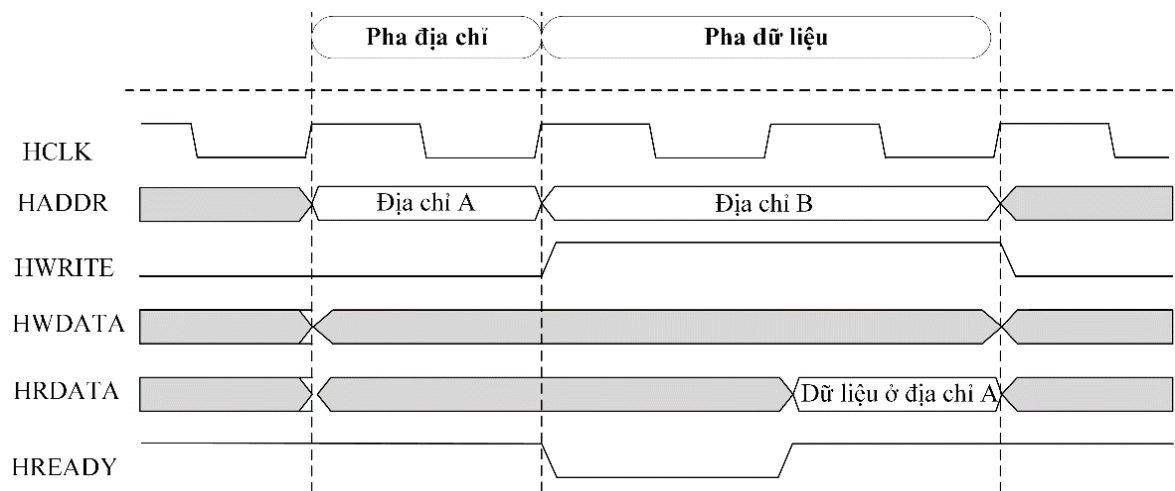


Hình 4.11: Sơ đồ thời gian chi tiết của tín hiệu đọc cơ bản trong AHB.



Hình 4.12: Sơ đồ thời gian chi tiết của tín hiệu ghi cơ bản trong AHB.

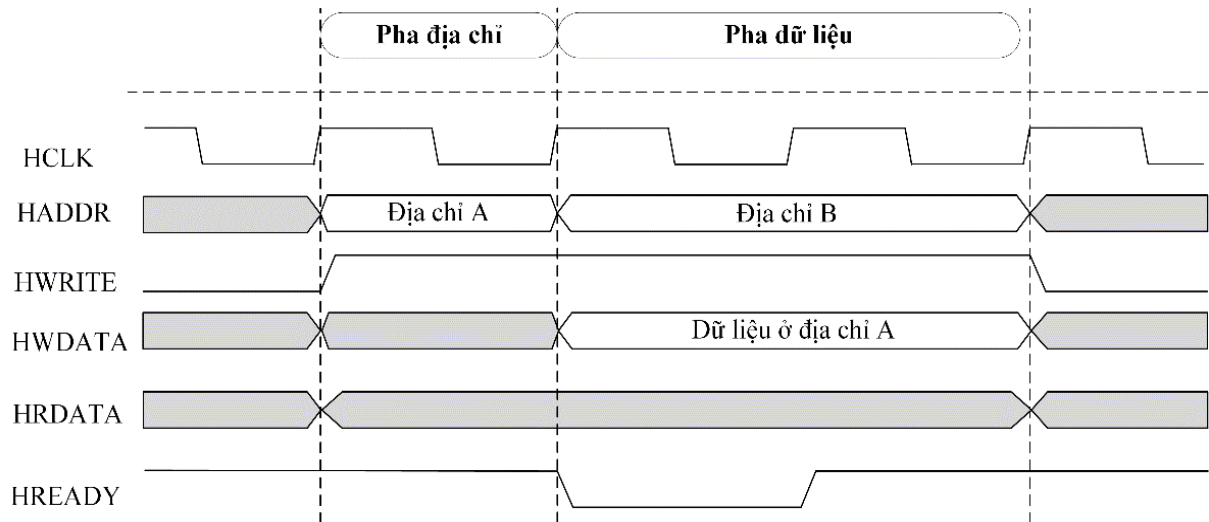
Một Tó có thể chèn các trạng thái chờ vào bất kỳ lần truyền dữ liệu nào để có thêm thời gian hoàn thành. Mỗi Tó có một tín hiệu HREADYOUT mà nó điều khiển trong pha dữ liệu của một lần truyền. Kết nối liên lạc chịu trách nhiệm kết hợp các tín hiệu HREADYOUT từ tất cả các Tó để tạo ra một tín hiệu HREADY duy nhất dùng để kiểm soát tiến trình tổng thể.



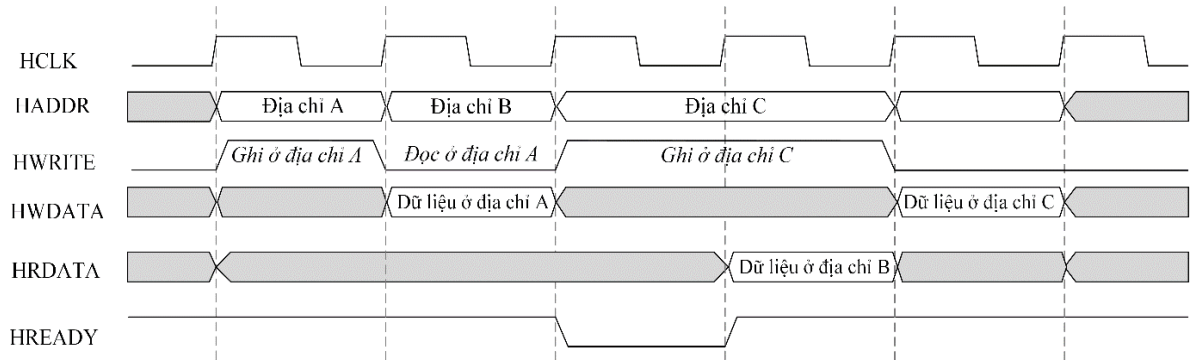
Hình 4.13: Sơ đồ thời gian chi tiết của tín hiệu đọc cơ bản với hai trạng thái chờ.

Trong Hình 4.13 và Hình 4.14, sơ đồ thời gian minh họa chi tiết các tín hiệu trong quá trình đọc và ghi dữ liệu cơ bản trên bus AHB khi Tó chèn hai trạng thái chờ. Trong Hình 4.13, quá trình đọc dữ liệu diễn ra như sau: HADDR chỉ định địa chỉ của dữ liệu cần đọc trong pha địa chỉ (địa chỉ A). Tín hiệu HWRITE ở mức thấp cho thấy đây là một quá trình đọc. Sau đó, tín hiệu HREADY ở mức thấp trong hai chu kỳ xung nhịp, biểu thị hai trạng thái chờ. Trong thời gian này, Tó chưa sẵn sàng cung cấp dữ liệu. Khi HREADY lên mức cao, HRDATA sẽ chứa dữ liệu tại địa chỉ A, hoàn thành quá trình

đọc. Tương tự, Hình 4.14 mô tả quá trình ghi dữ liệu với hai trạng thái chờ. Trong pha địa chỉ, HADDR xác định địa chỉ đích (địa chỉ A) và HWRITE được đặt ở mức cao để chỉ định quá trình ghi. Dữ liệu cần ghi sẽ được chuẩn bị trên HWDATA trong pha dữ liệu. Tín hiệu HREADY cũng được đặt ở mức thấp trong hai chu kỳ xung nhịp, biểu thị hai trạng thái chờ. Sau khi HREADY lên mức cao, dữ liệu tại HWDATA sẽ được ghi vào địa chỉ A, kết thúc quá trình ghi.



Hình 4.14: Sơ đồ thời gian chi tiết của tín hiệu ghi cơ bản với hai trạng thái chờ.



Hình 4.15: Sơ đồ thời gian chi tiết của tín hiệu kết hợp đọc và ghi cơ bản trong AHB.

Khi một quá trình truyền dữ liệu được kéo dài, nó sẽ kéo dài pha địa chỉ của lần truyền tiếp theo. Hình 4.15 minh họa ba lần truyền đến ba địa chỉ không liên quan: A, B, và C, với pha địa chỉ mở rộng cho địa chỉ C. Trong sơ đồ này, các lần truyền đến địa chỉ A và C không có trạng thái chờ (zero wait state), trong khi lần truyền đến địa chỉ B có một trạng thái chờ. Việc kéo dài pha dữ liệu của lần truyền đến địa chỉ B làm kéo dài pha địa chỉ của lần truyền đến địa chỉ C. Điều này được thể hiện rõ ràng khi pha địa chỉ của địa chỉ C bắt đầu sau khi pha dữ liệu của địa chỉ B kết thúc. Điều này minh họa cách

mà tín hiệu HREADY và các trạng thái chờ có thể ảnh hưởng đến các pha địa chỉ và dữ liệu liên tiếp trong hệ thống bus AHB.

4.4.4. Chi tiết tín hiệu của AHB trong việc ghi đọc dữ liệu với Burst

Truyền dữ liệu cơ bản trong AHB thường đơn giản trong thiết kế và dễ dàng triển khai cho các ứng dụng không yêu cầu băng thông cao. Tuy nhiên, nhược điểm của nó là không hiệu quả khi cần truyền một lượng lớn dữ liệu liên tiếp, vì mỗi lần truyền dữ liệu yêu cầu một chu kỳ địa chỉ và một chu kỳ dữ liệu riêng biệt, dẫn đến thời gian chờ giữa các lần truyền dài và sử dụng băng thông kém hiệu quả. Để khắc phục những hạn chế này, truyền Burst được sử dụng, cho phép truyền nhiều dữ liệu liên tiếp trong một chuỗi mà chỉ cần thiết lập địa chỉ một lần. Điều này giúp tối ưu hóa băng thông và tăng hiệu suất truyền dữ liệu, đặc biệt là trong các ứng dụng yêu cầu tốc độ cao như truyền dữ liệu từ bộ nhớ đến bộ xử lý hoặc các thiết bị ngoại vi tốc độ cao.

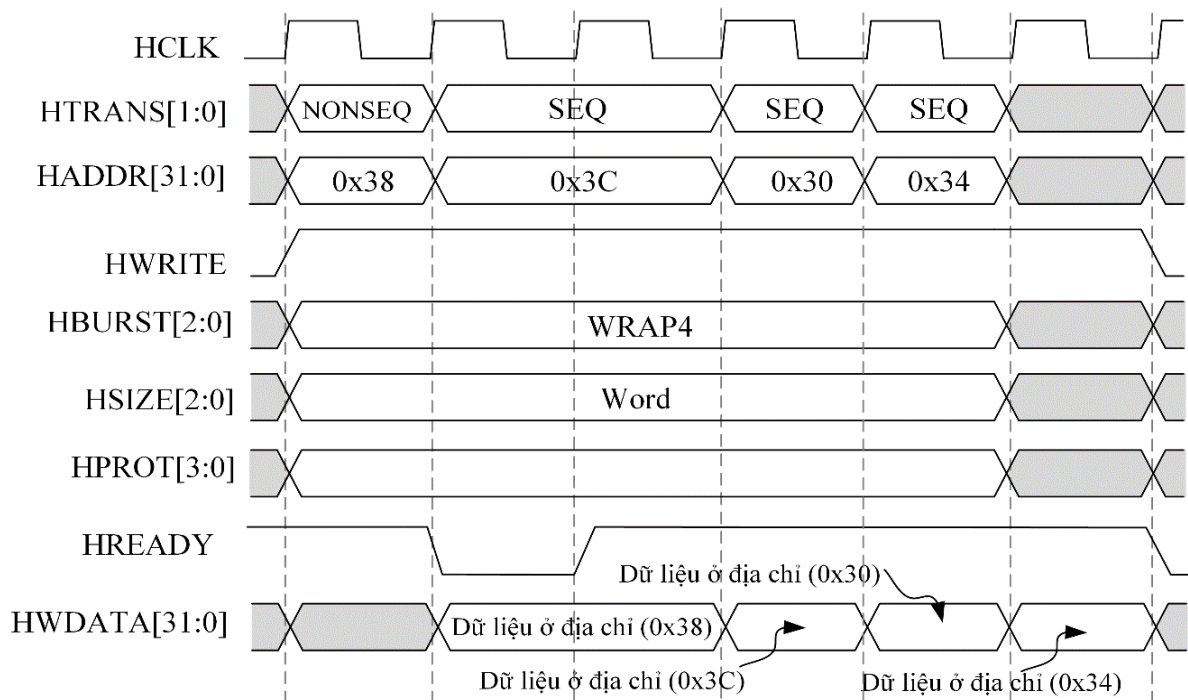
Bảng 4.3: Mã hóa tín hiệu Burst

HBURST[2:0]	Loại	Mô tả
000	SINGLE	Truyền đơn Burst
001	INCR	Truyền Burst gia tăng với độ dài không xác định
010	WRAP4	Truyền Burst vòng 4 nhịp
011	INCR4	Truyền Burst tăng dần 4 nhịp
100	WRAP8	Truyền Burst vòng 8 nhịp
101	INCR8	Truyền Burst gia tăng 8 nhịp
110	WRAP16	Truyền Burst vòng 16 nhịp
111	INCR16	Truyền Burst gia tăng 16 nhịp

Truyền Burst cho phép truyền nhiều dữ liệu liên tiếp trong một chuỗi, giúp tối ưu hóa băng thông và hiệu suất truyền dữ liệu trên bus. Trong giao thức AHB, các Burst có độ dài 4, 8, 16 nhịp, các Burst có độ dài không xác định, và các lần truyền đơn lẻ đều được định nghĩa. Giao thức này hỗ trợ các Burst gia tăng (incrementing) và các Burst vòng (wrapping). Các Burst gia tăng truy cập các vị trí địa chỉ liên tiếp, và địa chỉ của mỗi lần truyền trong Burst là một phần tăng của địa chỉ trước đó. Trong khi đó, Burst vòng sẽ quay lại khi chúng vượt qua một ranh giới địa chỉ. Ranh giới địa chỉ được tính toán như là tích của số lượng nhịp trong một Burst và kích thước của lần truyền. Số lượng nhịp được kiểm soát bởi tín hiệu HBURST và kích thước truyền dữ liệu được

kiểm soát bởi HSIZE. Ví dụ, một Burst vòng có 4 nhịp, mỗi nhịp là một từ (4 byte), sẽ vòng lại tại các ranh giới 16 byte. Do đó, nếu địa chỉ bắt đầu của Burst là 0x34, nó sẽ bao gồm bốn lần truyền đến các địa chỉ 0x34, 0x38, 0x3C và 0x30. Tín hiệu HBURST[2:0] điều khiển loại Burst, như được liệt kê trong Bảng 4.3.

Các Chủ không nên khởi đầu một Burst gia tăng vượt qua ranh giới địa chỉ 1KB. Chủ có thể thực hiện các lần truyền đơn lẻ bằng cách sử dụng SINGLE transfer Burst hoặc Burst có độ dài không xác định nhưng chỉ có một nhịp. Kích thước Burst cho biết số nhịp trong Burst chứ không phải số byte được truyền. Để tính tổng lượng dữ liệu được truyền trong một Burst, ta nhân số nhịp với lượng dữ liệu trong mỗi nhịp, như được chỉ định bởi HSIZE[2:0]. Tất cả các lần truyền trong một Burst phải được căn chỉnh với ranh giới địa chỉ bằng với kích thước của lần truyền. Ví dụ, các lần truyền từ phải căn chỉnh với các ranh giới từ (HADDR[1:0] = 0b00), và các lần truyền halfword phải căn chỉnh với các ranh giới halfword (HADDR[0] = 0).

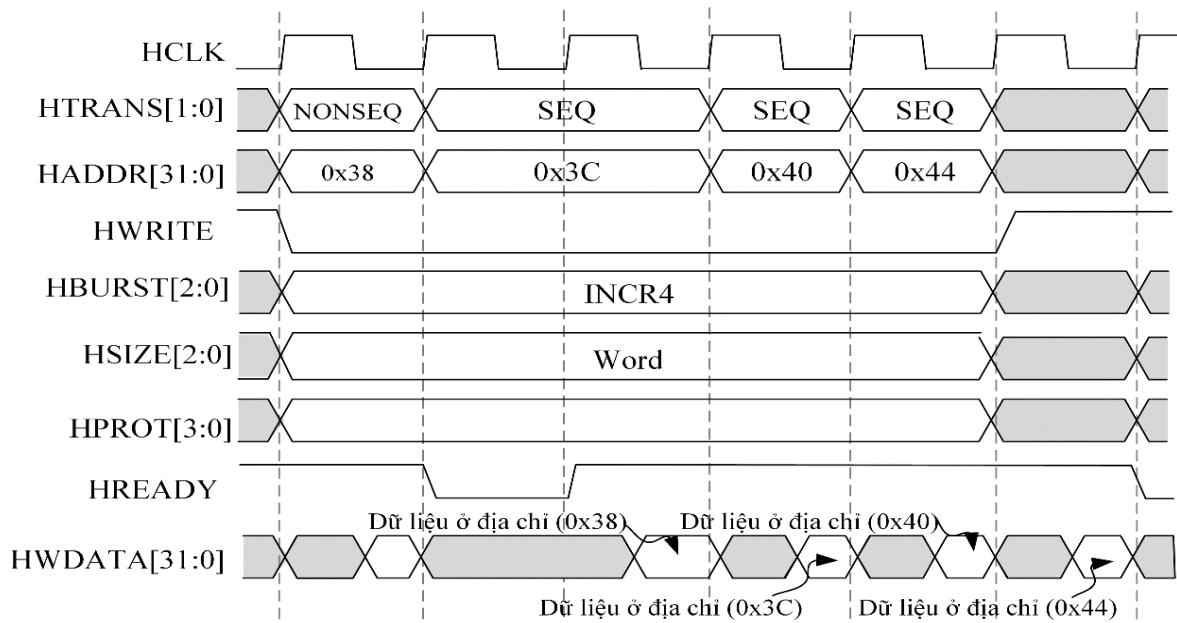


Hình 4.16: Sơ đồ thời gian chi tiết của tín hiệu kết hợp đọc với Burst vòng 4 nhịp.

Hình 4.16 mô tả một lần truyền ghi sử dụng Burst vòng 4 nhịp với một trạng thái chờ được thêm vào cho lần truyền đầu tiên. Do đây là một Burst vòng 4 nhịp cho các lần truyền dữ liệu kiểu từ, địa chỉ sẽ vòng lại tại các biên 16 byte. Cụ thể, lần truyền đến địa chỉ 0x3C sẽ được theo sau bởi lần truyền đến địa chỉ 0x30. Các trạng thái của HTRANS[1:0] được mô tả chi tiết trong Bảng 4.4.

Bảng 4.4: Mã hóa loại truyền dữ liệu

HTRANS[1:0]	Loại	Mô tả
00	IDLE	Chỉ ra rằng không cần truyền dữ liệu. Manager sử dụng truyền IDLE khi nó không muốn thực hiện truyền dữ liệu. Nên kết thúc một truyền bị khóa bằng truyền IDLE. Subordinate luôn phải cung cấp phản hồi OKAY trạng thái chờ bằng không (zero wait state) cho truyền IDLE và bỏ qua truyền này.
01	BUSY	Loại truyền BUSY cho phép Manager chen các chu kỳ nhàn rỗi vào giữa một Burst. Loại truyền này chỉ ra rằng Manager đang tiếp tục với một Burst nhưng lần truyền tiếp theo không thể diễn ra ngay lập tức. Khi Manager sử dụng loại truyền BUSY, các tín hiệu địa chỉ và điều khiển phải phản ánh lần truyền tiếp theo trong Burst. Chỉ những Burst có độ dài không xác định mới có thể có truyền BUSY làm chu kỳ cuối của Burst. Subordinate luôn phải cung cấp phản hồi OKAY trạng thái chờ bằng không cho các truyền BUSY và bỏ qua truyền này.
10	NONSEQ	Chỉ ra một truyền đơn lẻ hoặc lần truyền đầu tiên của một Burst. Các tín hiệu địa chỉ và điều khiển không liên quan đến lần truyền trước đó. Các lần truyền đơn lẻ trên bus được coi là Burst có độ dài một nhịp và do đó loại truyền này là NONSEQUENTIAL.
11	SEQ	Các lần truyền còn lại trong một Burst là SEQUENTIAL và địa chỉ có liên quan đến lần truyền trước đó. Thông tin điều khiển giống hệt với lần truyền trước. Địa chỉ bằng với địa chỉ của lần truyền trước đó cộng với kích thước truyền, tính bằng byte, với kích thước truyền được báo hiệu bằng tín hiệu HSIZE[2:0] . Trong trường hợp Burst vòng, địa chỉ của truyền sẽ vòng lại ở ranh giới địa chỉ.



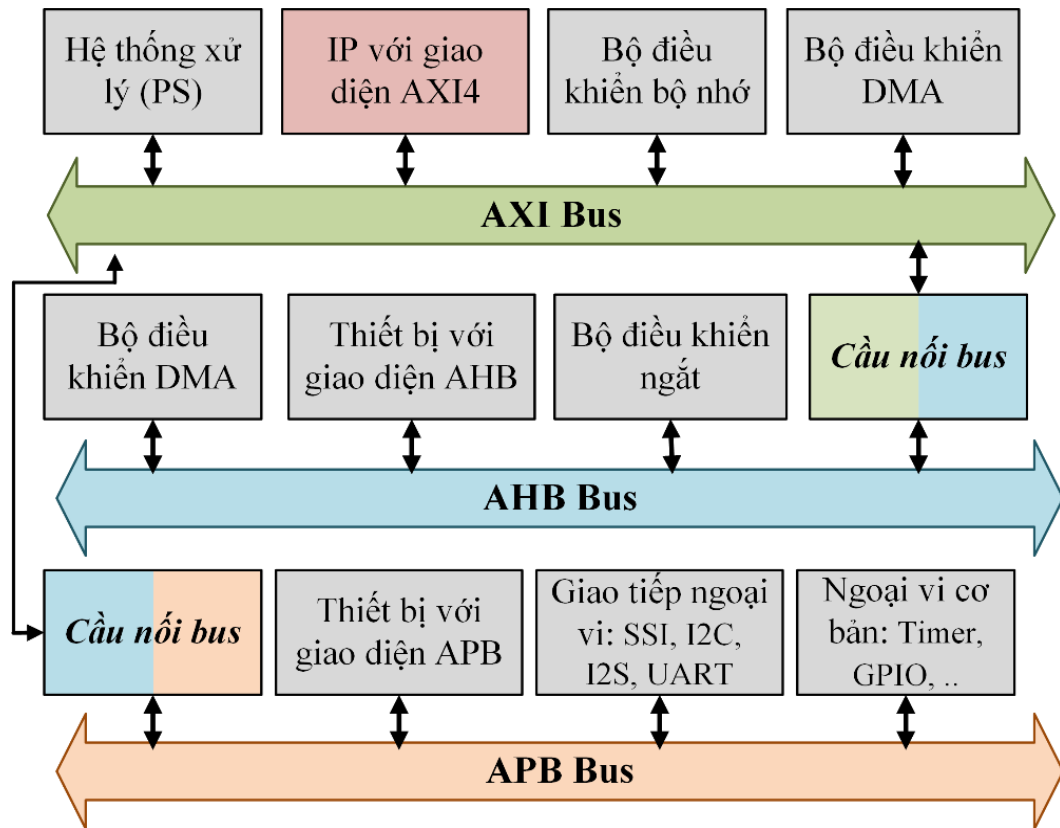
Hình 4.17: Sơ đồ thời gian chi tiết của tín hiệu kết hợp đọc với Burst gia tăng 4 nhịp.

Hình 4.17 minh họa quá trình truyền dữ liệu đọc bằng cách sử dụng một Burst gia tăng 4 nhịp (INC4) với một trạng thái chờ được thêm vào cho lần truyền đầu tiên. Trong trường hợp này, địa chỉ không quay vòng tại ranh giới 16 byte. Địa chỉ bắt đầu từ 0x38 và tiếp theo là 0x3C, sau đó đến 0x40 và 0x44. Điều này được thể hiện qua tín hiệu HTRANS[1:0] với trạng thái NONSEQ cho lần truyền đầu tiên và SEQ cho các lần truyền tiếp theo. Các tín hiệu dữ liệu HWDATA[31:0] hiển thị dữ liệu được đọc lần lượt tại các địa chỉ tương ứng, bắt đầu từ 0x38 và kết thúc tại 0x44. Việc truyền dữ liệu theo kiểu Burst gia tăng giúp cải thiện hiệu suất truy cập bộ nhớ trong hệ thống nhúng, giảm thiểu độ trễ bằng cách duy trì các giao dịch liên tiếp mà không cần khởi động lại quy trình truyền.

Việc ghi dữ liệu trong các trường hợp Burst vòng 4 nhịp (WRAP4) và Burst gia tăng 4 nhịp (INC4) cũng tuân theo nguyên tắc tương tự như quá trình đọc dữ liệu đã mô tả. Với Burst vòng 4 nhịp, địa chỉ sẽ quay vòng lại khi vượt qua ranh giới 16 byte, trong khi với Burst gia tăng 4 nhịp, địa chỉ sẽ tiếp tục tăng mà không quay vòng. Tương tự, các loại Burst khác như Burst vòng 8 nhịp (WRAP8), Burst gia tăng 8 nhịp (INC8), Burst vòng 16 nhịp (WRAP16), và Burst gia tăng 16 nhịp (INC16) đều hoạt động dựa trên nguyên tắc này. Sự khác biệt chính nằm ở số lần nhịp trong mỗi Burst và việc địa chỉ có quay vòng hay không.

4.5. Hệ thống bus AXI

4.5.1. Giới thiệu về hệ thống bus AXI



Hình 4.18: Hệ thống kết hợp các bus AXI, AHB, và APB trên nhiều Xilinx FPGA.

AMBA AXI có băng thông cao hơn AHB và APB nhờ thiết kế pipeline tối ưu, cho phép sử dụng hiệu quả bus khi chia sẻ giữa nhiều lõi CPU và các bộ điều khiển truy cập bộ nhớ trực tiếp DMA. AXI hỗ trợ các cấu trúc bus phức tạp, cho phép chia sẻ các điểm cuối tới giữa nhiều bus chủ với nhiều giao dịch cùng lúc. Trong nhiều hệ thống trên nhiều FPGA, nhiều loại bus như AXI, AHB, và APB được kết hợp lại với nhau như mô tả ở Hình 4.18. Ví dụ, phần hệ thống xử lý (PS) của Zynq FPGA sử dụng cả ba loại bus AXI, AHB và APB. Kết nối chính của hệ thống dựa trên AXI, kết nối lõi xử lý với SRAM trên chip, bộ điều khiển bộ nhớ DDR ngoài, và với FPGA. Các cổng AXI của FPGA bao gồm cả giao diện tớ và chủ. Các cổng tớ cho phép bộ xử lý truy cập các tài nguyên FPGA và các cổng chủ cho phép logic của FPGA truy cập các tài nguyên của PS. Các thiết bị ngoại vi phần cứng của Zynq FPGA được kết nối với APB thông qua cầu nối đến kết nối chính AXI. Các bộ điều khiển USB và Ethernet trong PS của Zynq FPGA có giao diện AHB Chủ, nhưng chúng được kết nối với AXI để có thể truy cập bộ nhớ của bộ xử lý, điều này được gọi là truy cập bộ nhớ trực tiếp.

Tiêu chuẩn AXI đã trải qua nhiều lần phát triển và mở rộng qua các thế hệ AMBA khác nhau để đáp ứng nhu cầu truyền nhận dữ liệu các hệ thống nhúng phức tạp khác nhau theo từng yêu cầu của hệ thống:

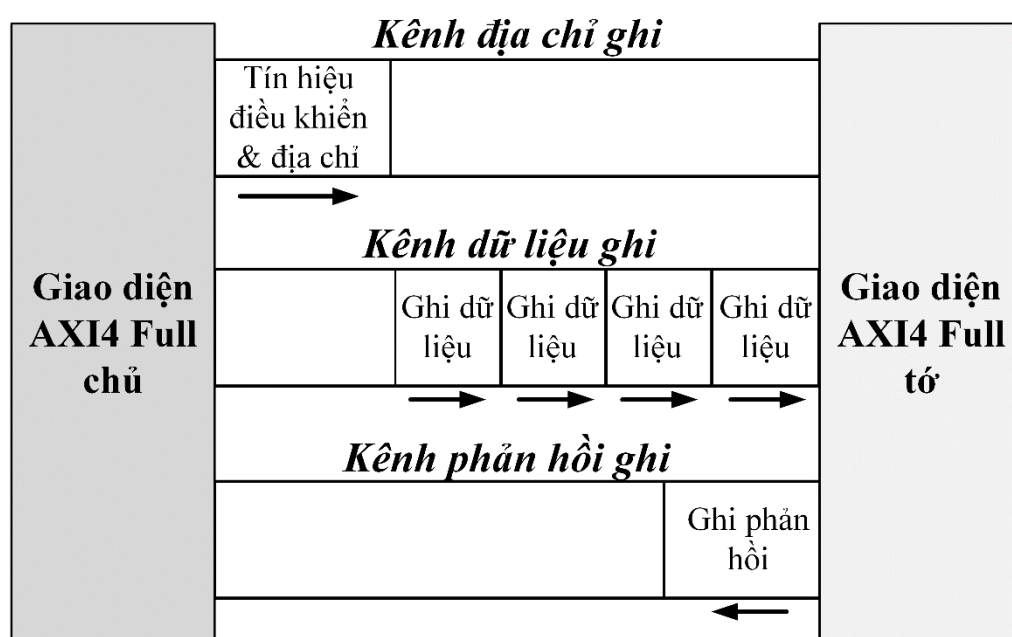
- ✓ AXI3: AXI3 là phiên bản đầu tiên của giao thức AXI được giới thiệu trong đặc tả AMBA 3. Phiên bản này hỗ trợ các giao dịch đọc và ghi không đồng bộ với các kênh dữ liệu, địa chỉ, và phản hồi độc lập, cho phép các giao dịch có thể diễn ra song song mà không bị chờ đợi. Điều này giúp cải thiện băng thông và tối ưu hóa việc sử dụng bus trong các hệ thống có nhiều chủ.
- ✓ AXI4: Phiên bản này mở rộng tính năng của AXI3 bằng cách hỗ trợ các Burst dài hơn (lên đến 256 nhịp), tăng cường hiệu suất truyền tải dữ liệu trong các hệ thống đòi hỏi băng thông cao. AXI4 cũng hỗ trợ các thiết kế đa chủ với nhiều tớ và các giao dịch phức tạp. AXI4 có ba biến thể bao gồm AXI Full, AXI-Lite, và AXI-Stream. Cụ thể, AXI4 Full hỗ trợ truyền Burst với độ dài và kích thước dữ liệu đa dạng. AXI4-Lite, đơn giản hóa chỉ với các giao dịch một từ, giảm thiểu số lượng tín hiệu và đơn giản hóa logic cần thiết cho các thiết kế không cần băng thông lớn. AXI4-Stream được tối ưu hóa cho các ứng dụng truyền dữ liệu tốc độ cao mà không cần truyền địa chỉ, chẳng hạn như truyền video, âm thanh, hoặc dữ liệu từ DMA. Giao thức này không sử dụng các kênh địa chỉ, chỉ tập trung vào kênh dữ liệu với tốc độ cao.
- ✓ AXI5: Trong đặc tả AMBA 5, AXI5 được cập nhật để hỗ trợ TrustZone và các ứng dụng bảo mật cao trên nền tảng Armv8-M. Nó bổ sung các tính năng kiểm soát giao dịch tốt hơn, hỗ trợ các thuộc tính bảo mật nâng cao và cải thiện khả năng quản lý băng thông trong các hệ thống có nhiều bus chủ và tớ.

Hầu hết các Xilinx FPGA hiện đại đều sử dụng chuẩn giao tiếp AXI4 cho việc thiết kế các IP và kết nối các khối chức năng trong hệ thống. AXI4 là một chuẩn giao tiếp linh hoạt và mạnh mẽ, cho phép truyền dữ liệu hiệu quả với băng thông cao, hỗ trợ nhiều loại Burst và khả năng truyền song song. Với ba biến thể chính là AXI4 Full, AXI4-Lite và AXI4-Stream, chuẩn AXI4 phù hợp với nhiều loại ứng dụng khác nhau từ điều khiển ngoại vi đơn giản đến truyền dữ liệu tốc độ cao. Do đó, phần này sẽ tập trung vào việc giới thiệu chi tiết về AXI4 để giúp hiểu rõ hơn cách thức hoạt động, thiết kế, và triển

khai các IP trên Xilinx FPGA, nhằm tối ưu hóa hiệu suất và khả năng mở rộng của hệ thống.

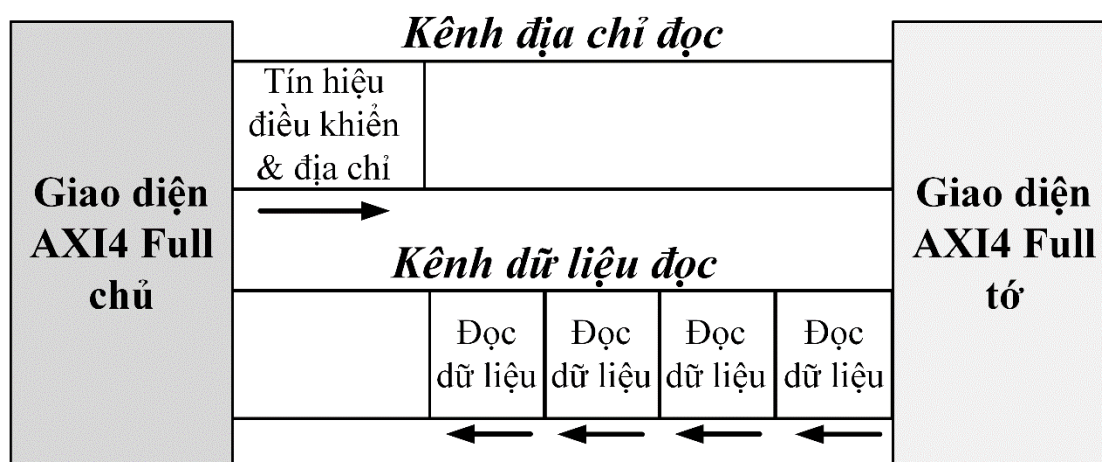
4.5.2. Tín hiệu và kết nối AXI4 Full

Trong phần này, chúng ta sẽ tìm hiểu về tín hiệu và kết nối của giao thức AXI4 Full trong hệ thống AMBA. Cụ thể, AXI4 Full hỗ trợ truyền tải dữ liệu phức tạp với khả năng xử lý nhiều giao dịch đồng thời, giúp tối ưu hóa băng thông và hiệu suất khi kết nối giữa các IP trên FPGA. Giao thức này sử dụng năm kênh độc lập để thực hiện truyền và nhận dữ liệu: kênh địa chỉ ghi, kênh dữ liệu ghi, kênh phản hồi ghi, kênh địa chỉ đọc, và kênh dữ liệu đọc.



Hình 4.19: Quá trình ghi dữ liệu trên AMBA AXI4 Full sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi.

Hình 4.19 mô tả chi tiết quá trình ghi dữ liệu trên giao thức AMBA AXI4 sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi, nhằm đảm bảo tính đồng bộ và hiệu suất cao của hệ thống. Trong quá trình ghi, chủ gửi tín hiệu điều khiển và địa chỉ cần ghi qua kênh địa chỉ ghi để chỉ định vị trí bộ nhớ, đảm bảo dữ liệu sẽ được ghi vào đúng địa chỉ. Tiếp theo, chủ truyền dữ liệu qua kênh dữ liệu ghi, nơi dữ liệu thực tế được chuyển từ chủ đến tớ. Cuối cùng, sau khi tớ ghi dữ liệu xong, kênh phản hồi ghi được sử dụng để tớ gửi phản hồi về trạng thái của giao dịch ghi, giúp chủ xác nhận việc ghi đã hoàn tất và tiếp tục các bước tiếp theo trong quá trình xử lý dữ liệu.



Hình 4.20: Quá trình đọc dữ liệu trên AMBA AXI4 Full sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 4.20 mô tả quá trình đọc dữ liệu trên giao thức AXI4 sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc. Trong quá trình đọc, chủ gửi tín hiệu điều khiển và địa chỉ qua kênh địa chỉ đọc để chỉ định địa chỉ nơi dữ liệu cần đọc. Sau đó, dữ liệu được truyền từ tớ tới chủ qua kênh dữ liệu đọc. Các kênh này cho phép việc truyền dữ liệu diễn ra đồng thời với các giao dịch khác, tối ưu hóa hiệu suất và giảm độ trễ trong hệ thống.

Sự phân chia rõ ràng của các kênh trong AXI4 Full giúp tăng cường khả năng kiểm soát và tối ưu hóa việc truyền tải dữ liệu, đặc biệt khi tích hợp nhiều IP với độ phức tạp cao trên FPGA.

Bảng 4.5 mô tả các tín hiệu được sử dụng trong 5 kênh của giao thức AXI Full, bao gồm các tín hiệu cho kênh địa chỉ ghi, kênh dữ liệu ghi, kênh phản hồi ghi, kênh địa chỉ đọc và kênh dữ liệu đọc. Theo đó, có đến 45 tín hiệu khác nhau được sử dụng trên 5 kênh. Mỗi kênh đóng vai trò quan trọng trong việc truyền tải dữ liệu, địa chỉ, và phản hồi giữa các chủ và tớ trong hệ thống.

Bảng 4.5: Các tín hiệu ở 5 kênh dùng trong AXI Full

Tín hiệu	Chiều tín hiệu	Mô tả
ACLK	Nguồn xung nhịp → tất cả các khối AXI	Tín hiệu xung nhịp chung
ARESETn	Nguồn reset → tất cả các khối AXI	Tín hiệu reset mức thấp chung

<i>Các tín hiệu kênh địa chỉ ghi</i>	AWID	Chủ → Tớ	ID địa chỉ ghi. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu địa chỉ ghi.
	AWADDR	Chủ → Tớ	Địa chỉ ghi. Địa chỉ ghi cung cấp địa chỉ của lần truyền đầu tiên trong một giao dịch Burst.
	AWLEN	Chủ → Tớ	Độ dài Burst. Độ dài Burst cung cấp số lượng truyền chính xác trong một Burst. Thông tin này xác định số lần truyền dữ liệu liên quan đến địa chỉ.
	AWSIZE	Chủ → Tớ	Kích thước Burst. Tín hiệu này chỉ ra kích thước của mỗi lần truyền trong Burst.
	AWBURST	Chủ → Tớ	Loại Burst. Loại Burst và thông tin kích thước xác định cách tính địa chỉ cho mỗi lần truyền trong Burst.
	AWLOCK	Chủ → Tớ	Loại khóa. Cung cấp thông tin bổ sung về đặc tính của lần truyền.
	AWCACHE	Chủ → Tớ	Loại bộ nhớ. Tín hiệu này chỉ ra cách các giao dịch cần tiến hành qua hệ thống.
	AWPROT	Chủ → Tớ	Loại bảo vệ. Tín hiệu này chỉ ra mức độ quyền và an ninh của giao dịch, và liệu giao dịch đó là truy cập dữ liệu hay lệnh.
	AWQOS	Chủ → Tớ	Chất lượng dịch vụ (QoS). Tập định danh QoS cho mỗi lần truyền ghi. Chỉ được triển khai trong AXI4.
	AWREGION	Chủ → Tớ	Định danh vùng. Cho phép một giao diện vật lý đơn trên tớ được sử dụng cho nhiều giao diện logic.

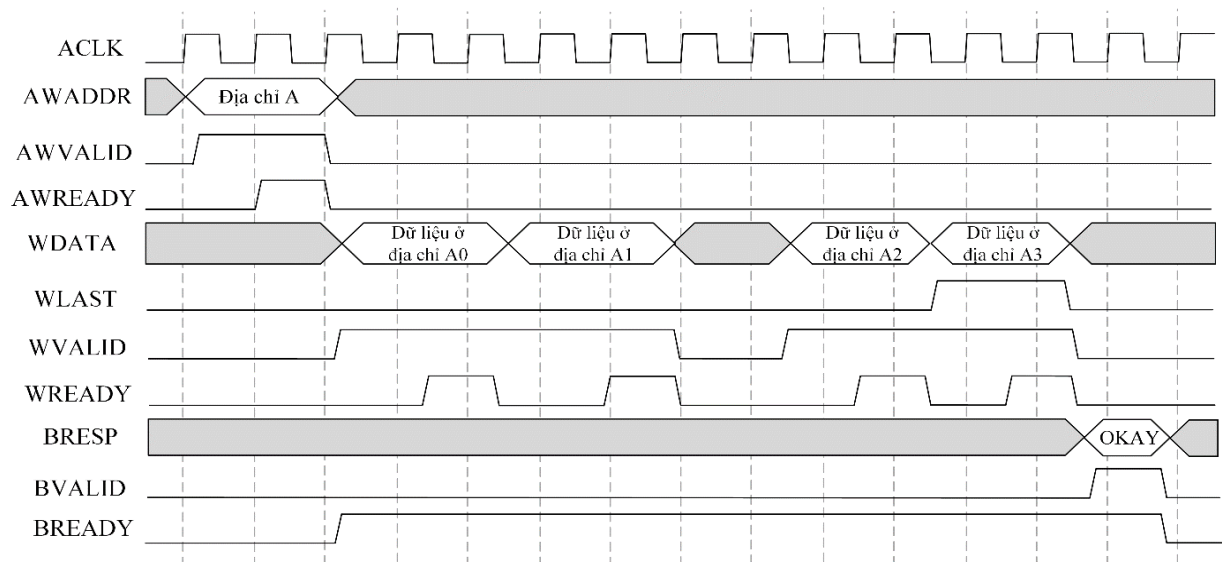
	AWUSER	Chủ → Tó	Tín hiệu người dùng. Tín hiệu do người dùng tùy chọn trong kênh địa chỉ ghi. Chỉ hỗ trợ trong AXI4.
	AWVALID	Chủ → Tó	Địa chỉ ghi hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu địa chỉ ghi hợp lệ và thông tin điều khiển.
	AWREADY	Tó → Chủ	Địa chỉ ghi sẵn sàng. Tín hiệu này chỉ ra rằng tó đã sẵn sàng nhận địa chỉ ghi và thông tin điều khiển liên quan.
<i>Các tín hiệu kênh dữ liệu ghi</i>	WID	Chủ → Tó	ID ghi. Tín hiệu này là ID của việc truyền dữ liệu ghi. Chỉ hỗ trợ AXI3.
	WDATA	Chủ → Tó	Dữ liệu ghi. Dữ liệu có thể điều chỉnh tùy thuộc vào yêu cầu dữ liệu của IP với độ lớn thường là 32, 64, 128, và 256 bit.
	WSTRB	Chủ → Tó	Strobes ghi. Tín hiệu này chỉ ra các byte nào chứa dữ liệu hợp lệ. Mỗi 8 bit của bus dữ liệu ghi có một bit strobe ghi.
	WLAST	Chủ → Tó	Kết thúc ghi. Tín hiệu này chỉ ra lần truyền cuối cùng trong một chuỗi ghi.
	WUSER	Chủ → Tó	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh dữ liệu ghi.
	WVALID	Chủ → Tó	Ghi hợp lệ. Tín hiệu này chỉ ra rằng dữ liệu ghi hợp lệ và strobes có sẵn.
	WREADY	Tó → Chủ	Sẵn sàng ghi. Tín hiệu này chỉ ra rằng tó có thể chấp nhận dữ liệu ghi.

<i>Các tín hiệu kênh phản hồi</i>	BID	Tớ → Chủ	ID phản hồi. Tín hiệu này là thẻ ID của phản hồi ghi.
	BRESP	Tớ → Chủ	Phản hồi ghi. Tín hiệu này chỉ ra trạng thái của giao dịch ghi.
	BUSER	Tớ → Chủ	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh phản hồi ghi. Chỉ hỗ trợ trong AXI4.
	BVALID	Tớ → Chủ	Phản hồi ghi hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu một phản hồi ghi hợp lệ.
	BREADY	Chủ → Tớ	Phản hồi sẵn sàng. Tín hiệu chỉ ra rằng chủ có thể chấp nhận phản hồi ghi.
<i>Các tín hiệu kênh địa chỉ đọc</i>	ARID	Chủ → Tớ	ID địa chỉ đọc. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu địa chỉ đọc.
	ARADDR	Chủ → Tớ	Địa chỉ đọc. Tín hiệu địa chỉ đọc cung cấp địa chỉ của lần chuyển đầu tiên trong một giao dịch đọc dạng Burst.
	ARLEN	Chủ → Tớ	Độ dài Burst. Tín hiệu này chỉ ra số lượng chuyển chính xác trong một Burst.
	ARSIZE	Chủ → Tớ	Kích thước Burst. Tín hiệu chỉ ra kích thước của mỗi lần chuyển trong Burst.
	ARBURST	Chủ → Tớ	Loại Burst. Loại Burst và thông tin kích thước xác định cách thức địa chỉ cho mỗi lần chuyển trong Burst được tính toán.
	ARLOCK	Chủ → Tớ	Loại khóa. Tín hiệu này cung cấp thông tin bổ sung về đặc điểm của lần chuyển.
	ARCACHE	Chủ → Tớ	Loại bộ nhớ. Tín hiệu này chỉ ra cách thức các giao dịch cần phải tiến hành trong hệ thống.

	ARPROT	Chủ → Tớ	Loại bảo vệ. Tín hiệu này chỉ ra mức độ đặc quyền và an ninh của giao dịch, và liệu giao dịch là truy cập dữ liệu hay truy cập lệnh.
	ARQOS	Chủ → Tớ	Chất lượng dịch vụ (QoS). Mã định danh QoS được gửi cho mỗi giao dịch đọc. Chỉ được triển khai trong AXI4.
	ARREGION	Chủ → Tớ	Mã định danh vùng. Cho phép một giao diện vật lý đơn lẻ trên tớ được sử dụng cho nhiều giao diện logic. Chỉ được triển khai trong AXI4.
	ARUSER	Chủ → Tớ	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh địa chỉ đọc. Chỉ hỗ trợ trong AXI4.
	ARVALID	Chủ → Tớ	Địa chỉ đọc hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu địa chỉ đọc hợp lệ và thông tin điều khiển.
	ARREADY	Tớ → Chủ	Địa chỉ đọc sẵn sàng. Tín hiệu này chỉ ra rằng tớ đã sẵn sàng chấp nhận một địa chỉ và các tín hiệu điều khiển liên quan.
	RID	Tớ → Chủ	ID đọc. Tín hiệu này là thẻ nhận dạng cho nhóm tín hiệu dữ liệu đọc được tạo ra bởi tớ
<i>Các tín hiệu kênh dữ liệu đọc</i>	RDATA	Tớ → Chủ	Dữ liệu đọc.
	RRESP	Tớ → Chủ	Phản hồi đọc. Tín hiệu này chỉ ra trạng thái của lần chuyển đọc.
	RLAST	Tớ → Chủ	Dữ liệu đọc cuối cùng. Tín hiệu này chỉ ra lần chuyển cuối cùng trong một Burst đọc.

	RUSER	Tớ → Chủ	Tín hiệu người dùng. Tín hiệu tùy chọn do người dùng định nghĩa trong kênh dữ liệu đọc.
	RVALID	Tớ → Chủ	Dữ liệu đọc hợp lệ. Tín hiệu này chỉ ra rằng kênh đang báo hiệu dữ liệu đọc yêu cầu.
	RREADY	Chủ → Tớ	Dữ liệu đọc sẵn sàng. Tín hiệu này chỉ ra rằng chủ có thể chấp nhận dữ liệu đọc và thông tin phản hồi.

4.5.3. Chi tiết tín hiệu của AXI4 Full trong việc đọc và ghi dữ liệu

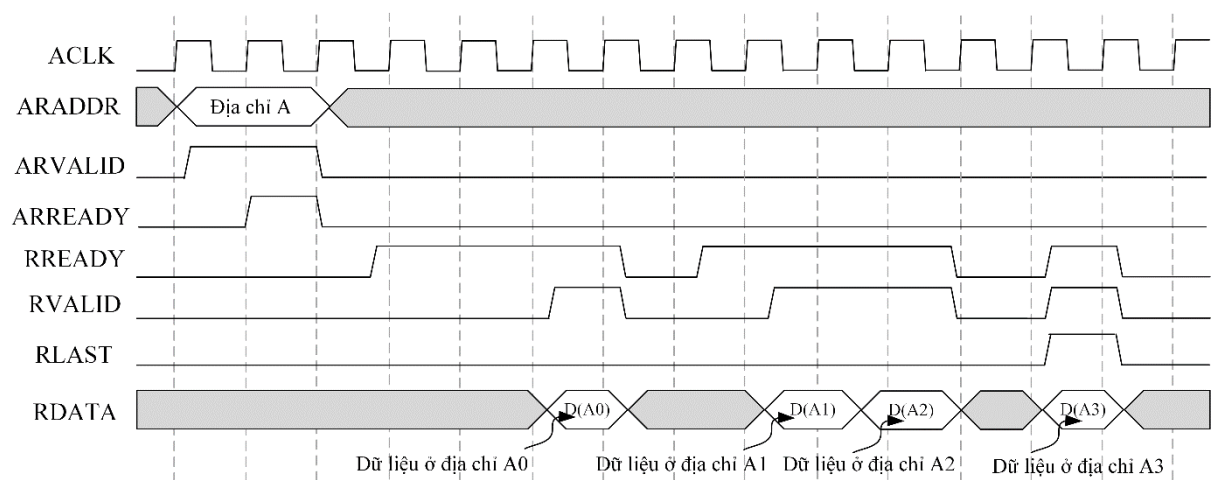


Hình 4.21: Sơ đồ thời gian chi tiết của tín hiệu chính cho ghi dữ liệu với Burst trên kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi.

Trong phần này, chúng ta sẽ tập trung vào việc phân tích chi tiết các tín hiệu của giao thức AXI4 Full liên quan đến các hoạt động đọc và ghi dữ liệu với chế độ Burst. Chế độ Burst trong AXI4 Full cho phép truyền một loạt dữ liệu liên tiếp chỉ với một lệnh địa chỉ duy nhất, giúp cải thiện hiệu suất truyền thông và băng thông dữ liệu.

Hình 4.21 minh họa sơ đồ thời gian chi tiết cho giao dịch ghi dữ liệu trên bus AXI với chế độ truyền dữ liệu Burst, bao gồm ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi. Giao dịch bắt đầu khi chủ gửi địa chỉ và thông tin điều khiển trên kênh địa chỉ ghi (AWADDR, AWVALID, AWREADY). Khi tín hiệu AWVALID và AWREADY đều lên mức cao, địa chỉ được gửi đi và được tớ chấp nhận. Sau đó, chủ

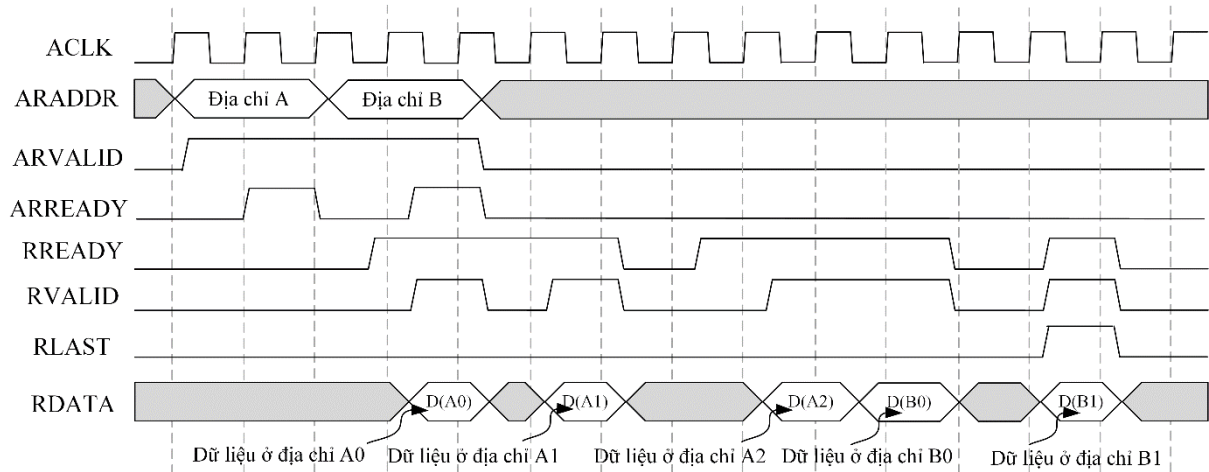
gửi các mục dữ liệu trên kênh dữ liệu ghi (WDATA) kèm theo tín hiệu WVALID để báo rằng dữ liệu trên bus là hợp lệ. Tín hiệu WREADY từ từ cho biết từ đã sẵn sàng nhận dữ liệu. Trong chế độ Burst, dữ liệu được gửi liên tiếp đến khi hết dữ liệu; khi dữ liệu cuối cùng được gửi, tín hiệu WLAST sẽ lên mức cao. Khi tất cả dữ liệu đã được từ chấp nhận, từ sẽ gửi phản hồi trên kênh phản hồi ghi (BVALID, BREADY) để thông báo giao dịch ghi đã hoàn tất. Khi tín hiệu BVALID và BREADY đều lên mức cao, phản hồi ghi "OKAY" được gửi đến chủ, kết thúc quá trình giao dịch ghi. Hình này minh họa sự phối hợp chặt chẽ giữa các tín hiệu điều khiển và dữ liệu trong quá trình truyền thông tin giữa chủ và từ trên bus AXI.



Hình 4.22: Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 4.22 minh họa sơ đồ thời gian chi tiết cho quá trình đọc dữ liệu với chế độ Burst trên bus AXI, tương tự như quá trình ghi dữ liệu đã trình bày trước đó. Trong ví dụ này, chủ sẽ gửi địa chỉ đọc trên kênh địa chỉ đọc (ARADDR) cùng với tín hiệu điều khiển báo hiệu chiều dài và loại của Burst (các tín hiệu này không hiển thị trong hình để đơn giản hóa). Tín hiệu ARVALID được chủ kích hoạt để báo rằng địa chỉ và thông tin điều khiển trên bus là hợp lệ, và khi tín hiệu ARREADY từ từ lên mức cao sau một chu kỳ, địa chỉ đã được từ chấp nhận. Sau khi địa chỉ xuất hiện trên bus địa chỉ, quá trình truyền dữ liệu xảy ra trên kênh dữ liệu đọc (RDATA). Trong quá trình này, từ giữ tín hiệu RVALID ở mức thấp cho đến khi dữ liệu đọc sẵn sàng, và dữ liệu được truyền từ từ đến chủ với các tín hiệu đồng bộ theo từng chu kỳ. Đối với dữ liệu cuối cùng trong chuỗi Burst, tín hiệu RLAST được từ kích hoạt để báo hiệu rằng đây là dữ liệu cuối cùng đang được truyền.

Để tăng cường hiệu quả và tối ưu hóa băng thông trong các hệ thống sử dụng giao thức AXI4 Full, việc thực hiện các giao dịch đọc chồng chéo (overlapping read transactions) là một kỹ thuật quan trọng. Kỹ thuật này cho phép chủ gửi một địa chỉ Burst mới ngay sau khi địa chỉ Burst trước đó đã được chấp nhận. Nhờ đó, nó có thể chuẩn bị xử lý dữ liệu cho Burst tiếp theo trong khi Burst hiện tại vẫn đang diễn ra. Điều này giúp tăng cường hiệu suất tổng thể bằng cách tận dụng khả năng xử lý song song của hệ thống và giảm thiểu thời gian chờ giữa các giao dịch.



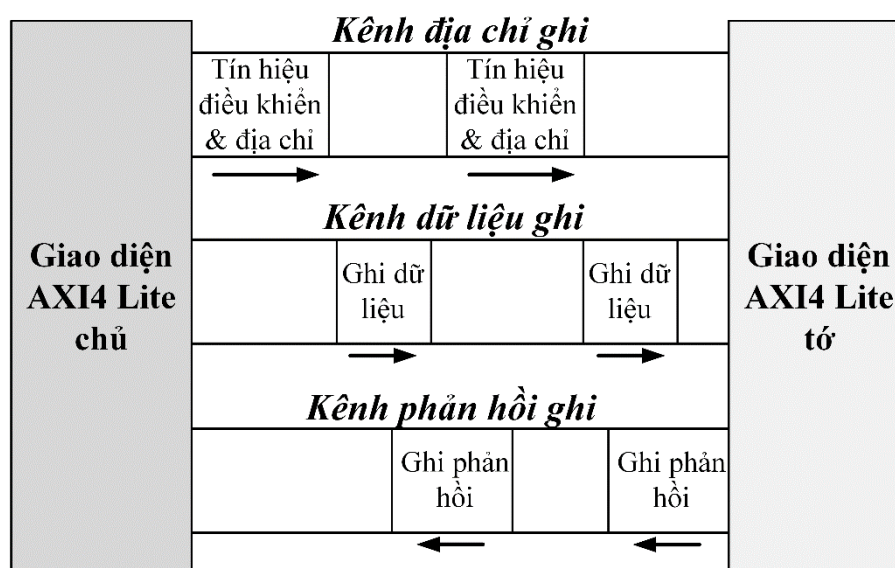
Hình 4.23: Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu chồng chéo với Burst trên kênh địa chỉ đọc và kênh dữ liệu đọc.

Trong Hình 4.23, quá trình đọc chồng chéo được minh họa rõ ràng khi chủ điều khiển địa chỉ Burst thứ hai (địa chỉ B) ngay sau khi địa chỉ Burst đầu tiên (địa chỉ A) được chấp nhận. Khi địa chỉ A được gửi và tín hiệu ARREADY từ đó lên mức cao, địa chỉ A đã được chấp nhận, và ngay sau đó, chủ tiếp tục gửi địa chỉ B. Trong khi dữ liệu từ địa chỉ A được truyền trên kênh dữ liệu đọc (RDATA), nó đồng thời chuẩn bị dữ liệu cho địa chỉ B. Khi dữ liệu tương ứng từ cả hai địa chỉ sẵn sàng, nó sẽ sử dụng tín hiệu RVALID và RLAST để đồng bộ hóa việc truyền dữ liệu cho Burst.

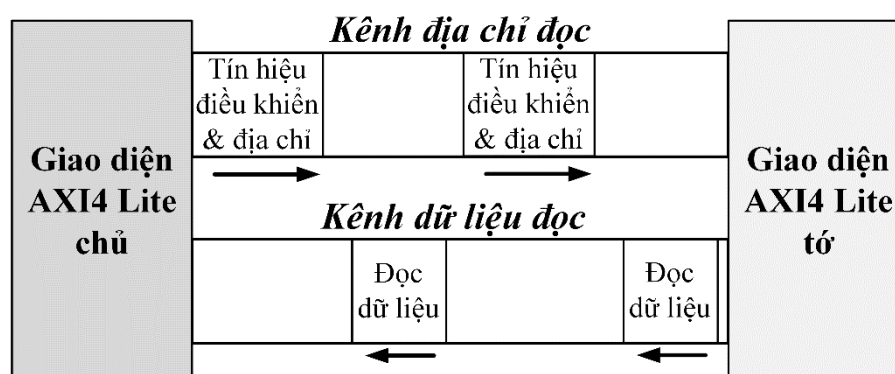
4.5.4. Tín hiệu và kết nối AXI4 Lite

Trong phần này, chúng ta sẽ tìm hiểu về tín hiệu và kết nối của giao thức AXI4-Lite trong hệ thống AMBA. AXI4-Lite là một phiên bản đơn giản hóa của AXI4 Full, được thiết kế cho các ứng dụng không yêu cầu truyền dữ liệu phức tạp hoặc hiệu suất cao. Giao thức này vẫn giữ cấu trúc giao dịch tương tự như AXI4 Full nhưng đơn giản hơn, không hỗ trợ burst, giúp giảm chi phí tài nguyên và độ phức tạp khi triển khai trong các thiết bị nhúng hoặc các IP đơn giản trên FPGA.

Mặc dù giảm bớt độ phức tạp, AXI4-Lite vẫn sử dụng ba kênh chính cho quá trình ghi dữ liệu: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi. Điều này đảm bảo các giao dịch ghi dữ liệu diễn ra mạch lạc và có phản hồi từ tớ.



Hình 4.24: Quá trình ghi dữ liệu trên AMBA AXI4-Lite sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi.



Hình 4.25: Quá trình đọc dữ liệu trên AMBA AXI4-Lite sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 4.24: Quá trình ghi dữ liệu trên AXI4-Lite sử dụng ba kênh chính: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi. Trong quá trình này, chủ bắt đầu bằng cách gửi tín hiệu điều khiển và địa chỉ qua kênh địa chỉ ghi để chỉ định địa chỉ nơi dữ liệu sẽ được ghi vào. Sau đó, dữ liệu được truyền từ chủ đến tớ qua kênh dữ liệu ghi. Khi dữ liệu đã được ghi xong, tớ sẽ gửi tín hiệu phản hồi về trạng thái của giao dịch qua kênh phản hồi ghi.

Hình 4.25: Quá trình đọc dữ liệu trên AXI4-Lite sử dụng hai kênh chính: kênh địa chỉ đọc và kênh dữ liệu đọc. Chủ gửi tín hiệu điều khiển và địa chỉ qua kênh địa chỉ đọc

để chỉ định địa chỉ nơi dữ liệu cần đọc. Dữ liệu được đọc từ tớ và truyền về chủ qua kênh dữ liệu đọc. Việc sử dụng hai kênh trong AXI4-Lite cho phép quá trình đọc dữ liệu diễn ra đồng thời với các giao dịch khác, giúp tối ưu hóa hiệu suất.

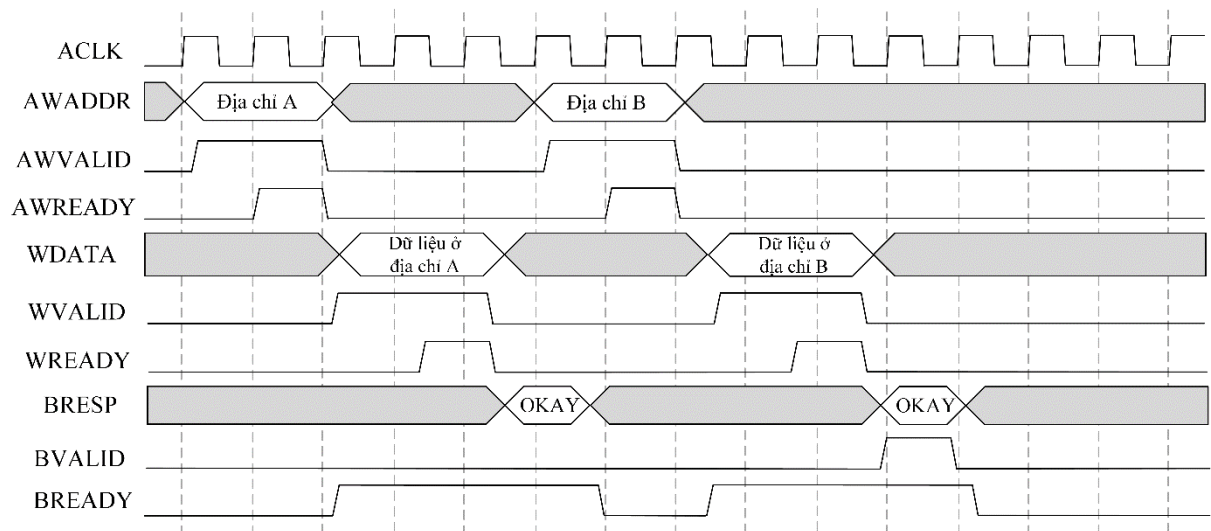
Bảng 4.6 mô tả các tín hiệu được sử dụng trong 5 kênh của giao thức AXI Lite, bao gồm các tín hiệu cho kênh địa chỉ ghi, kênh dữ liệu ghi, kênh phản hồi ghi, kênh địa chỉ đọc, và kênh dữ liệu đọc. Nội dung về chiều tín hiệu và mô tả của các tín hiệu trong bảng này tương tự như đã trình bày trong Bảng 4.5. Do đó, Bảng 4.6 chỉ liệt kê tên các tín hiệu sử dụng trong giao thức AXI Lite, không đi sâu vào chi tiết mô tả từng tín hiệu. AXI Lite, với thiết kế đơn giản hơn AXI Full, chỉ sử dụng 19 tín hiệu khác nhau trên 5 kênh này. Mỗi kênh trong AXI Lite đảm nhận vai trò cụ thể trong việc truyền tải thông tin điều khiển, địa chỉ, và dữ liệu giữa các chủ và tớ trong hệ thống. So với AXI Full, số lượng tín hiệu ít hơn giúp cho giao thức AXI Lite dễ dàng triển khai và tối ưu hóa hơn trong các ứng dụng yêu cầu đơn giản hóa kết nối và giao tiếp giữa các thành phần trong hệ thống.

Bảng 4.6: Các tín hiệu ở 5 kênh dùng trong AXI Lite

Kênh	Tên tín hiệu
Kênh địa chỉ ghi	AWVALID, AWREADY, AWADDR, AWPROT
Kênh dữ liệu ghi	WVALID, WREADY, WDATA,WSTRB
Kênh phản hồi ghi	BVALID, BREADY, BRESP
Kênh địa chỉ đọc	ARVALID, ARREADY, ARADDR, ARPROT
Kênh dữ liệu đọc	RVALID, RREADY, RDATA, RRESP

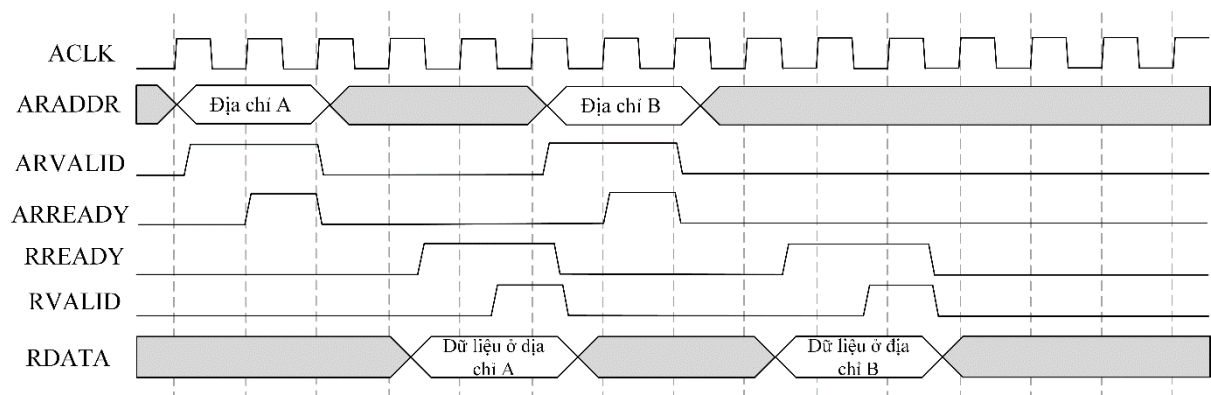
4.5.5. Chi tiết tín hiệu của AXI4-Lite trong việc đọc và ghi dữ liệu

Trong phần này, chúng ta sẽ tập trung vào việc phân tích chi tiết các tín hiệu của giao thức AXI4-Lite liên quan đến các hoạt động đọc và ghi dữ liệu. Không giống như AXI4 Full, AXI4-Lite được thiết kế cho các ứng dụng yêu cầu truyền thông đơn giản và không cần độ phức tạp của chế độ Burst. Trong AXI4-Lite, mỗi giao dịch đọc hoặc ghi dữ liệu chỉ liên quan đến một địa chỉ duy nhất và không có khả năng thực hiện các giao dịch Burst.



Hình 4.26: Sơ đồ thời gian chi tiết của tín hiệu chính cho ghi dữ liệu của AXI4-Lite trên kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi.

Hình 4.26 minh họa quá trình ghi dữ liệu trên giao thức AXI4-Lite, sử dụng ba kênh: kênh địa chỉ ghi, kênh dữ liệu ghi, và kênh phản hồi ghi. Quy trình ghi dữ liệu bắt đầu khi chủ gửi địa chỉ ghi (AWADDR) và tín hiệu hợp lệ địa chỉ ghi (AWVALID) trên kênh địa chỉ ghi. Tờ xác nhận nhận địa chỉ khi tín hiệu sẵn sàng địa chỉ ghi (AWREADY) được kích hoạt. Sau đó, dữ liệu ghi (WDATA) được truyền từ chủ sang tớ thông qua kênh dữ liệu ghi khi tín hiệu hợp lệ dữ liệu ghi (WVALID) được kích hoạt và tớ sẵn sàng nhận dữ liệu (WREADY). Khi quá trình ghi hoàn tất, tớ gửi tín hiệu phản hồi ghi (BRESP) qua kênh phản hồi ghi để báo hiệu trạng thái của giao dịch, thường là "OKAY", cho biết quá trình ghi đã thành công. Tín hiệu hợp lệ phản hồi ghi (BVALID) và tín hiệu sẵn sàng phản hồi ghi (BREADY) cũng được sử dụng để đồng bộ hóa việc truyền tải phản hồi.



Hình 4.27: Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu của AXI4-Lite trên kênh địa chỉ đọc và kênh dữ liệu đọc.

Hình 4.27 minh họa quá trình đọc dữ liệu trên giao thức AXI4-Lite, sử dụng hai kênh: kênh địa chỉ đọc và kênh dữ liệu đọc. Quá trình đọc bắt đầu khi chủ gửi địa chỉ đọc (ARADDR) và tín hiệu hợp lệ địa chỉ đọc (ARVALID) trên kênh địa chỉ đọc. Tờ xác nhận đã nhận địa chỉ khi tín hiệu sẵn sàng địa chỉ đọc (ARREADY) được kích hoạt. Sau khi nhận địa chỉ, dữ liệu đọc (RDATA) được truyền từ tờ tới chủ qua kênh dữ liệu đọc khi tín hiệu hợp lệ dữ liệu đọc (RVALID) được kích hoạt và chủ sẵn sàng nhận dữ liệu (RREADY).

4.5.6. Tín hiệu và kết nối AXI4 Stream

Trong phần này, chúng ta sẽ tìm hiểu về tín hiệu và kết nối của giao thức AXI4 Stream trong hệ thống AMBA. AXI4 Stream là một biến thể của giao thức AXI trong hệ thống AMBA, được thiết kế để truyền dữ liệu liên tục mà không cần địa chỉ hoặc sự điều khiển phức tạp như AXI4 Full hoặc AXI4 Lite. Giao thức này chủ yếu được sử dụng cho các ứng dụng đòi hỏi truyền dữ liệu lớn như video, âm thanh, hoặc dữ liệu sensor trong các thiết bị xử lý tín hiệu số (DSP) và FPGA. Khác với AXI4 Full và AXI4 Lite, AXI4 Stream không phân chia thành 5 kênh riêng biệt (địa chỉ đọc, địa chỉ ghi, dữ liệu đọc, dữ liệu ghi, và phản hồi ghi). Thay vào đó, nó chỉ có một kênh duy nhất cho truyền dữ liệu.

Bảng 4.7: Các tín hiệu trong AXI Stream

Tín hiệu	Chiều tín hiệu	Mô tả
ACLK	Nguồn xung nhịp → tất cả các khối AXI Stream	Tín hiệu xung nhịp chung
ARESETn	Nguồn reset → tất cả các khối AXI Stream	Tín hiệu reset mức thấp chung
TVALID	Chủ → Tờ hoặc (Tờ → Chủ)	TVALID chỉ ra rằng Chủ đang truyền dữ liệu hợp lệ. Một lần truyền xảy ra khi cả TVALID và TREADY đều bằng 1.
TREADY	Tờ → Chủ hoặc (Chủ → Tờ)	TREADY chỉ ra rằng Tờ có thể chấp nhận dữ liệu.

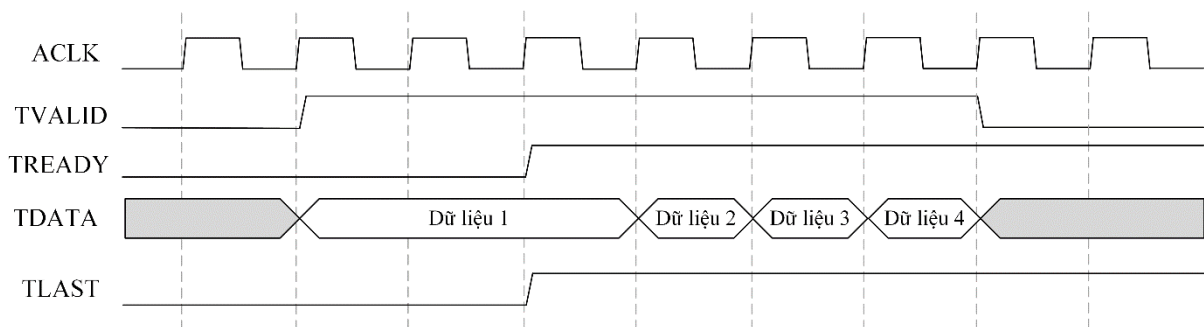
TDATA	Chủ → Tớ hoặc (Tớ → Chủ)	TDATA là tải trọng chính được sử dụng để cung cấp dữ liệu đi qua giao diện. Chiều rộng bit phải là số nguyên của byte và được khuyến nghị là 8, 16, 32, 64, 128, 256, 512 hoặc 1024 bit.
TSTRB	Chủ → Tớ hoặc (Tớ → Chủ)	TSTRB là tín hiệu byte qualifier chỉ ra nội dung của byte liên quan của TDATA được xử lý như byte dữ liệu hay byte vị trí.
TKEEP	Chủ → Tớ hoặc (Tớ → Chủ)	TKEEP là tín hiệu byte qualifier chỉ ra nội dung của byte liên quan của TDATA được xử lý như một phần của luồng dữ liệu.
TLAST	Chủ → Tớ hoặc (Tớ → Chủ)	TLAST chỉ ra ranh giới của một gói dữ liệu.
TID	Chủ → Tớ hoặc (Tớ → Chủ)	TID là tín hiệu định danh luồng dữ liệu. Chiều rộng bit được khuyến nghị không quá 8.
TDEST	Chủ → Tớ hoặc (Tớ → Chủ)	TDEST cung cấp thông tin định tuyến cho luồng dữ liệu. Chiều rộng bit được khuyến nghị không quá 8.
TUSER	Chủ → Tớ hoặc (Tớ → Chủ)	TUSER là thông tin sideband do người dùng định nghĩa có thể được truyền kèm theo luồng dữ liệu. Chiều rộng bit được khuyến nghị là bội số nguyên của chiều rộng bit của TSTRB.
TWAKEUP	Chủ → Tớ hoặc (Tớ → Chủ)	TWAKEUP xác định bất kỳ hoạt động nào liên quan đến giao diện AXI-Stream.

Bảng 4.7 liệt kê các tín hiệu được sử dụng trong giao thức AXI Stream, giúp điều phối quá trình truyền dữ liệu giữa các thành phần trong hệ thống. Các tín hiệu như ACLK và ARESETn đóng vai trò quan trọng trong việc cung cấp xung nhịp đồng bộ và khởi tạo lại trạng thái của hệ thống. TVALID và TREADY có thể sử dụng từ chủ sang tớ hoặc tớ sang chủ, điều phối quá trình truyền dữ liệu và đảm bảo chỉ những dữ liệu hợp

lệ được truyền. TDATA chứa dữ liệu thực tế được truyền đi, trong khi các tín hiệu như TSTRB và TKEEP xác định nội dung byte của dữ liệu và đảm bảo việc truyền tải dữ liệu đúng cách. Các tín hiệu bổ sung như TLAST, TID, TDEST, TUSER, và TWAKEUP có thể truyền từ chủ sang tớ hoặc ngược lại, cung cấp các thông tin bổ sung để xác định ranh giới dữ liệu, định danh luồng dữ liệu, và truyền thông tin phụ trợ, hỗ trợ tối ưu hóa quá trình giao tiếp và định tuyến dữ liệu qua hệ thống.

4.5.7. Chi tiết tín hiệu của AXI4-Stream trong việc ghi dữ liệu

Trong phần này, chúng ta sẽ tập trung vào việc phân tích chi tiết các tín hiệu của giao thức AXI4-Stream liên quan đến các hoạt động truyền dữ liệu. Khác với AXI4 Full và AXI4 Lite, AXI4-Stream được thiết kế chuyên biệt cho các ứng dụng truyền dữ liệu liên tục, nơi dữ liệu được chuyển đi dưới dạng một luồng liên tục mà không có khái niệm về địa chỉ bộ nhớ. Điều quan trọng cần nhấn mạnh là giao thức AXI4-Stream chỉ hỗ trợ ghi dữ liệu và dữ liệu chỉ được truyền từ chủ sang tớ. Tuy nhiên, chủ và tớ có thể thay phiên nhau trong các giao dịch, ví dụ như thiết bị A là chủ để ghi dữ liệu cho tớ B, sau đó B sẽ trở thành chủ để ghi dữ liệu ngược lại cho A là tớ. Quá trình này cho phép luồng dữ liệu liên tục và linh hoạt, tối ưu hóa hiệu suất truyền tải dữ liệu trong hệ thống. Giao thức AXI4-Stream hỗ trợ truyền dữ liệu một chiều từ chủ sang tớ mà không cần phải quản lý địa chỉ bộ nhớ hoặc các giao dịch phản hồi phức tạp.



Hình 4.28: Sơ đồ thời gian chi tiết của tín hiệu chính cho ghi dữ liệu của AXI4-Stream.

Hình 4.28 minh họa quá trình ghi dữ liệu của giao thức AXI4-Stream, một giao thức truyền dữ liệu hiệu quả trong hệ thống AMBA. Trong quá trình này, tín hiệu ACLK đóng vai trò xung nhịp đồng bộ hóa toàn bộ hoạt động. Tín hiệu TVALID chỉ ra rằng dữ liệu trên đường TDATA hiện tại là hợp lệ và sẵn sàng để truyền. Khi cả hai tín hiệu TVALID và TREADY đều được kích hoạt (ở mức cao), dữ liệu sẽ được truyền từ chủ sang tớ. Trong quá trình truyền, dữ liệu được đóng gói trên đường TDATA và có thể bao gồm nhiều từ dữ liệu liên tiếp, như minh

họa trong hình với các từ "Dữ liệu 1", "Dữ liệu 2", "Dữ liệu 3", và "Dữ liệu 4". Tín hiệu TLAST được kích hoạt để chỉ ra rằng từ dữ liệu hiện tại là từ cuối cùng trong một gói dữ liệu, đánh dấu kết thúc của quá trình truyền.

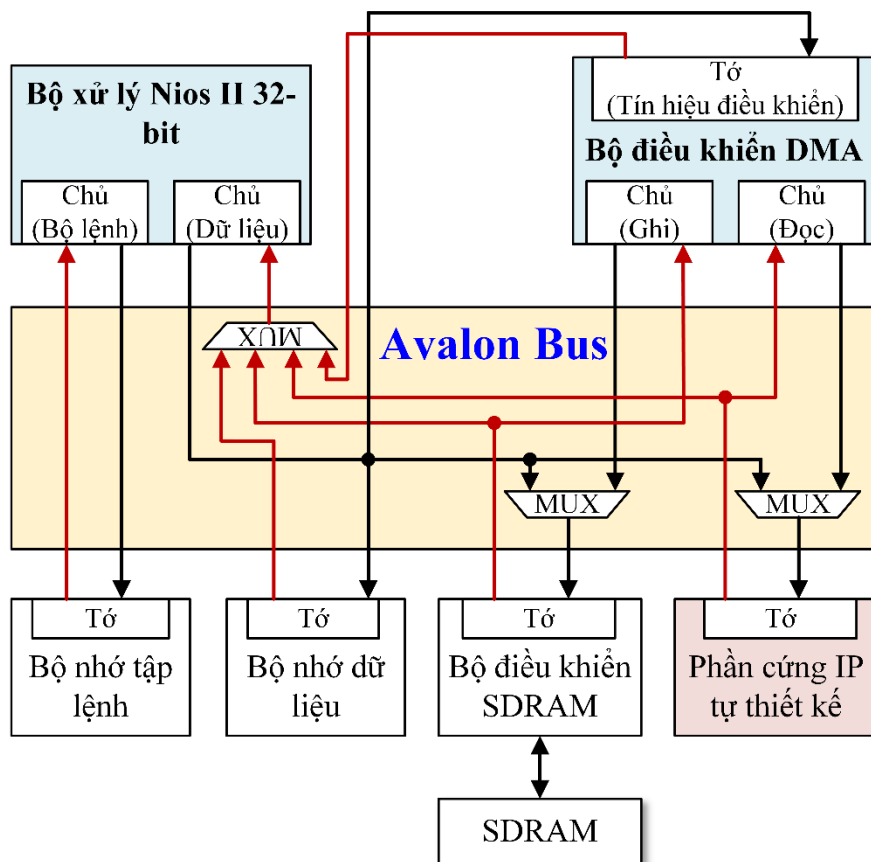
4.6. Hệ thống bus Avalon

4.6.1. Giới thiệu về hệ thống bus Avalon

Hệ thống bus Avalon là một kiến trúc bus đồng bộ đơn giản được thiết kế để kết nối các bộ xử lý và thiết bị ngoại vi trong các hệ thống SoC. Avalon bus giúp liên kết các thành phần như bộ xử lý, bộ nhớ và các thiết bị ngoại vi với giao diện đồng bộ, mang đến sự thuận tiện và hiệu quả trong thiết kế các hệ thống nhúng. Hệ thống bus Avalon được sử dụng rộng rãi trong các FPGA của Intel (trước đây là Altera), đặc biệt trong các dòng FPGA như Cyclone và Stratix. Avalon bus là lựa chọn chính trong các thiết kế SoC của Intel, giúp kết nối các bộ xử lý, bộ nhớ và thiết bị ngoại vi với nhau.

Ghi chú:

↔ Ghi dữ liệu và tín hiệu điều khiển ↔ Đọc dữ liệu



Hình 4.29: Sơ đồ khối mô-đun bus Avalon trong hệ thống vi dụ.

Hệ thống bus Avalon hỗ trợ các giao dịch giữa các thiết bị chủ và tớ thông qua các chu kỳ bus với kích thước dữ liệu có thể là byte (8-bit), half-word (16-bit) hoặc word (32-bit). Sau mỗi giao dịch, bus sẽ sẵn sàng cho một giao dịch mới, có thể là giữa các cặp chủ-tớ tương tự hoặc giữa các cặp chủ và tớ khác. Điều này giúp tối ưu hóa hiệu suất và băng thông trong các hệ thống có nhiều thiết bị hoạt động đồng thời. Avalon bus cũng hỗ trợ các giao thức như truyền tải dữ liệu theo luồng và các giao dịch với độ trễ thấp, giúp tăng hiệu quả truyền tải trong các ứng dụng đòi hỏi băng thông cao. Hình 4.29 mô tả sơ đồ khối của mô-đun bus Avalon trong một hệ thống ví dụ, bao gồm bộ xử lý Nios II 32-bit, bộ điều khiển DMA, bộ nhớ SDRAM và các thiết bị ngoại vi kết nối với bus Avalon. Các thiết bị này bao gồm các thiết bị chủ và tớ, với các giao dịch giữa các thiết bị được quản lý và điều phối qua bus Avalon, tối ưu hóa việc chia sẻ dữ liệu và tín hiệu điều khiển trong hệ thống. Cấu trúc này giúp hệ thống có thể thực hiện các giao dịch hiệu quả và liên tục mà không gặp phải sự cố về hiệu suất, đặc biệt là trong các ứng dụng yêu cầu băng thông cao và tốc độ truyền tải dữ liệu lớn.

Một điểm mạnh của bus Avalon là giao thức dễ hiểu và tối ưu hóa việc sử dụng tài nguyên bus. Điều này giúp cho việc chia sẻ dữ liệu giữa các thiết bị trở nên hiệu quả, dễ dàng triển khai và tích hợp trong các hệ thống nhúng.

So với hệ thống bus AXI, Avalon bus có một số đặc điểm khác biệt:

- ✓ Đơn giản: Avalon bus có giao thức đơn giản hơn AXI, điều này giúp dễ dàng triển khai và quản lý trong các hệ thống nhỏ hoặc khi yêu cầu hệ thống bus không quá phức tạp.
- ✓ Tính đồng bộ: Avalon bus hoàn toàn đồng bộ, điều này giúp dễ dàng tích hợp với các hệ thống khác trong một FPGA mà không gặp phải vấn đề đồng bộ hóa phức tạp. Trong khi đó, AXI hỗ trợ nhiều kênh giao dịch bất đồng bộ, giúp các giao dịch có thể diễn ra song song mà không cần phải chờ đợi..
- ✓ Độ phức tạp trong cấu trúc: AXI có khả năng hỗ trợ các giao dịch phức tạp và các hệ thống đa chủ với độ phức tạp cao hơn, giúp đáp ứng tốt hơn với các ứng dụng yêu cầu băng thông và hiệu suất cao. Avalon, mặc dù có sự hỗ trợ cho nhiều giao dịch đồng thời, nhưng vẫn đơn giản hơn trong việc cấu hình và triển khai.

Mặc dù có nhiều lợi thế, Avalon cũng có một vài điểm yếu so với AXI Bus như sau:

- ✓ Thiếu tính linh hoạt cao: Mặc dù Avalon hỗ trợ nhiều giao dịch, nhưng tính linh hoạt và khả năng mở rộng của nó không bằng AXI, đặc biệt là trong các hệ thống có nhiều chủ và tớ hoặc yêu cầu băng thông rất cao.
- ✓ Không hỗ trợ burst dài như AXI: AXI có khả năng hỗ trợ burst dài hơn và có khả năng truyền dữ liệu với băng thông cao hơn, rất phù hợp với các ứng dụng yêu cầu truyền tải dữ liệu lớn và nhanh.

Tóm lại, Avalon bus là một giải pháp hiệu quả cho các hệ thống đơn giản và linh hoạt, trong khi AXI bus phù hợp với các hệ thống phức tạp và yêu cầu băng thông cao.

4.6.2. Tín hiệu và kết nối Avalon Bus

Trong phần này, chúng ta sẽ tìm hiểu về tín hiệu và kết nối của giao thức Avalon Bus. Avalon Bus là một kiến trúc bus đồng bộ đơn giản, được thiết kế để kết nối các bộ xử lý và thiết bị ngoại vi trong các hệ thống SoC. Các tín hiệu trong Avalon Bus có vai trò quan trọng trong việc điều phối và quản lý giao tiếp giữa các thiết bị chủ và tớ, giúp tối đảm bảo sự đồng bộ giữa các thiết bị trong hệ thống.

Bảng 4.8: Các tín hiệu trong Avalon Bus

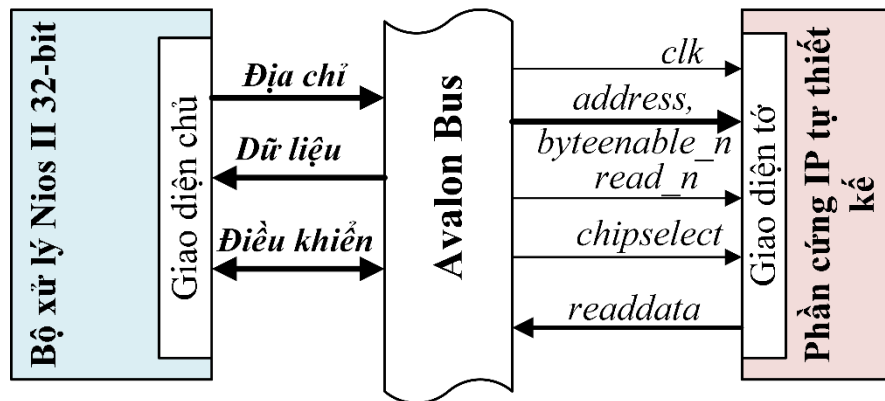
Tín hiệu	Chiều tín hiệu	Mô tả
<i>clk</i>	Nguồn xung nhịp → tất cả các khối Avalon	Tín hiệu xung clock toàn cục cho mô-đun hệ thống và mô-đun bus Avalon. Tất cả các giao dịch bus đều đồng bộ với <i>clk</i> . Chỉ có các cổng tớ không đồng bộ mới có thể bỏ qua <i>clk</i> .
<i>reset</i>	Nguồn reset → tất cả các thiết bị ngoại vi	Tín hiệu thiết lập lại toàn cục. Triển khai cụ thể cho thiết bị ngoại vi
<i>chipselect</i>	Chủ → Tớ	Tín hiệu 1 bit chọn chip đến tớ. Cổng tớ phải bỏ qua tất cả các đầu vào tín hiệu Avalon khác trừ khi <i>chipselect</i> được khẳng định.
<i>address</i>	Chủ → Tớ	Dòng địa chỉ của mô-đun bus Avalon. Tín hiệu này có độ rộng từ 1 đến 32 bit.

<i>begintransfer</i>	Chủ → Tớ	Tín hiệu được kích hoạt trong chu kỳ bus đầu tiên của mỗi giao dịch mới trên bus Avalon. Tín hiệu này có độ rộng 1 bit.
<i>byteenable</i>	Chủ → Tớ	Tín hiệu byte-enable để kích hoạt các byte lane cụ thể trong các giao dịch với bộ nhớ có độ rộng lớn hơn 8 bit. Tín hiệu này có độ rộng 0, 2 hoặc 4 bit.
<i>read</i>	Chủ → Tớ	Tín hiệu yêu cầu đọc gửi đến tớ. Không cần thiết nếu tớ không bao giờ xuất dữ liệu ra chủ. Nếu sử dụng, tín hiệu <i>readdata</i> bắt buộc phải được dùng. Tín hiệu này có độ rộng 1 bit.
<i>readdata</i>	Tớ → Chủ	Đường dữ liệu đến mô-đun bus Avalon để chuyển dữ liệu đọc. Không bắt buộc nếu tớ không bao giờ xuất dữ liệu đến chủ. Nếu sử dụng, cũng phải sử dụng tín hiệu <i>read</i> . Tín hiệu này có độ rộng từ 1 đến 32 bit.
<i>write</i>	Chủ → Tớ	Ghi tín hiệu yêu cầu vào tớ. Không bắt buộc nếu tớ không bao giờ nhận dữ liệu từ chủ. Nếu sử dụng, tín hiệu <i>writedata</i> cũng phải được sử dụng. Tín hiệu này có độ rộng 1 bit.
<i>writedata</i>	Chủ → Tớ	Đường dữ liệu từ mô-đun bus Avalon để chuyển dữ liệu ghi. Không bắt buộc nếu tớ không bao giờ nhận dữ liệu từ chủ. Nếu sử dụng, cũng phải sử dụng tín hiệu ghi. Tín hiệu này có độ rộng 1 đến 32 bit.
<i>readdatavalid</i>	Tớ → Chủ	Chỉ được sử dụng bởi các tớ có độ trễ thay đổi. Đánh dấu cạnh lên xung clock

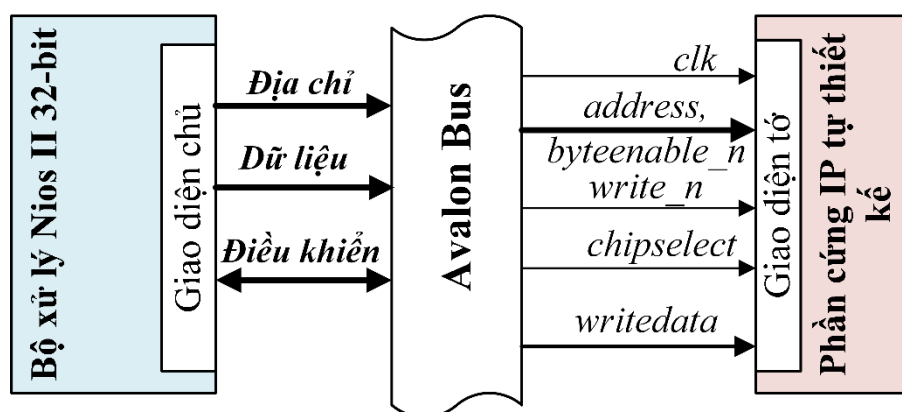
		khi tó khẳng định <i>readdata</i> hợp lệ. Tín hiệu này có độ rộng 1 bit.
<i>waitrequest</i>	Tó → Chủ	Được sử dụng để dừng mô-đun bus Avalon khi cổng phụ không thể phản hồi ngay lập tức. Tín hiệu này có độ rộng 1 bit.
<i>readyfordata</i>	Tó → Chủ	Tín hiệu cho việc truyền dữ liệu trực tuyến. Chỉ ra rằng tó trực tuyến có thể nhận dữ liệu. Tín hiệu này có độ rộng 1 bit.
<i>dataavailable</i>	Tó → Chủ	Tín hiệu cho việc truyền dữ liệu trực tuyến. Chỉ ra rằng tó truyền dữ liệu có sẵn. Tín hiệu này có độ rộng 1 bit.
<i>endofpacket</i>	Tó → Chủ	Tín hiệu cho truyền phát trực tuyến. Có thể được sử dụng để chỉ ra tình trạng “kết thúc gói tin” cho cổng chính. Triển khai cụ thể cho từng thiết bị ngoại vi. Tín hiệu này có độ rộng 1 bit.
<i>irq</i>	Tó → Chủ	Yêu cầu ngắt. Tó khẳng định <i>irq</i> khi nó cần được dùng bởi chủ. Tín hiệu này có độ rộng 1 bit.
<i>irqnumber</i>	Tó → Chủ	Mức độ ưu tiên ngắt của cổng tó ngắt. Giá trị thấp hơn có mức độ ưu tiên cao hơn. Tín hiệu này có độ rộng 6 bit.
<i>resetrequest</i>	Tó → Chủ	Tín hiệu đặt lại cho phép thiết bị ngoại vi đặt lại toàn bộ mô-đun hệ thống. Tín hiệu này có độ rộng 1 bit.
<i>flush</i>	Chủ → Tó	Tín hiệu cho các chuyển giao đọc có độ trễ. Chủ có thể xóa bất kỳ chuyển giao đọc tiềm ẩn nào đang chờ xử lý bằng

		cách tín hiệu <i>flush</i> . Tín hiệu này có độ rộng 1 bit.
--	--	---

Bảng 4.8 liệt kê các tín hiệu trong Avalon Bus với các mô tả chi tiết về chức năng và vai trò của mỗi tín hiệu. Các tín hiệu này được phân thành các nhóm chính như tín hiệu xung clock (*clk*), tín hiệu reset (*reset*), tín hiệu chọn chip (*chipselect*), tín hiệu địa chỉ (*address*), và các tín hiệu dữ liệu (*read*, *write*, *writedata*, *readdata*), cùng với một số tín hiệu hỗ trợ khác. Tín hiệu *clk* là nguồn xung nhịp toàn cục, giúp đồng bộ hóa toàn bộ hệ thống và đảm bảo các giao dịch trên bus Avalon diễn ra một cách chính xác. Tín hiệu *reset* được sử dụng để thiết lập lại toàn bộ hệ thống, đảm bảo các thiết bị ngoại vi bắt đầu trong trạng thái mặc định khi cần thiết. Tín hiệu *chipselect* giúp chọn chip tớ trong một giao dịch Avalon, giúp xác định thiết bị nhận dữ liệu và điều khiển các giao dịch. Tín hiệu *address* xác định địa chỉ bộ nhớ hoặc thiết bị ngoại vi trong hệ thống, là cơ sở cho việc định tuyến dữ liệu đến đúng nơi cần thiết. Tín hiệu *begintransfer* đánh dấu sự bắt đầu của một giao dịch mới trên bus Avalon. Các tín hiệu *read* và *write* lần lượt yêu cầu đọc và ghi dữ liệu giữa chủ và tớ, trong khi *writedata* và *readdata* chuyển dữ liệu vào và ra từ các thiết bị. Bên cạnh đó, Avalon Bus còn sử dụng các tín hiệu như *byteenable* để điều chỉnh byte lane trong các giao dịch với bộ nhớ có độ rộng lớn hơn 8 bit, *waitrequest* để tạm dừng giao dịch khi cần thiết, và *readyfordata* cùng *dataavailable* để thông báo liệu dữ liệu đã sẵn sàng để truyền tải. Các tín hiệu *endofpacket*, *irq*, và *irqnumber* hỗ trợ việc truyền dữ liệu, xử lý ngắt và quản lý các sự kiện ngoại lệ trong hệ thống. Các tín hiệu này không chỉ giúp điều phối quá trình truyền tải dữ liệu mà còn đảm bảo tính đồng bộ và hiệu suất trong giao tiếp giữa các thiết bị.



Hình 4.30: Sơ đồ sử dụng Avalon Bus cơ bản cho quá trình đọc để giao tiếp giữa bộ xử lý Nios II và phần cứng IP tự thiết kế.



Hình 4.31: Sơ đồ sử dụng Avalon Bus cơ bản cho quá trình ghi để giao tiếp giữa bộ xử lý Nios II và phần cứng IP tự thiết kế.

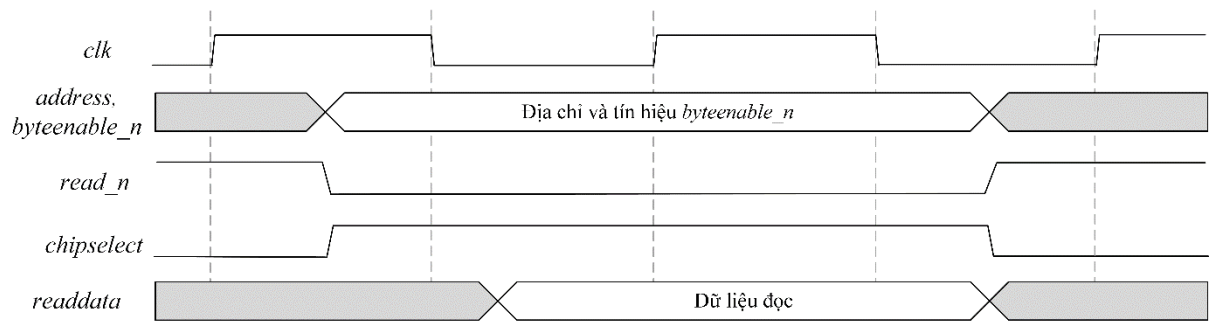
Giả sử trong một hệ thống cơ bản giao tiếp giữa bộ xử lý Nios II và phần cứng IP tự thiết kế, như thể hiện trong Hình 4.30 và Hình 4.31, các tín hiệu trong Avalon Bus sẽ hoạt động như sau:

Hình 4.30 mô tả quá trình đọc dữ liệu từ IP tự thiết kế (tớ) về bộ xử lý Nios II (chủ) thông qua bus Avalon. Cụ thể, chủ sẽ yêu cầu dữ liệu từ tớ thông qua tín hiệu *address* để xác định địa chỉ của bộ nhớ hoặc thiết bị ngoại vi cần đọc dữ liệu. Khi tín hiệu *read* được kích hoạt, thiết bị tớ nhận biết yêu cầu và truyền dữ liệu qua tín hiệu *readdata* về cho chủ. Tín hiệu *byteenable* được sử dụng khi có yêu cầu kích hoạt các byte lane cụ thể trong các giao dịch với bộ nhớ có độ rộng lớn hơn 8 bit. Hình 4.31 mô tả quá trình ghi dữ liệu từ bộ xử lý Nios II (chủ) sang IP tự thiết kế (tớ) thông qua bus Avalon. Tín hiệu *writedata* chứa dữ liệu mà chủ muốn ghi vào tớ. Đồng thời, tín hiệu *chipselct* được sử dụng để chọn chip tớ, đảm bảo rằng tín hiệu ghi và dữ liệu chỉ có hiệu lực khi thiết bị tớ đã được chọn. Tín hiệu *write* chỉ ra yêu cầu ghi dữ liệu và *byteenable* cho phép chủ chọn byte lane để ghi dữ liệu vào thiết bị tớ.

Nhìn chung, Avalon Bus có số lượng tín hiệu ít và đơn giản, nhưng vẫn đóng vai trò quan trọng trong việc đảm bảo sự đồng bộ và hiệu quả trong quá trình truyền dữ liệu giữa bộ xử lý Nios II và phần cứng IP tự thiết kế.

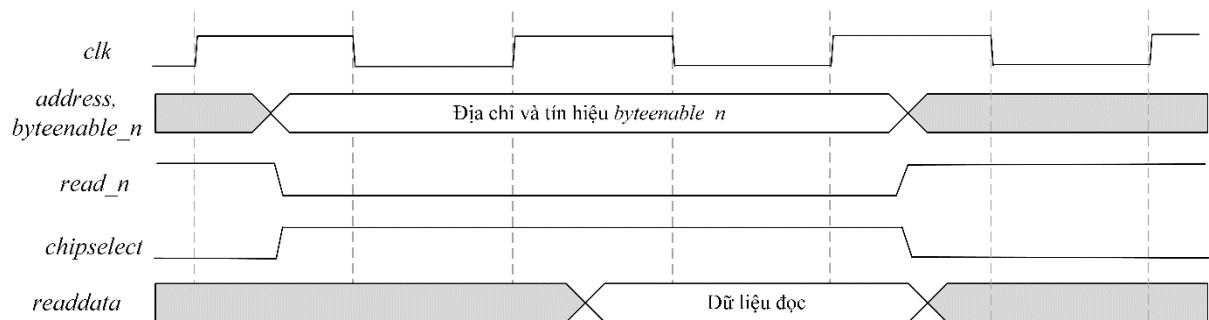
4.6.3. Chi tiết tín hiệu của Avalon bus trong việc ghi đọc dữ liệu

Trong phần này, chúng ta sẽ phân tích chi tiết các tín hiệu của Avalon Bus liên quan đến quá trình đọc dữ liệu. Avalon Bus hỗ trợ giao tiếp giữa bộ xử lý Nios II và phần cứng IP tự thiết kế, với các tín hiệu được điều phối qua các chu kỳ đồng bộ để đảm bảo dữ liệu được truyền tải chính xác.



Hình 4.32: Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu cơ bản của Avalon Bus.

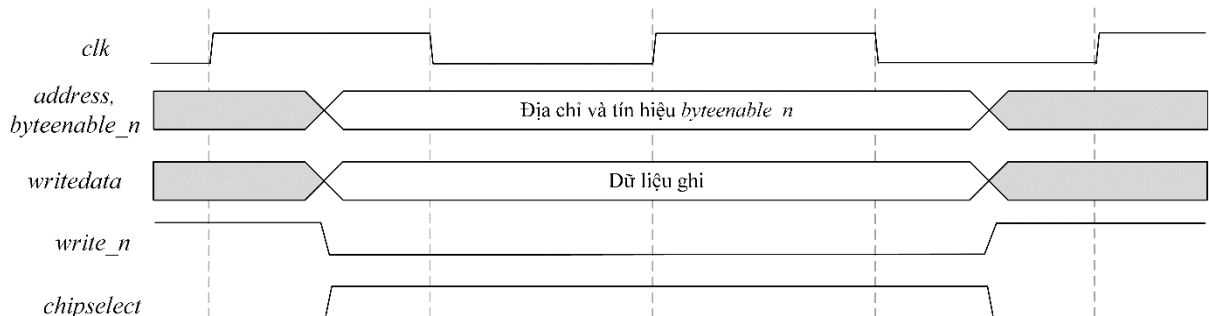
Trong quá trình đọc dữ liệu cơ bản với không có trạng thái chờ, như được mô tả trong Hình 4.32, bộ xử lý Nios II (chủ) yêu cầu dữ liệu từ phần cứng IP tự thiết kế (tớ) qua bus Avalon. Tín hiệu *address* xác định địa chỉ bộ nhớ hoặc thiết bị ngoại vi cần đọc dữ liệu, và khi tín hiệu *read* được kích hoạt, tớ nhận biết yêu cầu và chuẩn bị dữ liệu để trả về. Quá trình này không yêu cầu thời gian chờ, vì dữ liệu phải được trình bày ngay lập tức khi địa chỉ được chọn hoặc thay đổi. Tín hiệu *readdata* của tớ phải có giá trị hợp lệ và ổn định ngay trong chu kỳ xung clock tiếp theo để dữ liệu có thể được truyền tải chính xác từ tớ về chủ. Điều này đảm bảo quá trình đọc dữ liệu diễn ra ngay lập tức mà không có sự chậm trễ. Nếu dữ liệu không được truyền kịp thời, giao dịch có thể không thành công và bus Avalon sẽ không thu thập được dữ liệu chính xác từ tớ. Tuy nhiên, đối với các tớ đồng bộ, tức là các thiết bị có cổng nhập hoặc xuất đã được đăng ký, không thể sử dụng kiểu chuyển giao này vì chúng yêu cầu ít nhất một chu kỳ xung clock để lưu trữ và truyền tải dữ liệu hay đọc dữ liệu với trạng thái chờ cố định.



Hình 4.33: Sơ đồ thời gian chi tiết của tín hiệu chính cho đọc dữ liệu với một trạng thái chờ cố định của Avalon Bus.

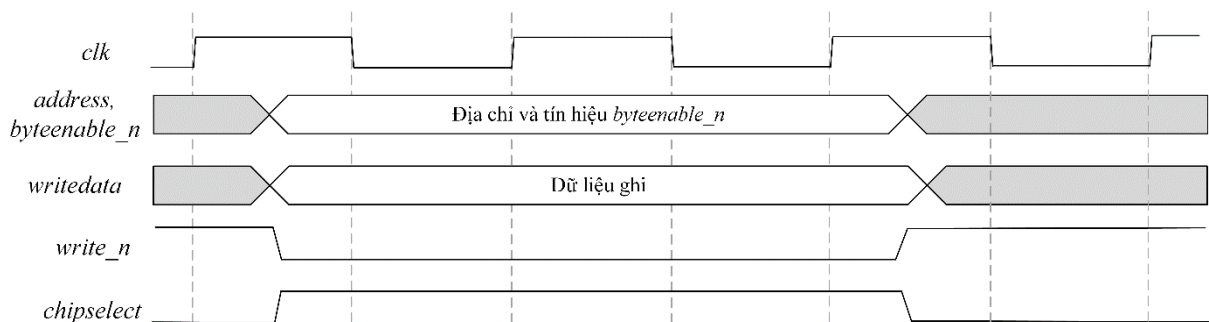
Trong quá trình đọc dữ liệu với trạng thái chờ cố định, như được mô tả trong Hình 4.33, bus Avalon sẽ cung cấp địa chỉ và tín hiệu điều khiển trong chu kỳ xung clock đầu tiên, nhưng sẽ phải chờ thêm một chu kỳ xung clock trước khi nhận *readdata* từ tớ. Điều này cho phép tớ có đủ thời gian để chuẩn bị dữ liệu. Các tín hiệu như *address*,

byteenable_n, và *chipselect* vẫn được truyền tải từ chủ đến tớ, nhưng tín hiệu *readdata* chỉ được truyền vào chu kỳ xung clock thứ hai và thu nhận bởi bus Avalon tại cạnh lên xung clock thứ ba, hoàn thành quá trình chuyển giao dữ liệu. Sự khác biệt với giao dịch đọc cơ bản là ở thời gian chờ, cho phép tớ đồng bộ chuẩn bị dữ liệu một cách hợp lý.



Hình 4.34: Sơ đồ thời gian chi tiết của tín hiệu chính cho ghi dữ liệu cơ bản của Avalon Bus.

Tương tự với quá trình đọc, trong quá trình ghi dữ liệu cơ bản với không có trạng thái chờ, như được mô tả trong Hình 4.34, chủ sẽ gửi dữ liệu đến tớ qua bus Avalon. Tín hiệu *address* xác định địa chỉ bộ nhớ hoặc thiết bị ngoại vi cần ghi dữ liệu, và khi tín hiệu *write* được kích hoạt, tớ sẽ nhận yêu cầu ghi dữ liệu. Tín hiệu *writedata* chứa dữ liệu mà chủ muốn ghi vào tớ. Quá trình này không yêu cầu thời gian chờ, vì dữ liệu phải được ghi ngay lập tức khi địa chỉ được chọn hoặc thay đổi. Tín hiệu *chipselect* giúp chọn tớ để thực hiện giao dịch ghi. Nếu tín hiệu *writedata* không được truyền kịp thời, giao dịch có thể không thành công và bus Avalon sẽ không ghi dữ liệu chính xác vào tớ.



Hình 4.35: Sơ đồ thời gian chi tiết của tín hiệu chính cho ghi dữ liệu với một trạng thái chờ cố định của Avalon Bus.

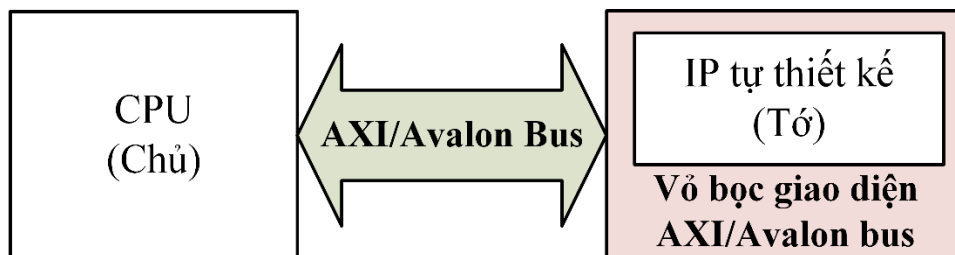
Trong quá trình ghi dữ liệu với trạng thái chờ cố định, như được mô tả trong Hình 4.35, bus Avalon sẽ cung cấp địa chỉ và tín hiệu điều khiển trong chu kỳ xung clock đầu tiên, nhưng sẽ phải chờ thêm một chu kỳ xung clock trước khi nhận *writedata* từ chủ. Điều này cho phép tớ có đủ thời gian để chuẩn bị ghi dữ liệu. Các tín hiệu như *address*, *byteenable_n*, và *chipselect* vẫn được truyền tải từ chủ đến tớ, nhưng tín hiệu *writedata*

chỉ được truyền vào chu kỳ xung clock thứ hai và thu nhận bởi bus Avalon tại cạnh lên xung clock thứ ba, hoàn thành quá trình ghi dữ liệu.

Nhìn chung, Avalon Bus cung cấp một cơ chế đồng bộ đơn giản và hiệu quả để truyền tải dữ liệu giữa bộ xử lý Nios II và phần cứng IP tự thiết kế, hỗ trợ cả giao dịch đọc và ghi với các tính năng như không có trạng thái chờ và có trạng thái chờ cố định. Các tín hiệu trong Avalon Bus, như *address*, *read*, *write*, và *writedata*, điều phối việc truyền dữ liệu qua các chu kỳ xung clock để đảm bảo hiệu quả và đồng bộ trong giao tiếp giữa chủ và tớ

4.7. Kết nối IP tự thiết kế vào hệ thống bus

Sau khi chúng ta cùng tìm hiểu chi tiết về các tín hiệu của AXI/Avalon bus trong việc ghi và đọc dữ liệu, phần này sẽ giải thích việc kết nối các IP tự thiết kế vào hệ thống bus và sử dụng các tín hiệu AXI/Avalon để giao tiếp với các thành phần khác. Đây là một phần quan trọng trong việc phát triển các hệ thống nhúng, giúp các IP tự thiết kế giao tiếp hiệu quả với các thành phần khác trong hệ thống như CPU và bộ nhớ.



Hình 4.36: Minh họa hệ thống đơn giản kết nối giữa CPU và IP tự thiết kế thông qua AXI/Avalon Bus, với IP tự thiết kế đóng vai trò Tớ kết nối vào hệ thống AXI/Avalon bus.

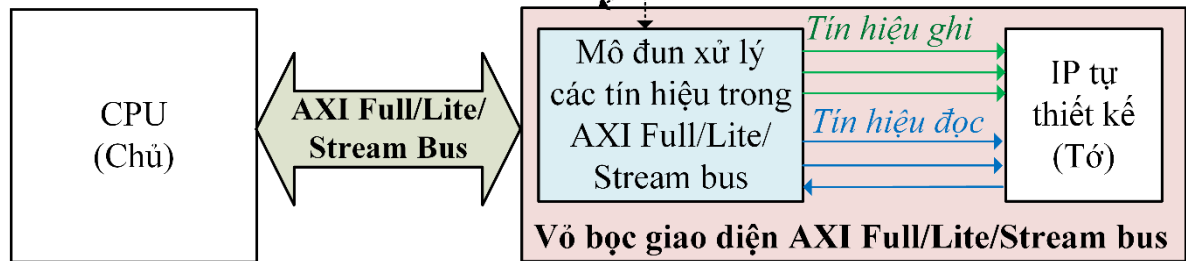
Hình 4.36 minh họa một hệ thống đơn giản, trong đó CPU (Chủ) và IP tự thiết kế (Tớ) kết nối với nhau thông qua AXI/Avalon Bus. Trong đó, IP tự thiết kế đóng vai trò là Tớ và sẽ giao tiếp với CPU qua hệ thống bus AXI hoặc Avalon. Phần tiếp theo sẽ đi vào chi tiết cách thức thiết kế và cấu hình giao diện AXI và Avalon để kết nối IP tự thiết kế vào hệ thống, bao gồm các bước trong việc xác định địa chỉ, xử lý tín hiệu điều khiển, và dữ liệu.

4.7.1. Kết nối IP tự thiết kế vào hệ thống AXI bus

Hệ thống AXI bus hỗ trợ ba chuẩn khác nhau: AXI Full, AXI Lite và AXI Stream, mỗi chuẩn có cách thức hoạt động và số lượng tín hiệu khác nhau. Tuy nhiên, tất cả chúng đều có điểm chung là sử dụng vỏ bọc và mô-đun xử lý các tín hiệu trong AXI

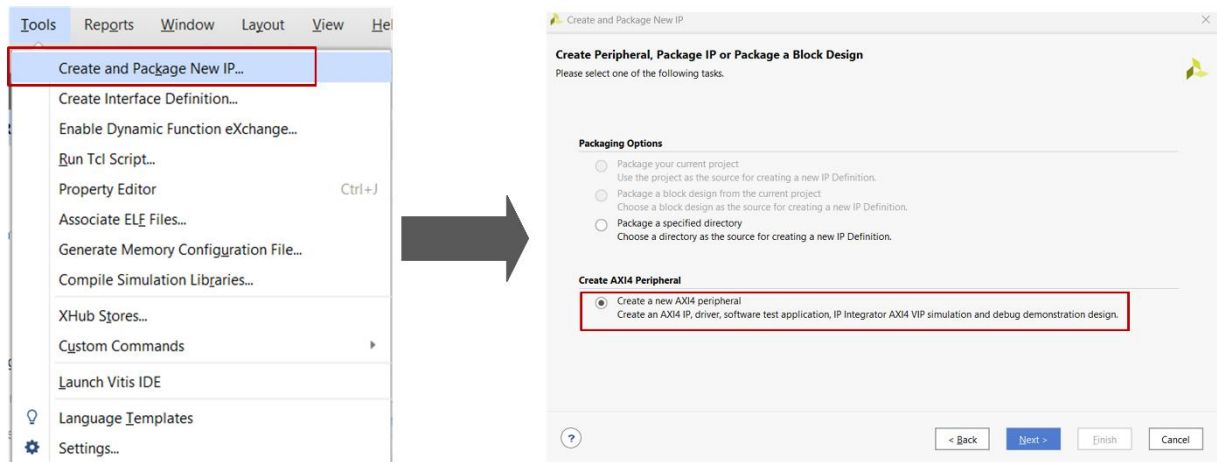
bus. Hình 4.37 minh họa sự kết nối giữa CPU và IP tự thiết kế qua AXI Full/Lite/Stream Bus, trong đó mô-đun xử lý tín hiệu sẽ đảm nhận việc truyền tải các tín hiệu ghi và đọc.

Được tạo ra từ các mẫu có sẵn trong phần mềm Xilinx Vivado



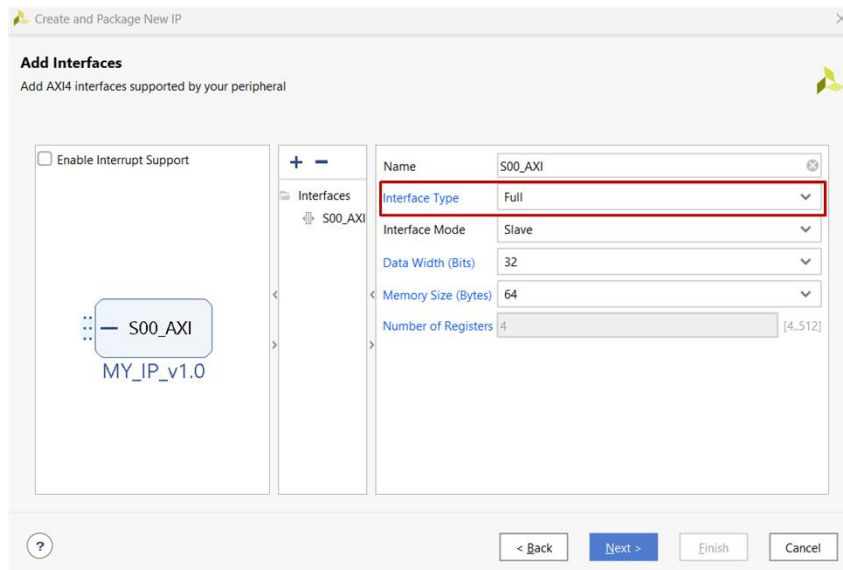
Hình 4.37: Minh họa hệ thống kết nối giữa CPU và IP tự thiết kế qua AXI Full/Lite/Stream Bus.

Người dùng có thể tự thiết kế mô-đun xử lý tín hiệu trong AXI bus để đáp ứng các yêu cầu cụ thể của hệ thống. Tuy nhiên, việc tự thiết kế mô-đun này là khá phức tạp. Do đó, phần này sẽ hướng dẫn người dùng sử dụng các mẫu có sẵn trong phần mềm Xilinx Vivado, giúp việc kết nối trở nên dễ dàng hơn. Các IP tự thiết kế sẽ chỉ sử dụng một vài tín hiệu cơ bản trong mô-đun xử lý tín hiệu được tạo ra từ các mẫu có sẵn, giúp CPU có thể ghi và đọc dữ liệu vào IP tự thiết kế thông qua hệ thống bus AXI.



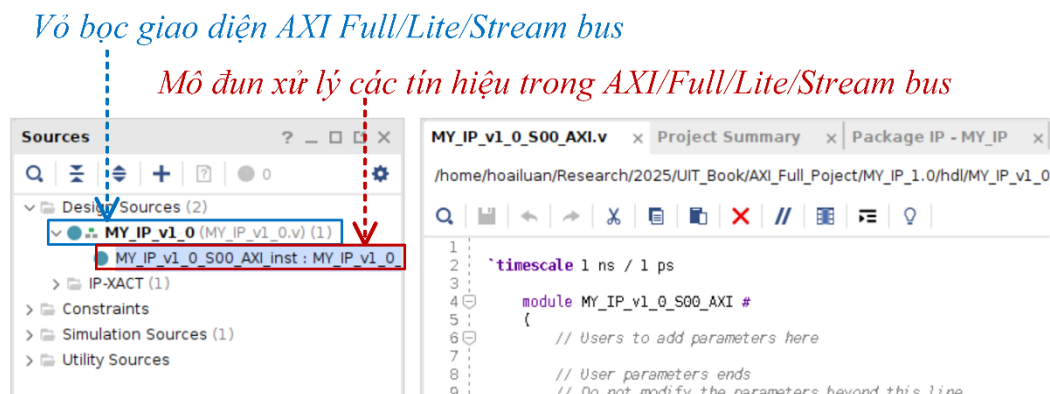
Hình 4.38: Bước tạo một vỏ bọc giao diện AXI bus từ mẫu có sẵn cho IP tự thiết kế trong phần mềm Xilinx Vivado.

Bước tiếp theo là chọn Tools > Create and Package New IP từ thanh menu chính của phần mềm Vivado. Sau khi chọn tùy chọn này, cửa sổ Create Peripheral, Package IP or Package a Block Design sẽ xuất hiện, cho phép bạn chọn loại tác vụ muốn thực hiện. Trong trường hợp này, bạn sẽ chọn Create AXI Peripheral để tạo một vỏ bọc giao diện AXI cho IP tự thiết kế, như mô tả ở Hình 4.38. Đây là bước quan trọng để dễ dàng tích hợp IP vào hệ thống sử dụng AXI bus mà không cần thiết kế lại mô-đun xử lý tín hiệu từ đầu.



Hình 4.39: Chọn loại giao diện (Interface Type) khi tạo vỏ bọc AXI, với các tùy chọn AXI Full hoặc AXI Lite để xác định ba chuẩn khác nhau của giao diện AXI.

Trong phần Interface Type, bạn có thể chọn một trong ba loại giao diện AXI: AXI Full, AXI Lite, hoặc AXI Stream, như mô tả ở Hình 4.39. Sau khi chọn loại giao diện phù hợp, bạn sẽ tiếp tục cấu hình các tham số khác như Interface Mode (Tớ hoặc Chủ), Data Width, Memory Size, và Number of Registers để phù hợp với yêu cầu thiết kế của bạn.



Hình 4.40: Hai file code Verilog tương ứng với vỏ bọc giao diện AXI Full/Lite/Stream và mô đun xử lý các tín hiệu trong AXI Full/Lite/Stream bus được tạo ra sau bước chọn giao diện trong phần mềm Xilinx Vivado.

Sau bước chọn giao diện trong phần mềm Xilinx Vivado, hai file code Verilog tương ứng với vỏ bọc giao diện AXI Full/Lite/Stream bus và mô đun xử lý các tín hiệu trong AXI Full/Lite/Stream bus được tạo ra như Hình 4.40. Để làm rõ cách IP tự thiết kế lấy các tín hiệu điều khiển đọc ghi từ mô đun xử lý các tín hiệu AXI Full/Lite/Stream bus, chúng tôi phân tích code Verilog cho từng loại giao diện. Trong đó, IP tự thiết kế

thực hiện phép tính $y = a * x + b$, sử dụng các tín hiệu từ mô đun xử lý để ghi và đọc dữ liệu từ các thanh ghi a, b, x, và y, qua đó thực hiện phép toán này.

```

reg [31:0] y, a, b, x;

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 ) begin
        a <= 0;
        b <= 0;
        x <= 0;
    end
    else begin
        if(axi_awv_awr_flag && (axi_awaddr == 32'hA0000000))
            a <= S_AXI_WDATA;
        else if(axi_awv_awr_flag && (axi_awaddr == 32'hA0000004))
            b <= S_AXI_WDATA;
        else if(axi_awv_awr_flag && (axi_awaddr == 32'hA0000008))
            x <= S_AXI_WDATA;
    end
end

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 )
        y <= 0;
    else
        y <= a*x + b;
end

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 )
        axi_rdata <= 0;
    end
    else begin
        if (axi_arv_arrr_flag && (axi_araddr == 32'hA0000000))
            axi_rdata <= y;
        else
            axi_rdata <= axi_rdata;
        end
    end
end

```

Xử lý ghi dữ liệu vào biến a b x

*Tính toán chính của IP tự thiết kế ($y = a*x + b$)*

Xử lý đọc dữ liệu của biến y

Hình 4.41: Code Verilog mô tả IP tự thiết kế sử dụng 6 tín hiệu từ mô đun xử lý các tín hiệu AXI Full bus để ghi, đọc dữ liệu vào các biến và thực hiện tính toán $y = a*x + b$.

Trong Hình 4.41, mã Verilog mô tả cách IP tự thiết kế sử dụng mô đun xử lý các tín hiệu AXI Full bus để thực hiện các thao tác ghi và đọc dữ liệu vào các biến, đồng thời thực hiện tính toán $y = a * x + b$. Cụ thể, mô đun này sử dụng 6 tín hiệu từ AXI Full bus để xử lý các yêu cầu ghi và đọc dữ liệu. Tín hiệu *axi_awv_awr_flag* cho biết có yêu cầu ghi từ chủ (CPU) đến tớ (IP tự thiết kế). Tín hiệu *axi_awaddr* chỉ ra địa chỉ của thanh ghi mà dữ liệu sẽ được ghi vào. Trong ví dụ trên, các địa chỉ như 32'hA0000000, 32'hA0000004, và 32'hA0000008 được sử dụng để lưu trữ các giá trị vào các thanh ghi a, b, và x. Tín hiệu *S_AXI_WDATA* mang theo dữ liệu ghi từ chủ đến tớ. Khi cần đọc dữ liệu, tín hiệu *axi_arv_arrr_flag* được kích hoạt để xác nhận yêu cầu đọc, và *axi_araddr* chỉ ra địa chỉ của thanh ghi mà chủ muốn đọc dữ liệu, ở đây là địa chỉ cho thanh ghi y là 32'hA0000000. Cuối cùng, tín hiệu *axi_rdata* mang dữ liệu trả về từ tớ về chủ. Các

tín hiệu này giúp thực hiện các thao tác ghi và đọc dữ liệu vào các thanh ghi a, b, x, y, và thực hiện tính toán ở IP tự thiết kế.

```

reg [31:0] y, a, b, x;

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 ) begin
        a <= 0;
        b <= 0;
        x <= 0;
    end
    else begin
        if(slv_reg_wren && (axi_awaddr == 32'hA0000000))
            a <= S_AXI_WDATA;
        else if(slv_reg_wren && (axi_awaddr == 32'hA0000004))
            b <= S_AXI_WDATA;
        else if(slv_reg_wren && (axi_awaddr == 32'hA0000008))
            x <= S_AXI_WDATA;
    end
end
end

```

Xử lý ghi dữ liệu vào biến a b x

```

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 )
        y <= 0;
    else
        y <= a*x + b;
end

```

*Tính toán chính của IP tự thiết kế ($y = a * x + b$)*

```

always @(posedge S_AXI_ACLK) begin
    if ( S_AXI_ARESETN == 1'b0 ) begin
        axi_rdata <= 0;
    end
    else begin
        if (slv_reg_rden && (axi_araddr == 32'hA0000000))
            axi_rdata <= y;
        else
            axi_rdata <= axi_rdata;
    end
end
end

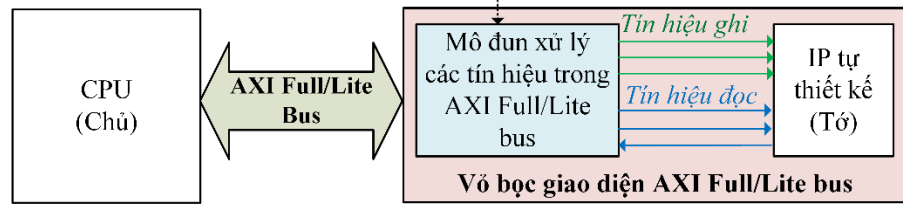
```

Xử lý đọc dữ liệu của biến y

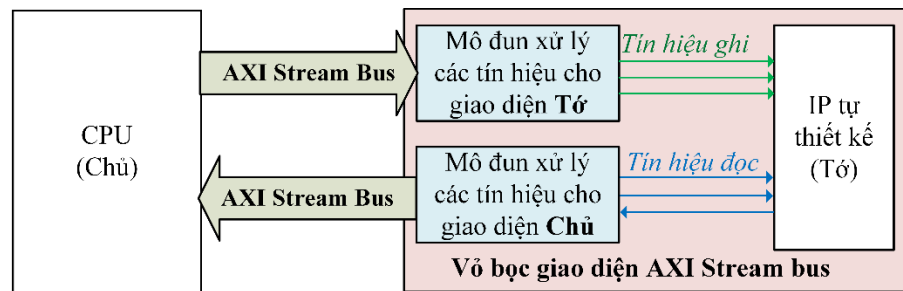
Hình 4.42: Code Verilog mô tả IP tự thiết kế sử dụng 6 tín hiệu từ mô đun xử lý các tín hiệu AXI Lite bus để ghi, đọc dữ liệu vào các biến và thực hiện tính toán $y = a * x + b$.

Hình 4.42 mô tả mã Verilog cho IP tự thiết kế sử dụng mô đun xử lý các tín hiệu từ AXI Lite bus để ghi, đọc dữ liệu vào các biến và thực hiện tính toán $y = a * x + b$. Mô đun này sử dụng các tín hiệu như *slv_reg_wren* để xác nhận yêu cầu ghi dữ liệu từ chủ (CPU) đến tớ (IP tự thiết kế), tín hiệu *axi_awaddr* chỉ ra địa chỉ của thanh ghi để ghi dữ liệu vào (các địa chỉ như 32'hA0000000, 32'hA0000004 và 32'hA0000008 cho các thanh ghi a, b, và x), và tín hiệu *S_AXI_WDATA* mang dữ liệu ghi từ chủ đến tớ. Để thực hiện đọc dữ liệu, tín hiệu *slv_reg_rden* xác nhận yêu cầu đọc từ chủ, *axi_araddr* chỉ ra địa chỉ thanh ghi mà chủ muốn đọc dữ liệu từ (ví dụ, thanh ghi y ở địa chỉ 32'hA0000000), và tín hiệu *axi_rdata* mang dữ liệu trả về từ tớ về chủ. Các tín hiệu này giúp thực hiện thao tác ghi và đọc dữ liệu vào các thanh ghi a, b, x, y, đồng thời thực hiện phép toán $y = a * x + b$ ở IP tự thiết kế.

Chế độ giao diện (Interface mode) chỉ cần thiết lập là Slave, vì nó đã bao gồm cả tín hiệu đọc và ghi.

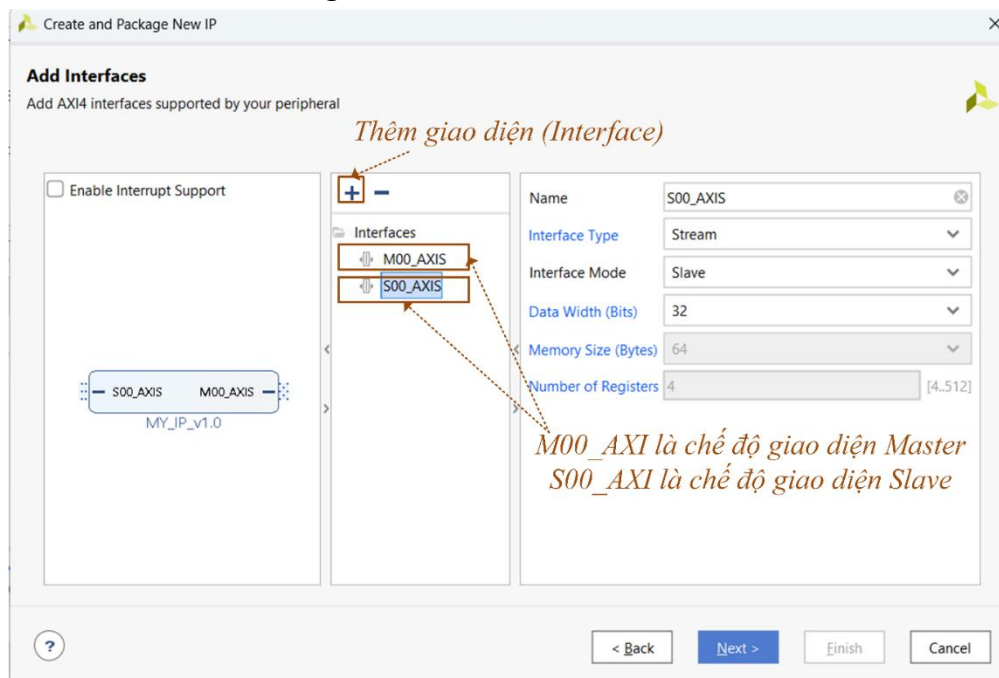


(a)



(b)

Hình 4.43: Khác biệt cấu hình vỏ bọc giao diện AXI Full/Lite và AXI Stream: (a) Vỏ bọc giao diện AXI Full/Lite với chế độ Tớ hỗ trợ cả tín hiệu đọc và ghi; (b) Vỏ bọc giao diện AXI Stream với chế độ Tớ chỉ hỗ trợ tín hiệu ghi và chế độ Chủ chỉ hỗ trợ tín hiệu đọc.



Hình 4.44: Thêm giao diện AXI Stream cho IP tự thiết kế, với M00_AXIS là chế độ giao diện Chủ và S00_AXIS là chế độ giao diện Tớ.

Lưu ý là có sự khác biệt giữa giao diện AXI Full/Lite và AXI Stream ở cấu hình vỏ bọc giao diện và chế độ giao diện, như mô tả trong Hình 4.43. Đối với AXI Full/Lite, khi chế độ giao diện được thiết lập là Tớ, nó có thể hỗ trợ cả tín hiệu đọc và ghi, như mô tả trong Hình 4.43 (a). Trong khi đó, với AXI Stream, chế độ giao diện Tớ chỉ hỗ

trợ tín hiệu ghi, và chế độ giao diện Chủ chỉ hỗ trợ tín hiệu đọc, như mô tả trong Hình 4.43 (b). Vì vậy, cấu hình vỏ bọc và mô-đun xử lý tín hiệu trong AXI Stream sẽ có sự khác biệt so với AXI Full/Lite, do yêu cầu về tín hiệu đọc và ghi được phân tách giữa Tớ và Chủ trong hệ thống AXI Stream.

Để thêm giao diện IP thiết kế có cả giao diện Chủ và Tớ, cần thêm hai giao diện AXI Stream cho IP tự thiết kế như mô tả trong Hình 4.44. Trong đó, M00_AXIS được cấu hình là chế độ giao diện Chủ, và S00_AXIS được cấu hình là chế độ giao diện Tớ. Đây là sự khác biệt quan trọng khi làm việc với AXI Stream, vì chế độ giao diện Tớ chỉ hỗ trợ tín hiệu ghi, trong khi chế độ giao diện Chủ chỉ hỗ trợ tín hiệu đọc. Chế độ giao diện (Interface mode) cần phải chọn Stream cho cả hai giao diện Chủ và Tớ để đảm bảo hoạt động chính xác trong hệ thống AXI Stream.

```

reg [31:0] y, a, b, x;

always @(posedge S_AXI_ACLK) begin
    if ( S_AXIS_ARESETN == 1'b0 ) begin
        a <= 0;
        b <= 0;
        x <= 0;
    end
    else begin
        if(fifo_wren && (write_pointer == 0))
            a <= S_AXIS_TDATA;
        else if(fifo_wren && (write_pointer == 1))
            b <= S_AXIS_TDATA;
        else if(fifo_wren && (write_pointer == 2))
            x <= S_AXIS_TDATA;
    end
end

always @(posedge S_AXI_ACLK) begin
    if ( M_AXIS_ARESETN == 1'b0 )
        y <= 0;
    else
        y <= a*x + b;
end

always @(posedge S_AXI_ACLK) begin
    if ( M_AXIS_ARESETN == 1'b0 ) begin
        stream_data_out <= 0;
    end
    else begin
        if (tx_en)
            stream_data_out <= y;
    end
end

```

Xử lý ghi dữ liệu vào biến a b x

*Tính toán chính của IP tự thiết kế ($y = a*x + b$)*

Xử lý đọc dữ liệu của biến y

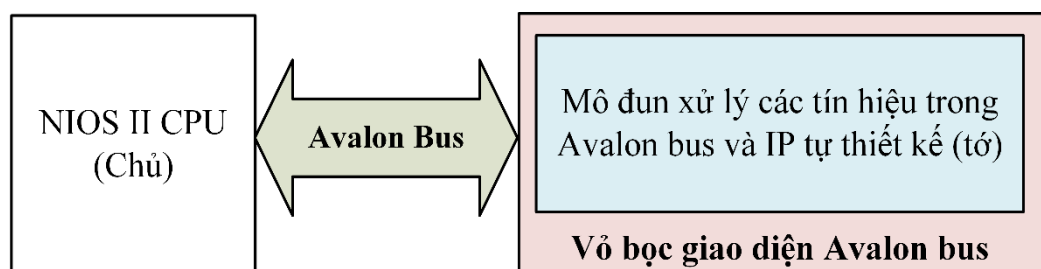
Hình 4.45: Code Verilog mô tả IP tự thiết kế sử dụng 5 tín hiệu từ mô-đun xử lý các tín hiệu AXI Stream bus của giao diện Chủ và Tớ để ghi, đọc dữ liệu vào các biến và thực hiện tính toán $y = a*x + b$.

Trong Hình 4.45, mã Verilog mô tả IP tự thiết kế sử dụng 6 tín hiệu từ mô đun xử lý các tín hiệu AXI Stream của giao diện Chủ và Tớ để ghi, đọc dữ liệu vào các biến và thực hiện tính toán $y = a * x + b$. Vì không có địa chỉ trong AXI Stream, tín hiệu `write_pointer` (từ giao diện tớ) được sử dụng để đếm số lượng input vào và xác định việc ghi dữ liệu vào các thanh ghi `a`, `b`, và `x`. Tín hiệu `fifo_wren` (từ giao diện tớ) kiểm tra và điều khiển quá trình ghi dữ liệu vào các thanh ghi này. Tín hiệu `M_AXIS_ARESETN` (từ giao diện chủ) và `S_AXIS_ARESETN` (từ giao diện tớ) được sử dụng để thiết lập lại các giá trị khi có reset. Phép toán chính của IP tự thiết kế được thực hiện trong phần code thứ hai, nơi giá trị của `y` được tính toán từ `a` và `x`, rồi cộng với `b`. Cuối cùng, tín hiệu `stream_data_out` (từ giao diện chủ) đảm bảo việc truyền dữ liệu từ IP ra ngoài khi có yêu cầu đọc, với tín hiệu `tx_en` (từ giao diện chủ) điều khiển việc xuất dữ liệu ra ngoài hệ thống. Các tín hiệu này giúp thực hiện các thao tác ghi, đọc và tính toán tại IP tự thiết kế trong môi trường AXI Stream.

Nhìn chung, khi kết nối IP tự thiết kế vào hệ thống AXI bus, do tính phức tạp của AXI, người dùng thường sử dụng các vỏ bọc giao diện theo mẫu có sẵn do Xilinx cung cấp, kết hợp với việc sử dụng 5 hoặc 6 tín hiệu cần thiết để quản lý các thao tác ghi và đọc dữ liệu một cách đơn giản. Đây là giải pháp phù hợp cho các hệ thống đơn giản hoặc yêu cầu tích hợp nhanh chóng. Tuy nhiên, đối với các hệ thống phức tạp, người dùng có thể cần phải tự thiết kế toàn bộ quá trình xử lý các tín hiệu trong AXI bus để đáp ứng yêu cầu cụ thể của ứng dụng.

4.7.2. Kết nối IP tự thiết kế vào hệ thống Avalon bus

Trong phần này, chúng ta sẽ tìm hiểu về cách kết nối IP tự thiết kế vào hệ thống Avalon bus. Avalon bus là một giao thức bus đơn giản, yêu cầu chỉ một vài tín hiệu để kết nối các thành phần trong hệ thống, điều này làm cho việc triển khai trở nên dễ dàng hơn so với các giao thức phức tạp hơn như AXI bus.



Hình 4.46: Minh họa hệ thống kết nối giữa CPU và IP tự thiết kế qua Avalon Bus.

Như mô tả trong Hình 4.46, kết nối giữa CPU và IP tự thiết kế qua Avalon bus được thực hiện thông qua mô đun xử lý các tín hiệu giao tiếp trong Avalon bus. Vì sự đơn giản của Avalon bus, phép toán tính toán trong IP tự thiết kế (như phép tính $y = a * x + b$) sẽ được gộp chung vào xử lý tín hiệu của Avalon bus, giúp giảm thiểu độ phức tạp trong thiết kế và triển khai hệ thống.

```

module Compute(
    input iClk,
    input iReset_n,
    input iChipSelect_n,
    input iWrite_n,
    input iRead_n,
    input [1:0] iAddress,
    input [31:0] iData,
    output reg [31:0] oData
);
    reg [31:0] a, b, x;

    always @(posedge iClk or negedge iReset_n) begin
        if (~iReset_n) begin
            oData <= 32'd0;
            a <= 32'd0;
            b <= 32'd0;
            x <= 32'd0;
        end else begin
            if (~iChipSelect_n & ~iWrite_n) begin
                case (iAddress)
                    2'd0: a <= iData[3:0];
                    2'd1: b <= iData[3:0];
                    2'd2: x <= iData[3:0];
                endcase
            end
            if (~iChipSelect_n & ~iRead_n) begin
                case (iAddress)
                    2'd0: oData <= a;
                    2'd1: oData <= b;
                    2'd2: oData <= x;
                    2'd3: oData <= a * x + b;
                endcase
            end
        end
    end
endmodule

```

Xử lý ghi dữ liệu vào biến a b x

*Kết hợp xử lý đọc dữ liệu và tính toán chính của IP tự thiết kế ($y=a*x+b$)*

Hình 4.47: Code Verilog mô tả IP tự thiết kế kết hợp với mô-đun xử lý các tín hiệu Avalon bus để ghi, đọc dữ liệu vào các biến và thực hiện phép tính $y = a * x + b$.

Trong Hình 4.47, mã Verilog mô tả cách IP tự thiết kế kết hợp với mô-đun xử lý các tín hiệu Avalon bus để ghi, đọc dữ liệu vào các biến và thực hiện phép tính $y = a * x + b$. Mô đun này sử dụng các tín hiệu như iChipSelect_n, iWrite_n, và iRead_n để xác nhận các yêu cầu ghi và đọc dữ liệu giữa CPU (Chủ) và IP tự thiết kế (Tớ). Khi chế độ giao diện là ghi (tín hiệu iWrite_n), các giá trị từ iData sẽ được ghi vào các thanh ghi a,

b, và x dựa trên địa chỉ được chỉ ra bởi iAddress. Ngược lại, khi chế độ giao diện là đọc (tín hiệu iRead_n), giá trị của các thanh ghi a, b, x, hoặc phép toán $y = a * x + b$ sẽ được trả về thông qua oData. Tín hiệu iChipSelect_n được sử dụng để chọn lựa việc thực hiện ghi hoặc đọc, và iAddress chỉ ra địa chỉ của thanh ghi mà dữ liệu sẽ được ghi vào hoặc từ đó sẽ được đọc. Qua đó, các tín hiệu này giúp thực hiện các thao tác ghi, đọc và phép toán ở IP tự thiết kế trong môi trường Avalon bus, làm cho việc kết nối và xử lý trở nên mượt mà và dễ dàng hơn.

File Templates Beta View

Name	Interface	Signal Type	Width	Direction
iClk	clock_sink	clk	1	input
iReset_n	reset_sink	reset_n	1	input
iChipSelect_n	avalon_slave_0	chipselect_n	1	input
iWrite_n	avalon_slave_0	write_n	1	input
iRead_n	avalon_slave_0	read_n	1	input
iAddress	avalon_slave_0	address	2	input
iData	avalon_slave_0	writedata	32	input
oData	avalon_slave_0	readdata	32	output

Buttons: Add Signal, Remove Signal

Hình 4.48: Cấu hình các tín hiệu của IP tự thiết kế (mô-đun Compute) trên phần mềm Quartus Prime.

Trong Hình 4.48, chúng ta có thể thấy cấu hình các tín hiệu của IP tự thiết kế (mô-đun Compute) trong phần mềm Quartus Prime. Bảng này liệt kê các tín hiệu đầu vào và đầu ra của mô-đun, bao gồm các tín hiệu như iClk (đầu vào clock), iReset_n (đầu vào reset), iChipSelect_n (đầu vào chip select), iWrite_n (đầu vào tín hiệu ghi), iRead_n (đầu vào tín hiệu đọc), iAddress (đầu vào địa chỉ), iData (đầu vào dữ liệu), và oData (đầu ra dữ liệu). Các tín hiệu này được cấu hình với các loại tín hiệu và chiều (input hoặc output) tương ứng, giúp IP tự thiết kế giao tiếp với các thành phần khác trong hệ thống, đặc biệt là với giao diện Avalon bus. Từ đây, việc cấu hình tín hiệu trong phần

mềm Quartus Prime giúp đơn giản hóa quá trình kết nối và điều khiển dữ liệu cho IP tự thiết kế trong hệ thống.

Nhìn chung, việc kết nối IP tự thiết kế vào hệ thống Avalon bus giúp đơn giản hóa quá trình giao tiếp giữa các thành phần trong hệ thống nhờ vào giao thức Avalon bus với số tín hiệu tối thiểu. Như đã trình bày, Avalon bus yêu cầu ít tín hiệu hơn so với các giao thức phức tạp như AXI bus, giúp giảm bớt độ phức tạp trong thiết kế hệ thống. Các phép toán và thao tác đọc, ghi dữ liệu trong IP tự thiết kế có thể được tích hợp trực tiếp vào xử lý tín hiệu Avalon bus, giúp việc triển khai hệ thống trở nên dễ dàng hơn.

4.8. Tóm tắt

Chương này đã trình bày chi tiết về các giao thức kết nối trong SoC, bao gồm APB, AHB, AXI4 Full, AXI4-Lite và AXI4-Stream, cùng với các tín hiệu và kết nối đi kèm. Mỗi giao thức này có những đặc điểm và ứng dụng cụ thể nhằm đáp ứng yêu cầu kết nối giữa các IP cores khác nhau trên SoC. APB được sử dụng chủ yếu cho các thiết bị ngoại vi với băng thông thấp và độ phức tạp đơn giản, trong khi AHB cung cấp một giải pháp kết nối tốc độ cao với nhiều chủ và tớ. AXI4 là giao thức linh hoạt nhất, hỗ trợ các giao dịch Burst và truyền dữ liệu song song, phù hợp cho các ứng dụng đòi hỏi hiệu suất cao và băng thông lớn. Chương này cũng phân tích chi tiết cách thức hoạt động của các tín hiệu trong các giao dịch đọc và ghi dữ liệu, bao gồm các sơ đồ thời gian cho từng loại giao thức.

Vì môn học này tập trung thực hành vào thiết kế SoC trên Xilinx FPGA, việc hiểu rõ về AXI4 là rất quan trọng. AXI4 Full, với tính năng hỗ trợ các giao dịch phức tạp và khả năng tối ưu hóa băng thông, là giao thức thường được sử dụng nhất khi thiết kế các hệ thống SoC trên nền tảng FPGA. Tuy nhiên, bên cạnh AXI4, Avalon Bus của Intel (trước đây là Altera) cũng là một giao thức phổ biến trong các hệ thống SoC, đặc biệt là trong các thiết kế FPGA của Intel. Avalon Bus nổi bật với tính đơn giản, dễ triển khai và hỗ trợ giao tiếp hiệu quả giữa các bộ xử lý và thiết bị ngoại vi. Do đó, sinh viên cần lưu ý kỹ về cách thức hoạt động, tín hiệu, và các chế độ giao dịch của cả AXI4 và Avalon Bus để có thể thiết kế và tối ưu hóa các kết nối trong các ứng dụng thực tế.

4.9. Câu hỏi và bài tập

1. Trình bày sự khác biệt giữa giao thức APB và AHB trong hệ thống SoC. Khi nào nên sử dụng APB thay vì AHB?
2. Giải thích cơ chế truyền dữ liệu trong giao thức AXI4 Full. Các kênh dữ liệu trong AXI4 Full hoạt động như thế nào để tối ưu hóa băng thông?
3. So sánh giữa AXI4 Full và AXI4-Lite. Đặc điểm nào khiến AXI4-Lite phù hợp hơn cho các ứng dụng yêu cầu kết nối đơn giản?
4. Mô tả chức năng và vai trò của các tín hiệu TVALID và TREADY trong giao thức AXI4-Stream. Tại sao chúng quan trọng đối với việc truyền dữ liệu?
5. Phân tích ưu và nhược điểm của việc sử dụng giao thức AXI4-Stream so với các giao thức khác trong AMBA khi truyền dữ liệu không cần địa chỉ.
6. Mô tả các tín hiệu chính của AXI4-Lite sử dụng trong giao dịch ghi và đọc dữ liệu. Tại sao AXI4-Lite không hỗ trợ giao dịch Burst?
7. Thiết kế một sơ đồ kết nối đơn giản sử dụng các giao thức APB, AHB, và AXI trong một hệ thống SoC. Giải thích cách các giao thức này tương tác với nhau thông qua các cầu nối (bridges).
8. Trong trường hợp sử dụng AXI4 Full, hãy mô tả quy trình giao dịch Burst từ chủ đến tớ. Những tín hiệu nào cần được chú ý để đảm bảo giao dịch diễn ra suôn sẻ?
9. Trình bày các tín hiệu chính trong giao thức Avalon Bus và giải thích vai trò của chúng trong quá trình giao tiếp giữa bộ xử lý Nios II và phần cứng IP tự thiết kế.
10. So sánh giữa Avalon Bus và AXI4 trong hệ thống SoC. Đặc điểm nào của Avalon Bus khiến nó trở thành một lựa chọn phù hợp cho các thiết kế FPGA của Intel?